nwergm

Exponential random graph models in Stata
(a package that builds on `nwcommands`)

User manual and reference

Thomas U. Grund
RWTH Aachen University

This manual describes `nwergm` 1.5.x
(examples produced with `nwergm` 1.5.9, 10 November 2026)

How to cite. This manual describes the series 1.5 of `nwergm` (it holds for all versions 1.5.x); the listings were produced with version 1.5.9 (10 November 2026). It is the citable reference for the package. Please cite it as:

Grund, T. U. (2026). *nwergm: Exponential random graph models in Stata. Manual.* Zenodo. https://doi.org/10.5281/zenodo.23661331

Please also report the exact version of the program that produced your results, as printed by `nwergm version` and stored in `e(version)`.

Acknowledgments

`nwergm` follows the statnet project closely and exists because of it. The exponential random graph model and the `ergm` package are due to Mark S. Handcock, David R. Hunter, Carter T. Butts, Steven M. Goodreau, Pavel N. Krivitsky, Martina Morris and many contributors [1, 2]. `ergm` 4.12.0 is the reference implementation on which the terms, the estimation algorithms, the defaults and the output of `nwergm` are modeled. The R source code and the manuals of `ergm` were the specification against which the Stata code was written and checked. The curved exponential families, the pseudolikelihood and the goodness-of-fit method of the package are those of Hunter and Handcock [3] and Hunter, Goodreau and Handcock [4]; the treatment of missing dyads is that of Handcock and Gile [5].

`nwergm` is not affiliated with or endorsed by the statnet project, and its errors are the author's. It is meant for researchers who work in Stata or prefer it to R. It is not a replacement for `ergm`, which remains the reference and the more complete program.

Use of an AI coding assistant. The software was developed with Claude Code (Anthropic), an AI coding assistant, working under the author's direction. It helped port algorithms from ergm into Stata, Mata and C and helped write the tests. The author specified the scope, reviewed the code, and is responsible for the package. Its correctness rests on validation, not on the assistant: the results are compared numerically with ergm, and the test suite reruns these comparisons.

Contents

Acknowledgments **2**

1 Introduction **6**
1.1 What `nwergm` is . . . 6
1.2 Relation to `ergm` . . . 6
1.3 What is covered . . . 7
1.4 Installation and requirements . . . 8
1.5 Dialog boxes . . . 9
1.6 License and how to cite . . . 9
1.7 How to read this manual . . . 10

2 The model and the estimators **11**
2.1 The exponential random graph model . . . 11
2.2 Estimators . . . 12
2.2.1 Maximum pseudolikelihood . . . 12
2.2.2 Monte Carlo maximum likelihood . . . 12
2.2.3 Log-likelihood, AIC and BIC . . . 15
2.2.4 Contrastive divergence . . . 15
2.2.5 Stochastic approximation . . . 15
2.2.6 Starting values and offsets . . . 15
2.3 Reproducibility . . . 16
2.4 Missing dyads . . . 16
2.5 Constraints and observation processes . . . 17
2.6 Curved terms . . . 18

3 Getting started **19**
3.1 The data . . . 19
3.2 A first model: a dyad-independent model . . . 20
3.3 A dyad-dependent model . . . 21
3.4 How coefficients are named . . . 22

4 Syntax and options **23**
4.1 Syntax . . . 23
4.2 Terms . . . 23
4.2.1 Node and dyadic covariates, and reciprocity . . . 23
4.2.2 Geometrically weighted terms . . . 24
4.2.3 Degree, star, triad and shared-partner terms . . . 24
4.2.4 Two-mode terms . . . 25
4.2.5 Further terms, operators and labels . . . 26
4.2.6 Level selection . . . 26
4.3 Estimation options . . . 26
4.3.1 Method . . . 26

4.3.2 Monte Carlo maximum likelihood, contrastive divergence, stochastic approximation . . . 27
4.3.3 The Markov chain, reproducibility and performance . . . 27
4.3.4 Log-likelihood by bridge sampling . . . 28
4.3.5 Constraints . . . 28
4.4 `nwergm simulate` . . . 29
4.4.1 From given coefficients . . . 29
4.4.2 From the last fit . . . 29
4.5 Postestimation options . . . 30

5 The terms **31**
5.1 Counts and how to read the tables . . . 31
5.2 Dyad-level and basic terms . . . 32
5.3 Degree and star terms . . . 32
5.4 Triad, cycle, path and shared-partner terms . . . 34
5.5 Curved terms: geometrically weighted and alternating statistics . . . 35
5.6 Covariate terms . . . 36
5.7 Two-mode (bipartite) terms . . . 38
5.8 Operators . . . 40
5.9 Terms that are not implemented . . . 41
5.10 Terms of the packages built on ergm . . . 42

6 Structural terms, operators, constraints and simulation **44**
6.1 Structural terms: `structural()` . . . 44
6.2 Operators: `operators()` . . . 45
6.3 Offsets: fixed coefficients . . . 46
6.4 Constraints and observation processes . . . 46
6.5 Simulation . . . 47

7 Diagnostics, goodness of fit and postestimation **48**
7.1 Diagnosing the Markov chain: `estat mcmcdiag` . . . 48
7.2 Goodness of fit: `estat gof` . . . 49
7.3 Information criteria and likelihood-ratio tests . . . 51
7.4 Prediction: `predict` . . . 52
7.5 Stored results . . . 52

8 Worked examples **54**
8.1 The Florentine families: estimation methods and stored results . . . 54
8.1.1 The same model by different estimators . . . 54
8.1.2 Reproducibility with `seed()` . . . 58
8.1.3 Stored results . . . 58
8.2 Mesa High School: dyad-independent models . . . 60
8.3 The Lazega lawyers: a directed network, diagnostics and goodness of fit . . . 64
8.3.1 Is the chain trustworthy? . . . 65
8.3.2 Goodness of fit . . . 68
8.3.3 Simulation from the fit . . . 72
8.4 Kapferer's tailor shop: a degenerate model . . . 73
8.5 Missing dyads . . . 75
8.6 Comparing models and predicting ties . . . 77
8.7 Constraints and observation processes . . . 79
8.7.1 An observation process . . . 81
8.8 A two-mode network, structural terms, operators and labels . . . 82
8.9 Network types: a valued network and a two-mode covariate . . . 87

9 Validation against ergm **90**
9.1 How the comparison was done . . . 90
9.2 A comparison run for this manual . . . 91
9.3 Where nwergm differs from ergm . . . 91
9.4 How to reproduce . . . 93

10 Limits, known issues and messages **95**
10.1 Dialog boxes . . . 95
10.2 Known issues . . . 95
10.3 Messages and what to do . . . 96

11 Technical notes **98**
11.1 The three layers . . . 98
11.2 The native plugin . . . 98
11.2.1 Which engine ran . . . 98
11.2.2 Platforms . . . 99
11.2.3 Source code of the plugin . . . 99
11.2.4 Building the plugin on your computer . . . 99
11.2.5 Fixed limits of the plugin . . . 99
11.3 Reproducibility . . . 100
11.4 Size limits . . . 100
11.4.1 What costs what . . . 100
11.4.2 What Stata limits . . . 100
11.4.3 Models with one coefficient per node . . . 100
11.4.4 Not measured . . . 100
11.5 Performance . . . 101
11.6 File layout . . . 102

A From R's `ergm` to `nwergm` **103**

B `ergm` definitions reproduced on purpose **105**

C Comparison with ergm: the same estimator, and where the speed comes from **107**
C.1 The same algorithm . . . 107
C.2 The same simulation . . . 108
C.3 The same answers . . . 109
C.4 Where the speed comes from . . . 109
C.4.1 The amount of work . . . 110
C.4.2 Time for a fixed amount of work . . . 111
C.4.3 The sampler itself . . . 111
C.4.4 Small models . . . 112
C.4.5 What was not measured . . . 112

Chapter 1

Introduction

1.1 What `nwergm` is

`nwergm` estimates exponential random graph models (ERGMs) in Stata. An ERGM is a statistical model for a single observed network: it describes the probability of the whole network as a function of a handful of network statistics, such as the number of ties, the number of reciprocated pairs, the number of triangles, or the number of ties between nodes of the same group. The coefficients of these statistics are estimated from the observed network, and the fitted model can be used to test hypotheses about the processes that shaped the network, to simulate new networks, and to check how well it reproduces features of the data that were not part of the model. Chapter 2 summarizes the model for readers who have not met it before; Robins et al. [8] give a standard introduction, and Frank and Strauss [10] and Wasserman and Pattison [9] are the papers from which the model grew.

`nwergm` is a Stata package of its own that builds on the `nwcommands` package. It works on networks that were declared with `nwset` (or created by one of the other `nw` commands), reads the node attributes from the ordinary Stata dataset, and returns its results in `e()` like any other Stata estimation command, so that `estimates store`, `lrtest`, `test`, `lincom` and `predict` work as usual. The sampling that a dyad-dependent model needs runs in a compiled C plugin that is called from Mata. No R installation or other external statistical software is used when you fit a model.

1.2 Relation to `ergm`

The reference implementation of ERGMs is the R package `ergm` [1], part of the statnet suite [2]. `nwergm` follows `ergm` closely in its model, in the names and definitions of its terms, in its estimation algorithms and in the conventions that matter for the numbers, so that the documentation of `ergm` and the published literature on ERGMs apply to `nwergm` as well. Coefficient names are those of `ergm` with a period written as an underscore.

`nwergm` is nevertheless an *independent reimplementation*. It was written in Stata, Mata and C, using the published manuals of `ergm` and, where the manuals were not specific enough, the open-source code of `ergm` as the methodological reference. During development `ergm` 4.12.0 was used to certify the results: for the terms that were validated, the observed statistics are identical to those of `ergm`, the pseudolikelihood estimates agree to many digits, and the Monte Carlo estimates agree within simulation noise. Chapter 9 documents what was compared and how. `nwergm` is not affiliated with, endorsed by or supported by the statnet project.

`nwergm` is meant for researchers who work in Stata, or who prefer it to R, and who want to fit the models that `ergm` fits. It is *not* a replacement for `ergm`. `ergm` remains the reference and the more complete program: it has more estimation controls, more constraints and proposals, parallel chains, the valued, temporal and multi-network extensions, and a much larger community and

literature.

Where the two programs differ in a way that matters to a user, this manual says so. The most important differences are these.

- Terms are chosen by Stata options (for example `nodematch(sex)`, `gwesp(0.5)`), not by a model formula. The option name is R's term name; where R's name has a digit followed by a letter, the digit moves to the end (`b1degree` is `bdegree1()`), because Stata does not allow the other spelling in an option name.
- Node attributes must be *numeric* variables of the current dataset (encode strings first), and networks that serve as dyadic covariates are networks in memory.
- `gwesp(#)` and the other geometrically weighted terms with a number in the option have a *fixed* decay; the curved form with an estimated decay is a separate option (`gwespfree(#)`). In `ergm` the default of `gwesp()` is the curved form.
- The Monte Carlo sampler follows `ergm`'s `control.ergm()`: R's default proposal SPDyad on one-mode networks, interval 1024, burn-in 16 times the interval (16384), and an adaptive sample size (a base sample of at least 256 draws, extended until the effective sample size of the estimating equations reaches 64) with `ergm`'s convergence test, computed with `ergm`'s multivariate autocorrelation correction (Section 4.3.3). The random-number streams differ, so Monte Carlo results match `ergm` within simulation noise, not to the last digit.

1.3 What is covered

- **Binary networks** that are undirected, directed or two-mode (bipartite, undirected).
- **Terms.** Of the 139 terms of the package `ergm` (version 4.12.0), 125 are implemented, 123 completely and 2 in part; this is 89.9% of the terms, and 125 of the 126 terms that can be used on a binary network at all. The terms cover structural, degree, triad and shared-partner statistics, node and dyadic covariates, curved (geometrically weighted) terms, the terms of two-mode networks, and the operator terms (`Sum`, `S`, `F`, `Symmetrize`, `Project`, ...). Chapter 5 lists every term.
- **Estimation.** Maximum pseudolikelihood (MPLE) with its variants, Monte Carlo maximum likelihood (MCMLE) with an adaptive sample size, the Hummel step length and `ergm`'s confidence-region convergence test, contrastive divergence, and stochastic approximation (Chapter 2).
- **Likelihood.** The log-likelihood, AIC and BIC as `logLik()` reports them, by bridge sampling for a dyad-dependent model.
- **Constraints.** The sample-space constraints of `ergm` (`constraints()`) and the observation-process constraints (`obsconstraints()`), and the matching Metropolis–Hastings proposals.
- **Missing data.** Missing dyads are handled as in `ergm`, by the method of Handcock and Gile [5].
- **Postestimation.** Goodness of fit (`estat gof`), Markov chain diagnostics (`estat mcmcdiag`), information criteria (`estat ic`), likelihood-ratio tests, conditional tie probabilities (`predict`) and simulation from a fit or from given coefficients (`nwergm simulate`).
- **Dialog boxes** for the model, for simulation and for the postestimation commands.

What `ergm` has that `nwergm` does not. `nwergm` models *binary, static* networks. Not implemented are the 13 terms of `ergm` for valued networks and the operator `B`, which exist only for the valued-network framework of Krivitsky [15]; the packages built on `ergm` for dynamic networks (`tergm` [6]), for count data (`ergm.count`) and for several networks (`ergm.multi`); the Curve operator beyond linear maps; the curved `altkstar` by Monte Carlo maximum likelihood; some constraints; parallel computation; and several controls of `control.ergm()`. Section 9.3 lists the differences in full. Temporal networks are rejected with an error and are never silently

collapsed; a valued network is fitted as the binary graph and says so (Section 3.1).

1.4 Installation and requirements

`nwergm` needs the `nwcommands` core package, which provides the network objects, `nwset` and the data-handling commands. Install the core first and then `nwergm`:

```
. net install nwcommands, from(https://nwcommands.org/net/) replace
. nwinstall, all
. nwinstall, modelpkg(nwergm)
```

The first line installs the core, `nwinstall, all` installs its help files, dialog boxes and menu, and `nwinstall, modelpkg(nwergm)` installs `nwergm` from its repository. To install the core and all four model packages in one step, use

```
. net install nwcommands, from(https://nwcommands.org/net/) replace
. nwinstall, everything
```

`nwergm` can also be installed by itself, once the core is there:

```
. net install nwergm, from(https://raw.githubusercontent.com/thomas-u-grund/nwergm/main/)
    replace
```

The core must be version 3.6.0 or later (the examples of this manual were run with core 3.10.0); `nwergm` checks this when it starts, and also whether the Mata libraries of the core and of `nwergm` were built together, and says how to reinstall if they were not. (Because the Mata library of `nwergm` contains the class definitions of the core that it was compiled against, a core of a different build would otherwise stop Mata with the error `class compiled at different times`.) The Mata library is indexed automatically, so that a fresh installation works in a new Stata session without further steps. The command `nwinstall, models` installs all four model packages (`nwsaom`, `nwergm`, `nwdynam`, `nwrem`) from their repositories, `nwinstall, modelpkg(nwergm)` installs only `nwergm`, and `nwinstall, everything` installs the core and the four model packages together and checks before it installs anything that every package can be reached and that no model needs a newer core than the one being installed. Updating the core never replaces `nwergm`, and updating `nwergm` never changes the core. Alternatively, add a clone of the `nwergm` repository to the ado-path with `adopath + "/path/to/nwergm"`; the tests of the repository find the core through the environment variable `NWCORE`.

Versions. `net install nwergm` installs the state of the main branch. A specific release is installed from its tag, for example

```
. net install nwergm, from(https://raw.githubusercontent.com/thomas-u-grund/nwergm/v1.5.4/)
    replace
```

Use a tagged release when an analysis has to be reproducible. `net describe nwergm` shows the installed version and distribution date. `nwergm version` reports the installed release of `nwergm`, the version of the core, the native plugin in use, the version of `ergm` that the results were compared with and the license, and stores them in `r(version)`, `r(date)`, `r(core)`, `r(plugin)`, `r(reference)` and `r(license)`. After estimation the release that produced the results is in `e(version)`; quote it when you report an analysis. `nwversion` lists the core and all installed model packages.

Software. Stata 17 or later is required. The package was tested with Stata 17 and Stata 18; older versions were not tested and are not claimed. The examples of this manual were run with

Stata/BE 18 on macOS; Stata/BE is enough. The Mata libraries are built with Stata 17, the oldest tested version, because a library built by a newer Stata cannot be used by an older one.

The native plugin. The sampler is a compiled plugin, `ergm_mcmc`. The repository contains builds for macOS (`ergm_mcmc_macos.plugin`), Linux (`ergm_mcmc_unix.plugin`) and Windows (`ergm_mcmc_windows.plugin`). The macOS build is compiled on the development machine; the Linux and Windows builds are produced by continuous integration. The examples and all validation runs of this manual used the macOS build. The full test suite also passes on Windows (Stata/MP 19.5 with the Windows build); the Linux build has not been tested on a Linux machine. Most of the terms of the option groups of Chapter 4 also have a pure Mata sampler, which is the reference implementation and runs when no plugin is available or when `nonative` is given, much more slowly (Chapter 11). The terms of `structural()` and `operators()` and the options `constraints()`, `obsconstraints()` and `mcmcmaxedges()` exist only in the plugin and stop with an error (return code 198) without it. `e(native)` tells whether a fit used the plugin. A plugin from an older installation in Stata's PLUS directory can shadow the plugin of the repository; test a rebuilt plugin through a scratch ado-path.

1.5 Dialog boxes

Every term and option of `nwergm` can be set in a dialog box. Three dialogs are installed with the package and appear in the *Network Analysis* menu when the menu is installed with `nwinstall`:

- `db nwergm` opens the dialog of the model. It has nine tabs: *Basic*, *Degree*, *Triads*, *Covariate*, *Two-mode*, *Curved*, *Operators*, *Constraint* and *Estimate*. The covariate, dyadic, curved, two-mode, `structural()` and `operators()` terms, the constraints and observation constraints, the missing-data and sample-space settings, the estimation method, the MCMC, stochastic-approximation and bridge-sampling settings, `offset()` and `coeflabel()` are chosen on these tabs; where an area has several sections, a drop-down list on the tab selects the section.
- `db nwergm_simulate` opens the dialog of `nwergm simulate` with given coefficients (nine tabs, *Basic* to *Simulate*).
- `db nwergm_estat` opens the dialog of the postestimation commands `estat gof`, `estat mcmcdiag`, `estat ic` and `nwergm simulate` from the last fit (tabs *Command*, *GOF*, *MCMC*, *IC* and *Simulate*).

A dialog builds a command line and runs it, so that the command appears in the Results window and in the log; you can copy it into a do-file. The dialogs are generated from one specification, so that they cover the same terms as the command.

1.6 License and how to cite

`nwergm` is free software under the GNU General Public License, version 3. It is derived from the R package `ergm` (Handcock, Hunter, Butts, Goodreau, Krivitsky, Morris and others; GPL-3). The source code is public; the notice file of the repository lists the sources and the version of `ergm` that each part was compared with.

How to cite. The manual is the citable reference for the package. Please cite it as: Grund, T. U. (2026). *nwergm: Exponential random graph models in Stata. Manual.* Zenodo. https://doi.org/10.5281/zenodo.23661331 The DOI is the one of all versions of the record; this manual describes `nwergm` 1.5.x. Please also report the exact version that produced your results, as printed by `nwergm version` (this manual: 1.5.9), and, for Monte Carlo fits, the `seed()`. If you use the terms or algorithms, cite `ergm` [1] as well.

1.7 How to read this manual

If you know ERGMs from R, read Chapter 3 for the syntax of a first model, use Chapter 4 and Appendix A as a dictionary from `ergm` to `nwergm`, and look up the terms in Chapter 5. If you are new to ERGMs, read Chapter 2 first; it is short and uses only the equations that you need to interpret the output.

Chapter 2
: The model and the estimators: pseudolikelihood, Monte Carlo maximum likelihood, bridge sampling, contrastive divergence, stochastic approximation, the proposals, the stopping rules, reproducibility, missing dyads, constraints.

Chapter 3
: How the data must look, a complete first analysis with real output, and the results that `nwergm` stores.

Chapter 4
: Reference of all options of `nwergm`, organized by topic.

Chapter 5
: The terms: every term of `ergm` with its `nwergm` syntax, its restrictions and how it was validated.

Chapter 6
: Constraints, observation processes, missing dyads, offsets, structural and operator terms, and simulation, in practice.

Chapter 7
: Goodness of fit, Markov chain diagnostics, information criteria, prediction, and the complete list of stored results.

Chapter 8
: Worked examples with real output.

Chapter 9
: The validation against `ergm`: how it was done, what it shows, and where the two programs differ.

Chapter 10
: Known issues, and the messages and what to do about them.

Chapter 11
: Technical notes: the layers of the program, the plugin and the platforms, reproducibility, size limits, run times and the source files.

Appendix A
: A translation table from `ergm` functions to `nwergm` commands.

Appendix B
: The definitions of `ergm` that `nwergm` reproduces on purpose.

Appendix C
: The comparison with `ergm`: the same estimator, the same answers, and where the speed comes from.

Typographic conventions. Stata commands and options are set in `typewriter`. Output shown in listings was produced by the commands above it with the version of `nwergm` described in this manual; the random-number seed is always given, so that the output can be reproduced. Simulation-based estimates change in the last digits when the model or the seed changes. The do-files that produced every listing are in the directory `manual/ex` of the repository. Paths such as `manual/ex`, `cscripts`, `dev`, `native` and `replication` are folders of the GitHub repository of `nwergm`, https://github.com/thomas-u-grund/nwergm, where every file named in this manual can be read and downloaded.

Chapter 2

The model and the estimators

This chapter summarizes the exponential random graph model, the estimators that `nwergm` offers, and the choices that govern how they behave: the sampler, the stopping rules, reproducibility, missing dyads and constraints. It follows `ergm`; the names of the corresponding arguments of R's `ergm()` and `control.ergm()` are given in parentheses. The syntax of the options is in Chapter 4.

2.1 The exponential random graph model

Let Y be the random adjacency matrix of a network with n nodes, with $Y_{ij} = 1$ if there is a tie from i to j (in an undirected network $Y_{ij} = Y_{ji}$; in a two-mode network the ties run between the two sets of nodes only). An ERGM specifies the probability of a network y as

$$P_\theta(Y = y) = \frac{\exp\{\eta(\theta)^\top g(y)\}}{\kappa(\theta)}, \qquad \kappa(\theta) = \sum_{y' \in \mathcal{Y}} \exp\{\eta(\theta)^\top g(y')\}, \tag{2.1}$$

where $g(y)$ is a vector of network statistics (the *terms* of the model), $\eta(\theta)$ the vector of their coefficients as a function of the parameters θ, and $\mathcal{Y}$ the set of networks on which the model is defined (all binary networks on the given nodes, unless constraints restrict it, Section 2.5). In an ordinary model $\eta(\theta) = \theta$. In a *curved* model η is a nonlinear function of θ: for the geometrically weighted terms with an estimated decay, a term $\sum_k w_k(\alpha)\, g_k(y)$ has weights $w_k(\alpha) = e^\alpha\{1 - (1 - e^{-\alpha})^k\}$ and the model has a weight and a decay parameter (Hunter and Handcock [3], Hunter [13]). The constant $\kappa(\theta)$ sums over all $2^{n(n-1)}$ (directed) networks and cannot be computed except for very small networks, which is what makes estimation hard.

Reading a coefficient. The model implies, for every dyad, the log odds of a tie given all other dyads,

$$\operatorname{logit} P_\theta(Y_{ij} = 1 \mid y_{-ij}) = \eta(\theta)^\top \delta_{ij}(y), \qquad \delta_{ij}(y) = g(y^+_{ij}) - g(y^-_{ij}), \tag{2.2}$$

where y^+_{ij} and y^-_{ij} are the network with the tie set to present and to absent, and δ_{ij} is the *change statistic* of the dyad. A coefficient is therefore the change in the conditional log odds of a tie when the corresponding statistic increases by one, holding the rest of the network fixed. For the term `edges` the change statistic is always 1, so that its coefficient is the log odds of a tie in a network with no other effect. For `mutual` the change statistic is 1 if the reverse tie is present and 0 otherwise, so that its coefficient is the additional log odds of a tie that reciprocates an existing one. Coefficients of dyad-dependent terms are conditional on the rest of the network and cannot be read as effects of a single covariate in isolation.

Dyad independence. If every change statistic depends only on the covariates of the two nodes of the dyad (as for `edges`, `nodecov`, `nodematch`, `edgecov`), the dyads are independent, the model is a logistic regression for the tie indicators, and $\kappa(\theta)$ factorizes. Any term that depends on

other ties (`mutual`, `triangle`, `gwesp`, degree terms, ...) makes the model dyad-dependent. The chapters on the terms record which is which.

2.2 Estimators

`nwergm` offers three estimators, chosen with `estimate()` (R: `estimate=`) and, for the Monte Carlo estimator, `mainmethod()` (R: `main.method=`). If `estimate()` is not given, the choice is made by the model: a dyad-independent model is fitted by the pseudolikelihood, which is then the exact maximum likelihood, and all other models by Monte Carlo maximum likelihood. The older option `method(mple|mcmle|cd)` is a synonym.

2.2.1 Maximum pseudolikelihood

The pseudolikelihood replaces the likelihood by the product of the conditional probabilities (2.2) over all dyads,

$$\mathrm{PL}(\theta) = \prod_{i,j} P_\theta(Y_{ij} = y_{ij} \mid y_{-ij}),$$

which is the likelihood of a logistic regression of the tie indicators on the change statistics, evaluated on the observed network. `nwergm` builds the matrix of change statistics (in the plugin when it can) and maximizes the pseudolikelihood by Newton's method with a tight convergence tolerance, so that the coefficients agree with R's to about 10^{-6} or better (Chapter 9). Perfectly collinear columns cannot be estimated and are reported as omitted. For a dyad-independent model the estimate and its standard errors are those of the maximum likelihood estimator. For a dyad-dependent model the standard errors of the pseudolikelihood are not valid and the estimate is mostly useful as a starting value for the Monte Carlo estimator; `nwergm` says so in a note under the table. Missing dyads are left out of the product (Section 2.4).

The variants (R's `control.ergm()` arguments):

`mpletype(glm|penalized) (MPLE.type)`
: `penalized` is the Firth (Jeffreys-prior) penalized pseudolikelihood, which is finite when the ordinary estimate does not exist because of separation. It cannot be combined with `offset( )`.

`mplecheck(on|off) (MPLE.check)`
: Tests whether the estimate exists (a linear program for separation, solved by a simplex routine). If it does not, `nwergm` warns `The MPLE does not exist!` and, with `estimate( mple)`, reports the last iterate of `glm()`, as R does (huge coefficients, standard errors in the thousands). With `mplecheck(off)` the iterate is reported without the test.

`mplecovariance(invhess|godambe|bootstrap) (MPLE.covariance.method)`
: Replaces the inverse-Hessian covariance of a dyad-dependent pseudolikelihood fit by the Godambe sandwich or by the covariance of 500 pseudolikelihood fits to networks simulated at the estimate (burn-in 1024, interval 1024).

2.2.2 Monte Carlo maximum likelihood

Monte Carlo maximum likelihood (MCMLE) follows Geyer and Thompson [12] and, for the step length and the stopping rules, Hummel, Hunter and Handcock [11]. For a fixed starting value η_0, the log-likelihood ratio is

$$\ell(\eta) - \ell(\eta_0) = (\eta - \eta_0)^\top g(y_{\mathrm{obs}}) - \log E_{\eta_0} \exp\{(\eta - \eta_0)^\top g(Y)\},$$

and the expectation is replaced by an average over a sample of networks simulated from the model at η_0. Each outer iteration therefore (1) draws a Markov chain sample of networks at the current coefficients, (2) takes a step towards the maximiser of the approximate log-likelihood ratio, with a step length that keeps the step inside the region where the approximation can be trusted (the

Hummel step length), and (3) tests whether the sample statistics are already consistent with the observed statistics. The starting values are the pseudolikelihood estimates (or the values of `init()`). On the models of this manual `nwergm` needed one to four iterations (eight for the Lazega model with the shared-partner term in Section 8.6). The number of iterations is stored in `e(mcmle_iterations)`, and the limit is `mcmleiterations()` (R: `MCMLE.maxit`, default 60); a fit that reaches the limit is reported with a warning and `e(converged)` $= 0$.

The sampler. The networks are simulated by a Metropolis–Hastings chain that proposes to toggle one dyad at a time. Its basic step is the *tie/no-tie* (TNT) proposal of Morris, Handcock and Hunter [14]: with probability one half it proposes to remove a randomly chosen existing tie, and otherwise to add a randomly chosen non-tie, with the acceptance probability corrected for the unequal chance of proposing a given toggle. Because most networks are sparse, this mixes much better than the uniform proposal (`proposal(uniform)`, or `proposal(random)`), which picks a dyad uniformly. On a one-mode network the default is R's default proposal, SPDyad (`MCMC.prop=~sparse+.triadic`): with probability 0.75 a step is a tie/no-tie step, and otherwise the dyad is drawn uniformly from the dyads whose two nodes have a shared partner (on a directed network a two-path $i \to k \to j$, R's type OTP), with the acceptance probability corrected for both ways in which the dyad could have been proposed. The shared partners of every dyad are kept up to date as the chain toggles ties. In a model with strong triadic effects the triadic steps reach the dyads that matter much more often; on the Mesa High School benchmark model the chain gives about four times the effective sample size per draw of the tie/no-tie proposal, 74 against 17 effective draws per 1000 (Appendix C). `mcmcprop(sparse)` gives the tie/no-tie proposal alone, and `mcmcprop(triadic(triFocus=#))` another share of triadic steps; `e(trifocus)` holds the share used (missing when SPDyad was not used). Two-mode networks and `bd` with only maximum bounds use the tie/no-tie proposal, as in R. In a fixed-size sample (`mcmleeffectivesize(0)`, and the fits by contrastive divergence, stochastic approximation and with `obsconstraints()`) the chain runs for `mcmcburnin()` proposals (default 16384), then records a network every `mcmcinterval()` proposals (default 1024) until `mcmcsamplesize()` (default 3000) networks are recorded. In the default adaptive sample of Monte Carlo maximum likelihood, described next, the number of draws is not fixed. The interval and the burn-in default to those of `ergm`'s `control.ergm()`. Constraints change the proposal; see Section 2.5.

Adaptive sample size and interval. Draws of a Markov chain are autocorrelated, so that a sample of a given size carries less information than the same number of independent draws. `nwergm` samples as `ergm`'s `ergm()` does (R: `MCMLE.effectiveSize`, `MCMLE.samplesize`), with a sample size that adapts to the autocorrelation of the chain and not a fixed number of draws. Each Monte Carlo iteration (1) draws a base sample whose size is `mcmcsamplesize()`, by default the larger of 256 and 32 times the number of coefficients (R: `MCMLE.samplesize.min`, `.per_theta`), at the current interval; (2) estimates from the sample itself how many initial draws to discard as burn-in (a trend test on the estimating equations, as `ergm` does); (3) estimates the effective sample size (ESS) of the estimating equations of the draws that remain, as `ergm` does: the number of draws divided by the inflation factor of the multivariate autoregressive spectral density at frequency zero (R's `spectrum0.mvar()`), which compares the determinant of the long-run covariance of the equations with that of their ordinary covariance; and (4) if it is below the target `mcmleeffectivesize()` (default 64), draws more, in at most 16 runs whose lengths are predicted from the shortfall, and thins the whole sample by two, doubling the interval, whenever it has doubled in size. The first iteration starts at `mcmcinterval()` (1024); every later iteration starts from half the interval that the previous sample ended with (R: `MCMLE.effectiveSize.interval_drop`=2). The base size is at least twice the ESS target (R: `MCMLE.min.depfac`=2). The burn-in test (step 2) is R's `geweke.diag.mv()`: a Hotelling-type test of the means of the first 10% and the last 50% of the draws, with the long-run covariance of both parts estimated by one multivariate autoregression of the two centered parts (R's `approx.hotelling.diff.test()` with equal variances); a failed

test counts as a trend. The number of draws kept in the last iteration is `e(mcmc_samplesize)` and the interval of that sample is `e(mcmc_interval_final)`. The explicit burn-in of each run is one interval; `mcmcburnin()` (default 16 times the interval) is the burn-in of fixed-size samples, of simulation and of the goodness-of-fit simulations. The ESS of the target is that of the estimating equations (for a curved term, of the equations in the coefficients that are estimated), so that it is smaller than the ESS of an individual statistic as `estat mcmcdiag` reports it and is, as in `ergm`, 64 and not several hundred; a failed convergence test raises the target and the base size (next paragraph). The sizes are capped at 32 times their starting values, a safeguard that `ergm` does not have and that matters only in a fit that never converges.

The interplay of the options. `mcmcsamplesize(#)` is the *base* size of the adaptive sample, not the number of draws kept: the fit draws at least that many and more if the ESS target has not been reached. `mcmleeffectivesize(#)` sets the target ESS; a larger number gives a longer, more accurate sample, and `mcmleeffectivesize(0)` switches the adaptive sample off, so that every iteration records exactly `mcmcsamplesize()` draws (3000 if the option is not given) at `mcmcinterval()` after `mcmcburnin()`, the interval grows when the ESS of the statistics stays below the larger of 200 and 64 times the number of coefficients, the convergence test is a Hotelling test of the mean estimating equation, the covariance comes from one further sample at the final coefficients, and the bridge samples have 3000 draws; this is the fixed-size algorithm. The options `mcmcinterval()` and `mcmcburnin()` apply to both, in the adaptive sample as the starting values described above. Fits by contrastive divergence and stochastic approximation and fits with `obsconstraints()` have no adaptive sample and use fixed-size samples whatever `mcmleeffectivesize()` says (3000 draws unless `mcmcsamplesize()` is given). A model whose chain does not mix can take very long: the sample is extended up to its caps, and 60 iterations can take many minutes. Lower `mcmleiterations()` for a model that you suspect to be degenerate.

Stopping rules. The option `mcmletermination()` (R: `MCMLE.termination`), with the values `confidence`, `hummel`, `hotelling` and `none`, chooses how the fit stops. The default, `confidence`, is `ergm`'s test: the fit stops when, with the confidence `mcmleconfidence()` (R: `MCMLE.confidence`, default 0.99), the estimating equations at the updated coefficients lie inside a tolerance region, an ellipsoid whose size is 0.1 of the covariance of the equations (R: `MCMLE.MCMC.precision`), taking the autocorrelation of the chain into account. As in `ergm`, the covariance of the mean estimating equation in this test is the multivariate long-run covariance of the equations (`spectrum0.mvar()` divided by the number of draws), and the degrees of freedom are the number of draws divided by the inflation factor, minus one. A fit that fails the test raises the target ESS by the factor min(critical value/T^2, 2) and the base sample size by its square root, as `ergm` does, and iterates; this is why an adaptive fit can take several iterations. With `mcmleeffectivesize(0)` the test is the earlier one, a Hotelling-type test that cannot distinguish the observed statistics from the mean of the sampled statistics at the confidence level. `hotelling` and `hummel` stop after the criterion has held in two successive iterations, with the sample size raised by a factor of four (`MCMLE.last.boost`) after the first; `none` runs all iterations. `mcmlesteplength()` and `mcmlestepmargin()` (R: `MCMLE.steplength`, `MCMLE.steplength.margin`) control the step length; the margin `.` switches the Hummel step length off. If the statistics of the sample do not vary at all the fit stops with R's error *unconstrained MCMC sampling did not mix at all*, which is what happens for a degenerate model (Section 8.4).

Standard errors. The information matrix is estimated by the covariance of the simulated statistics of the last iteration's sample (as in `ergm`; in a fixed-size fit, of one further sample drawn at the final coefficients, and in an adaptive fit that ran out of iterations before convergence, too). As in `ergm`'s default, the reported covariance is the inverse information plus the Monte Carlo error of the estimate, $V^{-1} + V^{-1}(S/n)V^{-1}$, where S is an autoregressive spectral estimate of the long-run covariance of the draws. Standard errors therefore grow when the chain mixes badly. A

coefficient table whose standard errors are larger than those of a similar pseudolikelihood fit is usually not an error: the pseudolikelihood standard errors of a dyad-dependent model are too small.

2.2.3 Log-likelihood, AIC and BIC

`nwergm` stores the log-likelihood as R's `logLik()`: exact for a dyad-independent model (where the pseudolikelihood is the likelihood), the pseudolikelihood after `estimate(mple)` of a dyad-dependent model, and, after Monte Carlo maximum likelihood, the likelihood evaluated by *bridge sampling* from the dyad-independent submodel (R: `ergm.bridge.dindstart.llk`). The log-likelihood ratio between the fitted model and the dyad-independent model that keeps only the dyad-independent terms is the integral of the expected statistic along a path between the two coefficient vectors; the integral is approximated by B bridges, each with its own Markov chain. The options are `bridgesteps()` (default 16, R: `bridge.nsteps`), `bridgesamplesize()` (default the number of draws of the last iteration, as in `ergm`, so a few hundred; with `mcmleeffectivesize(0)`, `mcmcsamplesize()`) and `bridgeinterval()` (default the final thinning interval divided by the number of bridges); `nologlik` skips the evaluation, in which case the log-likelihood, AIC and BIC are missing. The Monte Carlo standard error of the log-likelihood is `e(ll_se)`; it is zero when the log-likelihood is exact. The null model is the model with all coefficients zero, whose log-likelihood is $N \ln \frac{1}{2}$ with N the number of informative dyads. The deviance is -2 times the log-likelihood, AIC and BIC use the number of estimated coefficients (offsets excluded) and, for the BIC, the number of informative dyads. The likelihood is not evaluated for a model with a curved term fitted by Monte Carlo maximum likelihood.

2.2.4 Contrastive divergence

`estimate(cd)` (R: `estimate="CD"`) is contrastive divergence. Starting from zero, each draw is a short Markov chain of `cdnsteps()` proposals (R: `CD.nsteps`, default 8) started from the *observed* network, and the coefficients are updated by an importance-sampling Newton step with the Hummel step length. The number of draws per coefficient vector is `cdsamplesize()` (default 128 times the number of free coefficients, R: `CD.samplesize.per_theta`), and `mcmleiterations()` bounds the number of iterations (R: `CD.maxit`). Contrastive divergence is cheap and is meant to provide starting values for the Monte Carlo estimator; its estimates are not maximum likelihood estimates. R reports the covariance of the estimate as missing; because a Stata estimation result cannot hold missing values on the diagonal, `nwergm` stores the covariance of the contrastive-divergence sample in `e(V)` and says so under the table.

2.2.5 Stochastic approximation

`mainmethod(sa)` (R: `main.method="Stochastic-Approximation"`) replaces the Monte Carlo maximum likelihood iteration by the stochastic approximation of Snijders [7]: phase 1 estimates a gain from short runs, phase 2 consists of four sub-phases of Robbins–Monro updates with decreasing gain, and phase 3 draws a sample at the final coefficients for a Newton step and the covariance. The controls are `saburnin()` (R: `SA.burnin`, default 16384), `sainterval()` (`SA.interval`, 1024), `saphase3()` (`SA.phase3_n`, 1000), `sagain()` (`SA.initial_gain`, 0.1) and `sansub()` (`SA.nsubphases`, 4). Stochastic approximation is slower than Monte Carlo maximum likelihood on well-behaved models and is an alternative when the latter does not converge.

2.2.6 Starting values and offsets

`init(numlist)` (R: `control.ergm(init=)`) gives one starting value per coefficient, in the order of the coefficient table; a period or `NA` leaves a value to be estimated. Values for coefficients that are fixed by `offset()` replace the offset values; other values are the starting values of the Monte

Carlo fit and are held fixed in the pseudolikelihood step that finds the missing ones. If every value is given, the pseudolikelihood step is skipped. With `estimate(mple)` only offset values are used, except for curved terms, where `init()` gives the starting point of the pseudolikelihood fit (one value for each parameter, the weight and decay of the curved term last).

`offset(coefname # [coefname # ...])` (R: the `offset()` wrapper with `offset.coef`) fixes named coefficients at known values. For the pseudolikelihood, the contribution of the fixed term enters the logistic regression as an offset; for Monte Carlo maximum likelihood, the Newton step is restricted to the free coefficients, while the term still takes part in the simulation (fixing a coefficient does not remove its term from the model). An offset coefficient is reported at the given value with standard error zero and a zero row and column in `e(V)`. At least one coefficient must remain free, and offsets are not allowed for the weight and decay of a curved term.

2.3 Reproducibility

Monte Carlo estimates depend on the random numbers. `seed(#)` sets the seed of the sampler; it is drawn from Stata's random-number generator right after `set seed`, so that the stream is deterministic. The same `nwergm` call with the same `seed()`, the same data and the same plugin gives *identical* coefficients in any session, whatever was loaded or fitted before. (An earlier release failed this: the starting network was built in the order in which the core returns the neighbors of a node, which differs once another network has been loaded; the neighbors are now read in sorted order, and a test fits the same model after loading other networks.) Section 8.1.2 shows the guarantee at work.

What the seed does *not* guarantee:

- A different operating system or plugin build, or a change of `mcmcinterval()`, `mcmcsamplesize()` or another control, gives a different (statistically equivalent) Monte Carlo fit.
- The stream is not the random-number stream of R. Results match `ergm` only up to Monte Carlo error.
- Without `seed()` the current Stata seed is used; two runs in the same session then differ.

Report the seed with the results, and judge the Monte Carlo error by refitting with other seeds (Section 8.1.2 and Chapter 9 show how large the spread over seeds is).

2.4 Missing dyads

A tie that is missing (a missing value in the edge matrix of the network, off the diagonal) is not treated as a non-tie. As in `ergm`, `nwergm` detects missing dyads automatically and

- drops them from the (pseudo)likelihood; the number of informative dyads `e(N)` excludes them, and for a dyad-independent model the pseudolikelihood estimate is the maximum likelihood estimate;
- for a dyad-dependent model fitted by Monte Carlo maximum likelihood, draws a second, constrained sample in which only the missing dyads are free and every observed dyad is held fixed (Handcock and Gile [5]); the estimating equations compare the two samples, and the information matrix is the difference of their covariances. The missing dyads of the starting network are imputed at random with the observed density, as `ergm` does;
- reports the number of missing dyads in `e(nmissing)`.

A dyad-dependent model fitted by pseudolikelihood also imputes the missing dyads of the starting network at random (so the result depends on the seed, as in R). Missing dyads cannot yet be combined with `freedyads()`, `blockdiag()` or `fixdensity`. *Missing covariates* are not supported, as in `ergm`: a node attribute with a missing value in a term (`nodecov()`, `nodematch()`, ...) and a dyadic covariate with missing dyads stop the command with return code 416. Drop or impute

the missing values first. For two-mode terms only the attribute of the term's own mode is checked. An example is in Section 8.5.

2.5 Constraints and observation processes

`constraints(`*`formula`*`)` restricts the set $\mathcal{Y}$ of networks on which the model is defined, exactly as `constraints=` does in `ergm`. The formula is a list of constraint terms joined by `+` or `-`, with arguments written as in R, by position or by name. Two kinds of constraints exist. Constraints that *fix some dyads* (`blocks`, `blockdiag`, `fixedas`, `fixallbut`, `observed`, `egocentric`) mark dyads that may not change; they are combined from left to right, a term with `+` keeping only the dyads that may vary according to it and to everything to its left, and a term with `-` adding the dyads that may vary according to it. Constraints that *tie the dyads together* (`edges`, `degrees`, `odegrees`, `idegrees`, `b1degrees`, `b2degrees`, `degreedist`, `idegreedist`, `bd`) always restrict the sample space and may only carry a `+`. Constraints implied by others are dropped, as in R. The complete list is in Section 6.4.

Statistics that a constraint holds constant (for example `edges` under `constraints(edges)`, or `kstar()` under `constraints(degrees)`) are fixed at zero and ignored in the fit, with a note, as R does. A dyad-dependent constraint makes the sample space dyad-dependent, so that the default method is Monte Carlo maximum likelihood, started from zero or from the values given in `init()`. With `estimate(mple)` the pseudolikelihood is formed over the free dyads.

Proposals. The proposal is chosen from the constraints with R's rules. With no constraint, with dyad constraints and with `bd`, one dyad is toggled (SPDyad, tie/no-tie weighted or uniform, as above, with the counts of the proposal restricted to the free dyads). With `edges`, a tie is swapped with a non-tie. With `degrees`, or `idegrees` together with `odegrees`, or `b1degrees` together with `b2degrees`, tetrad moves (and, for directed networks, hexad moves) keep all degrees. With `odegrees`, `idegrees`, `b1degrees` or `b2degrees` alone, one end of a tie is moved. With `degreedist` or `idegreedist`, the ends of ties are exchanged between nodes whose degrees differ by one. With `dyadnoise`, a tie/no-tie proposal with the noise likelihood in the acceptance ratio is used. Other combinations stop with the message that R gives. The constraint options of the command are `mcmcprop()` (R's `MCMC.prop` hints: `sparse+.triadic` is the default and gives SPDyad as described above, also with dyad constraints and `bd`; `sparse` gives the tie/no-tie proposal; `triadic(#)` or `triadic(triFocus=#)` sets the share of triadic steps; on a two-mode network `.triadic` is skipped and `triadic` is refused; `strat` is accepted but not used, with R's note) and `mcmcmaxedges()` (R's `MCMC.maxedges`).

The older options `freedyads(`*`netname`*`)` (the dyads that are ties in a network in memory may vary, every other dyad is fixed: R's `fixallbut()`), `blockdiag(`*`varname`*`)` (only dyads within the same value of the variable may vary: R's `blockdiag()`) and `fixdensity` (R's `constraints=~edges`) remain available; they can be given one at a time and cannot be combined with `constraints()`.

Observation processes. `obsconstraints(`*`formula`*`)` (R: `obs.constraints`) describes how the network was *observed*, for example the dyads that were not observed, as `fixallbut(`*`unobserved`*`)`. Two chains are run at every iteration, one under `constraints()` and one under `constraints()` and `obsconstraints()` together, both started from the observed network. The coefficients solve the equation that sets the two expected statistics equal, and the information is the difference of the covariances of the two chains. This is the same estimator that handles missing dyads (Section 2.4) with a general constraint in place of the set of missing dyads. It requires Monte Carlo maximum likelihood. The constraint `observed`, which frees exactly the missing dyads, is the default observation constraint for a network with missing dyads; in `constraints()` on a network with missing dyads R itself stops with an internal error, and `nwergm` stops with a plain

message and points to `obsconstraints(observed)`. On a network without missing dyads the empty constraint is dropped, as in R, and `nwergm` prints a note.

2.6 Curved terms

A curved term has a weight and a decay parameter (reported as, for example, `gwesp_weight` and `gwesp_decay`). The fit follows R: the pseudolikelihood fit is a trust-region optimization in the curved parameters, and the Monte Carlo fit takes a Newton step in the space of η and projects it back onto the curve. A decay can be weakly identified on a small or sparse network, in which case the fit converges to a boundary solution (decay near 0, or very large). This is a property of the model, not a bug: R's `ergm` converges to the same kind of solution on the same data and can fail to converge at all on a poorly identified network, and so can `nwergm`, which then reports a non-converged fit rather than a plausible-looking wrong number (`e(converged)`). A large standard error of the decay is the usual sign; consider a longer burn-in, a larger sample, a different starting decay, or the fixed-decay form of the term. Fixed-decay terms (`gwesp(0.5)`) are ordinary terms and do not have this problem. The option `init()` gives starting values for the curved fit.

Chapter 3

Getting started

This chapter takes you from data to a first fitted model, shows how to read the output and explains how the coefficients are named. All commands were run with `nwergm` 1.5.9 on the Florentine families network of the statnet project.

3.1 The data

Networks are `nwset` objects. `nwergm` does not read the network from variables. It takes the *name* of a network that exists in the current `nwcommands` session, `nwergm` *netname*`,` *options*. If the name is left out, the currently set network is used. A network is declared from an adjacency matrix with `nwset, mat()`, from an edge list with `nwfromedge`, or loaded with `nwuse` or `nwwebuse`; see `help nwset`. One command fits one network.

Binary ties. `nwergm` models a binary, static network. A network with temporal metadata is rejected with an error; it is never silently collapsed to one time slice. Since the core 3.6.0 the type of a network is fixed when it is created: a network made with a value variable is flagged *valued* (also when every value is 1), a network made from a matrix of zeros and ones or from an edge list without values is *binary*, and a network declared with `bipartite` is *two-mode*. A valued network is fitted as the binary graph in which a tie is present when its value is not zero, as `ergm` does without `response=`; one note says that the tie values are ignored (no note when every value is 1, where nothing is lost). The tie values are never used as an outcome: valued ERGMs are a different model. Only the dyads between different nodes are part of the model. Directed, undirected and two-mode (bipartite, undirected) networks are supported. Two-mode networks are described by their incidence matrix: the *first mode* is formed by the *columns* of the matrix given to `nwset, mat(`*M*`) bipartite`, so that to have the n_1 rows of an $n_1 \times n_2$ matrix as the first mode, the transpose has to be passed (Section 8.8). Terms with a mode in their name, `bdegree1( )` for example, refer to these modes.

Node attributes. The attributes of the nodes are ordinary variables of the current dataset, one row per node in the node order of the network, and they must be *numeric*: a string variable is refused with an error and the advice to `encode` it. Categorical attributes (`nodematch()`, `nodefactor()`, `nodemix()`) use the distinct values of the variable as levels; the levels are the values sorted ascending, and the *position* in this sorted list is what `levels()` and `base()` refer to. Missing values in an attribute that a term uses stop the command with return code 416, as in `ergm`. A Stata expression is allowed in the covariate terms that take a variable, for example `nodecov(ln(wealth))`.

Dyadic covariates. A dyadic covariate is another network in memory, or an $n \times n$ Stata matrix, named in `edgecov()`, `dyadcov()` or `hamming()`. It must have the same number of nodes. On a

two-mode model the covariate network can itself be two-mode: only the cells between the two modes are read (the cells between nodes of the same mode are missing by construction and are not a problem).

The term `edges`. Every model includes the term `edges` (the density; the intercept). It is required, except that `structural(density)` or `structural(meandeg)`, which are collinear with it, can stand in for it.

The Florentine families

The data of this chapter are the marriage ties and the business ties among 16 Renaissance Florentine families, with the wealth of each family in thousands of lira (Padgett and Ansell [17]); the network is distributed with `ergm` as `flomarriage`. It is loaded by `nwwebuse`:

```
. nwwebuse florentine, nwclear

. nwset
(2 networks)
--------------------
      flobusiness
      flomarriage

. nwsummarize flomarriage
-------------------------------------------------
   Network name:  flomarriage
   Network id:  2
   Directed: false
   Valued: false
   Two-mode: false
   Nodes: 16
   Selfloop: false
   Edges: 20
   Minimum value:  0
   Maximum value:  1
```

3.2 A first model: a dyad-independent model

The simplest model has two terms: the density (`edges`) and a main effect of the wealth of the two families (`nodecov(wealth)`, the sum over the ties of the wealth of both endpoints). Both terms depend only on the attributes of the two nodes of a dyad, so the model is dyad-independent. `nwergm` detects this and fits it by pseudolikelihood, which is then the maximum likelihood estimate; no random numbers are involved.

```
. nwergm flomarriage, edges nodecov(wealth)

Exponential-family random graph model

Network:               =  flomarriage
Nodes:                 =  16
Ties:                  =  20
Directed:              =  No
Estimation:            =  MPLE

------------------------------------------------------------------------------
   flomarriage | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
---------------+--------------------------------------------------------------
         edges |  -2.594929   .5360561    -4.84   0.000     -3.64558   -1.544278
nodecov_wealth |   .0105459   .0046743     2.26   0.024     .0013845    .0197073
```

```
------------------------------------------------------------------------------

Maximum pseudolikelihood estimate; this is the maximum likelihood estimate only if the model is
    dyad-independent.

             Null Deviance:     166.4  on 120 degrees of freedom
         Residual Deviance:     103.1  on 118 degrees of freedom

Log-likelihood:   -51.5543 (df = 2)   AIC:     107.1   BIC:     112.7
```

The header names the network, its numbers of nodes and ties, whether it is directed, and the estimation method. The table lists each coefficient with its standard error, z statistic, p-value and confidence interval. The note under the table says what the estimate is. The deviances are those of the logistic regression: the null deviance is that of the model with all coefficients zero (every dyad has probability one half), and the residual deviance is -2 times the log-likelihood; their degrees of freedom are the number of informative dyads minus the number of estimated coefficients. The last line gives the log-likelihood, the degrees of freedom of the model, AIC and BIC, and the Monte Carlo standard error of the log-likelihood (zero when it is exact). The interpretation is that of a logistic regression: the change statistic of the term for a pair of families is the sum of their wealth, so that if one family is 10 units richer, the odds of a marriage tie between it and any given other family are multiplied by $\exp(0.0105 \times 10) \approx 1.11$.

3.3 A dyad-dependent model

Adding `gwesp(0.5)`, the geometrically weighted edgewise shared partners with a decay of 0.5, makes the model dyad-dependent: whether a tie closes triangles now enters the probability of the tie. `nwergm` switches to Monte Carlo maximum likelihood without being asked. The option `seed( )` makes the random numbers reproducible.

```
. nwergm flomarriage, edges nodecov(wealth) gwesp(0.5) seed(1)

Exponential-family random graph model

Network:               =  flomarriage
Nodes:                 =  16
Ties:                  =  20
Directed:              =  No
Estimation:            =  MCMLE
MCMC sample size:      =  243

------------------------------------------------------------------------------
   flomarriage | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
---------------+--------------------------------------------------------------
         edges |  -2.576975   .5269599    -4.89   0.000    -3.609798   -1.544153
nodecov_wealth |   .0104462   .0045714     2.29   0.022     .0014864    .0194061
     gwesp_0.5 |  -.0045009   .2390051    -0.02   0.985    -.4729422    .4639405
------------------------------------------------------------------------------

MCMLE converged after 1 iteration(s).
Evaluating log-likelihood at the estimate (bridge sampling with 16 bridges) ...

             Null Deviance:     166.4  on 120 degrees of freedom
         Residual Deviance:     103.1  on 117 degrees of freedom

Log-likelihood:   -51.5534 (df = 3)   AIC:     109.1   BIC:     117.5
Monte Carlo standard error of the log-likelihood:  .00086
```

The header now reports the Monte Carlo estimator and the sample size, the number of draws that the last iteration kept (243 here: `nwergm` samples until the effective sample size reaches its target,

so that the number depends on the model and the seed; Section 2.2.2). The line below the table reports that the convergence test of the Monte Carlo iteration was satisfied after one iteration (`e(converged)`, `e(mcmle_iterations)`), and that the log-likelihood was then evaluated at the estimate by bridge sampling with 16 bridges; the number in parentheses at the end of the last line is the Monte Carlo standard error of that log-likelihood. The coefficient of the shared-partner term is close to zero here: on this small network the wealth effect and the density explain the data about as well. The pseudolikelihood fit of the same model is available with `estimate(mple)` (Section 8.1.1).

Is the fit trustworthy? A satisfied convergence test is a necessary check, not a proof that the Markov chain explored the space of networks well. Chapter 7 shows the diagnostics, and Section 8.3 how to read the diagnostics of a fit.

3.4 How coefficients are named

The names of the coefficients are those of `ergm` with a period written as an underscore, because Stata reads a period in a matrix column name as a time-series operator. A node attribute is appended to the term name (`nodecov_wealth`), a level of a categorical attribute to the attribute (`nodefactor_race_n_2`, `nodemix_sex_n_1_2`), a decay or other parameter to the term (`gwesp_0.5`, `kstar_2`, `degree_1`; the name of a term is not always formed in the same way, for example `esp1`, so read the names off the table). The curved form of a term reports two coefficients, `gwesp_weight` and `gwesp_decay`. The terms of `structural()` have the names of `ergm` (`twopath`, `cycle3`, `threetrail_RRR`), and operator terms are named as in R, `Symmetrize(strong)~triangle`. `coeflabel(`*`old new ...`*`)` renames coefficients (R's `Label()` operator). The display is truncated for long names; the full name is in `e(b)`.

The order of the terms. The order of the coefficients is that in which `nwergm` processes the options, not the order in which you typed them: `edges`, `mutual`, the node-covariate family, the structural family, `sender` and `receiver`, the dyadic covariates, the geometrically weighted family, and so on. This matters for `init()` and `offset()`, which refer to the order or the names of the coefficient table, and for `nwergm simulate`, whose `theta()` lists one value per coefficient in the same order. Run the model once and read the order off the table.

Chapter 4

Syntax and options

This chapter lists every option of `nwergm` and of its subcommands. The terms are defined in Chapter 5; the estimators in Chapter 2. The names of the corresponding arguments of R's `ergm()` and `control.ergm()` are given where they exist.

4.1 Syntax

```
nwergm [netname], edges [terms] [estimation options]
nwergm simulate [nodes], edges [terms] theta(numlist) [simulation options]
nwergm simulate, [nsim(#) output(network|stats|edgelist) ...]     (from the last fit)
estat gof | estat mcmcdiag | estat ic
predict, [pr|xb] matrix(name) | frame(name)
nwergm version
```

netname is the network to fit; the default is the current network. The option `edges` is required (see Chapter 3). All other options are optional and can be given in any order. Options that take a *varlist* take numeric node variables; options that take a *numlist* accept ranges as in Stata, `degree(0/3)` for R's `d=0:3`. In the tables, *netname* is a network in memory and *matname* a Stata matrix.

4.2 Terms

The options that select the statistics of the model. Chapter 5 gives the definition, the restrictions and the validation of each term.

4.2.1 Node and dyadic covariates, and reciprocity

Option	Description
`edges`	Number of ties (the density, the intercept). Required.
`mutual`	Number of reciprocated pairs. Directed networks.
`nodematch(varlist)`	Homophily: ties between nodes with equal values; one coefficient per variable. Suboptions `diff`, `keep()`, `levels()`, `levelvalues()`.
`nodematchdiff(varlist)`	Differential homophily: one coefficient per level (R's `diff=TRUE`).
`nodecov(varlist)`	Main effect of a numeric covariate (sum over the ties of the values of both endpoints). Stata expressions are allowed.
`nodeicov(varlist)`, `nodeocov(varlist)`	Receiver and sender covariate effects. Directed networks.
`nodefactor(varlist)`	One coefficient per level but the first: number of tie endpoints at the level. Suboptions `levels()`, `levelvalues()`, `base()`.
`nodeofactor(varlist)`, `nodeifactor(varlist)`	The same for out-ties and in-ties. Directed networks.

Option	Description
`nodemix(varlist)`	Mixing matrix: ties between each pair of levels (ordered pairs in directed networks, the first cell omitted). Suboptions `levels()`, `levelvalues()`, `levels2()`, `base()`.
`mm(var)`	Mixing matrix of one attribute (the cells of `nodemix()`). Suboptions `levels()`, `levels2()`.
`absdist(varlist)`	Sum over ties of $\|x_i - x_j\|^{pow}$ (R's `absdiff`). Suboption `pow(#)`.
`absdiffcat(varlist)`	One count per distinct nonzero absolute difference. Suboptions `levels()`, `levelvalues()`, `base()`.
`diff(varlist)`	Sum of $(x_{tail} - x_{head})^{pow}$. Suboptions `pow()`, `dir()`, `signaction()`.
`smalldiff(varlist)`	Ties with $\|x_i - x_j\| \leq$ cutoff. Suboption `cutoff(#)` (required).
`sociality(varlist)`	Degree of each node (undirected); `sociality(.)` for no attribute. Suboptions `nodes()`, `levels()`.
`attrcov(varlist)`	Edge covariate $x(i,j) = M(\text{level of } i, \text{level of } j)$. Suboption `mat(matname)` (required).
`edgecov(netname|matname)`	Dyadic covariate from a network or an $n \times n$ matrix.
`dyadcov(netname|matname)`	Directed: three coefficients (mutual, upper and lower triangle); undirected: as `edgecov()`.
`hamming(netname)`	Number of dyads that differ from a reference network.
`nodecovrange(varlist)`, `nodeocovrange(varlist)`, `nodeicovrange(varlist)`	Sum over nodes of the range of the covariate among the neighbors (out-, in-neighbors). Dyad-dependent.
`nodefactordistinct(varlist)`, `nodeofactordistinct(varlist)`, `nodeifactordistinct(varlist)`	Sum over nodes of the number of distinct levels among the neighbors. Dyad-dependent.
`sender`, `receiver`	One coefficient per node but the first: out-degree, in-degree of the node. Directed networks.

4.2.2 Geometrically weighted terms

Option	Description
`gwesp(#)`	Geometrically weighted edgewise shared partners, fixed decay. Undirected, or directed with the definition of `type()`.
`gwdsp(#)`, `gwnsp(#)`	Geometrically weighted dyadwise and non-edgewise shared partners, fixed decay (gwdsp = gwesp + gwnsp).
`gwdegree(#)`	Geometrically weighted degree, fixed decay.
`gwodegree(#)`, `gwidegree(#)`	Out- and in-degree versions. Directed networks.
`gwespfree(#)`, `gwdspfree(#)`, `gwnspfree(#)`	Curved forms with an estimated decay; the number is the starting value of the decay. Report `_weight` and `_decay`.
`gwdegreefree(#)`	Curved gwdegree (undirected).
`gwodegreefree(#)`, `gwidegreefree(#)`	Curved out- and in-degree versions (directed).
`type(OTP|ITP|OSP|ISP|RTP)`	Definition of the shared partner for *every* term of the model that has one (`gwesp`, `gwdsp`, `gwnsp`, `esp`, `dsp`, `gwespfree`, ...) on a directed network; default OTP; ignored on an undirected network. OTP is $i \to k \to j$, ITP $i \leftarrow k \leftarrow j$, OSP $i \to k \leftarrow j$, ISP $i \leftarrow k \to j$, RTP $i \leftrightarrow k \leftrightarrow j$.

Only one curved term can be in a model, and it cannot be combined with the fixed-decay or count terms of its own family (`gwespfree()` with `gwesp()` or `esp()`, `gwdegreefree()` with `gwdegree()` or `degree()`, and so on); see Section 2.6.

4.2.3 Degree, star, triad and shared-partner terms

Option	Description
`degree(numlist)`	One coefficient per listed degree: number of nodes with that degree. Undirected.

Option	Description
`odegree(numlist)`, `idegree(numlist)`	The same for out- and in-degree. Directed.
`concurrent`	Number of nodes with degree 2 or more. Undirected.
`kstar(numlist)`	One coefficient per k: number of k-stars. Undirected.
`ostar(numlist)`, `istar(numlist)`	Out- and in-k-stars. Directed.
`degrange(numlist)` `degrangeto(numlist)`	Nodes with degree in $[from, to)$; the two lists pair by position; omit `degrangeto()` for an open upper bound. Undirected.
`odegrange() odegrangeto()`, `idegrange() idegrangeto()`	The same for out- and in-degree. Directed.
`esp(numlist)`	One coefficient per d: tied dyads with exactly d shared partners.
`dsp(numlist)`	One coefficient per d: all dyads with exactly d shared partners. An exhaustive list of d is collinear.
`triangle`	Number of triangles. Undirected.
`ctriple`	Number of cyclic triples. Directed.
`transitiveties`, `cyclicalties`	Tied arcs $i \to j$ for which a two-path $i \to k \to j$ (resp. $j \to k \to i$) exists. Directed.

4.2.4 Two-mode terms

Two-mode terms are valid on two-mode networks only; the mode digit of R's name moves to the end. Suboptions are given after a comma inside the parentheses.

Option	Description (R term)
`bcov1(varlist)`, `bcov2(varlist)`	Covariate main effect for one mode (`b1cov`, `b2cov`).
`bfactor1(varlist)`, `bfactor2(varlist)`	One coefficient per level but the first (`b1factor`, `b2factor`).
`bdegree1(numlist)`, `bdegree2(numlist)`	Nodes of the mode with the given degree (`b1degree`). Suboptions `by()`, `levels()`, `homophily`.
`bstar1(numlist)`, `bstar2(numlist)`	k-stars centered on a node of the mode (`b1star`). Suboptions `attr()`, `levels()`.
`bnodematch1(var)`, `bnodematch2(var)`	Homophily for one mode (`b1nodematch`). Suboptions `diff`, `alpha()`, `beta()`, `byattr()`, `keep()`, `levels()`.
`bgwdegree1(#)`, `bgwdegree2(#)`	Geometrically weighted degree of the mode, fixed decay (`gwb1degree`). Suboptions `attr()`, `levels()`.
`bgwdegreefree1(#)`, `bgwdegreefree2(#)`	The curved forms.
`bgwdsp1(#)`, `bgwdsp2(#)`, `bgwdspfree1(#)`, `bgwdspfree2(#)`	Geometrically weighted dyadwise shared partners of the mode, fixed and curved (`gwb1dsp`).
`bconcurrent1()`, `bconcurrent2()`	Nodes of the mode with degree 2 or more; `all` or `by(var)` (`b1concurrent`).
`bmindegree1(numlist)`, `bmindegree2(numlist)`	Nodes with degree of at least d (`b1mindegree`).
`bdegrange1()`, `bdegrange2()`	Nodes with degree in $[from, to)$ (`b1degrange`). Suboptions `to()`, `by()`, `levels()`, `homophily`.
`bcovrange1(var)`, `bcovrange2(var)`	Sum over nodes of the range of the neighbors' covariate (`b1covrange`).
`bfactordistinct1(var)`, `bfactordistinct2(var)`	Sum over nodes of the number of distinct levels among the neighbors (`b1factordistinct`). Suboption `levels()`.
`bsociality1(numlist)`, `bsociality2(numlist)`	Degree of single nodes; negative numbers exclude (`b1sociality`).
`bstarmix1(k)`, `bstarmix2(k)`	Mixing matrix of k-stars (`b1starmix`). Suboptions `attr()`, `nodiff`, `base()`.
`btwostar1(var)`, `btwostar2(var)`	Two-star census by centre and leaf attributes (`b1twostar`). Suboptions `attr2()`, `levels1()`, `levels2()`, `levels3()`, `base()`.
`coincidence()`	Number of common mode-1 neighbors for each pair of mode-2 nodes; `all` or `active(#)`.
`bdsp1(numlist)`, `bdsp2(numlist)`	Pairs of nodes of the mode with exactly d shared partners (`b1dsp`).

4.2.5 Further terms, operators and labels

Option	Description
`structural(termlist)`	Further structural terms written like R's terms, with R's argument names (`twopath`, `transitive`, `cycle(k=3 4)`, `altkstar(lambda=0.8 fixed)`, `degree(d=1 2 by=sex)`, `triangle(attr=sex diff)`, ...). Section 6.1.
`operators(oplist)`	Operator terms of R: `Sum`, `Prod`, `S`, `F`, `NodematchFilter`, `Symmetrize`, `Project`, `I`, `For`, `Label`, `Log`, `Exp`, `Offset`, `Curve`. Section 6.2.
`coeflabel(old new ...)`	Renames coefficients (R's `Label()`).
`logstat(names)`, `expstat(names)`	Replaces the statistic of each named coefficient by its natural logarithm (suboption `log0(#)` for the value of $\log 0$, default $-1/\sqrt{\epsilon}$) or by its exponential (R's `Log()`, `Exp()` applied to one term of the model). Cannot be combined with `fixdensity` or a curved term.
`submodel()`	Internal option used by `operators()`; not for direct use.
`offset(coefname # ...)`	Fixes coefficients at known values (R's `offset()` with `offset.coef`). Section 6.3.

4.2.6 Level selection

The attribute terms accept the following suboptions after a comma inside the parentheses, for example `nodematch(race, diff levels(-1))`; they apply to every variable in the same option.

Suboption	Description
`levels(numlist)`	Selects levels by position in the sorted list of distinct values, like a numeric `levels=` in R: positive numbers keep those positions, negative numbers drop them (`levels(-1)` drops the first level).
`levelvalues(numlist)`	Gives the level values themselves, in the order wanted (R's `levels=I(c(...))`).
`base(numlist)`	R's deprecated `base=`: positions to drop when `levels()` is not given. For `nodefactor()`, `nodeofactor()`, `nodeifactor()` the default is `base(1)`; `base(0)` keeps all levels (and the model is then collinear with `edges`).
`levels2(numlist)`	For `nodemix()` and `mm()`: selects the cells of the mixing matrix by position, column by column; the default drops the first cell.
`keep(numlist)`	For `nodematch()`: selects the level positions that count as matches; levels outside the selection never match.

The character-valued `levels2` of R (grouping of cells), the `form=` argument for valued networks, and the formula-valued `levels` arguments are not available.

4.3 Estimation options

4.3.1 Method

Option	Description
`estimate(mle|mple|cd)`	R's `estimate=`. `mle` (default): pseudolikelihood for a dyad-independent model, Monte Carlo maximum likelihood otherwise; `mple`: pseudolikelihood; `cd`: contrastive divergence.
`method(mple|mcmle|cd)`	Older spelling of `estimate()`; contradicting values are refused.
`mainmethod(mcmle|sa)`	R's `main.method=`: Monte Carlo maximum likelihood (default) or stochastic approximation.
`init(numlist)`	Starting values, one per coefficient, `.` or `NA` for a value to be estimated (Section 2.2).
`mpletype(glm|penalized)`	`MPLE.type`: ordinary or Firth-penalized pseudolikelihood.
`mplecheck(on|off)`	`MPLE.check`: test whether the pseudolikelihood estimate exists.

Option	Description
`mplecovariance(invhess|godambe|bootstrap)`	`MPLE.covariance.method`.

4.3.2 Monte Carlo maximum likelihood, contrastive divergence, stochastic approximation

Option	Description (default; R argument)
`mcmleiterations(#)`	Maximum number of outer iterations (60; `MCMLE.maxit`); also the maximum for contrastive divergence (`CD.maxit`). At the limit the fit is reported with a warning and `e(converged)` = 0.
`mcmletermination(confidence|hummel|hotelling|none)`	Stopping rule (`confidence`; `MCMLE.termination`).
`mcmleconfidence(#)`	Confidence level of the `confidence` rule (0.99; `MCMLE.confidence`).
`mcmlesteplength(#)`	Maximum step length (1; `MCMLE.steplength`).
`mcmlestepmargin(#)`	Margin of the Hummel step length (0.05; `MCMLE.steplength.margin`); `.` switches the Hummel step length off.
`mcmleeffectivesize(#)`	Target effective sample size of the adaptive sample of each iteration (64; `MCMLE.effectiveSize`). The fit samples until the effective sample size of the estimating equations reaches it. `0` switches the adaptive sample off: fixed-size samples of `mcmcsamplesize()` draws in every iteration, as with `MCMLE.effectiveSize=NULL`. No effect on contrastive divergence, stochastic approximation and `obsconstraints()`, which use fixed-size samples (Section 2.2.2).
`cdnsteps(#)`	Proposals per draw of contrastive divergence (8; `CD.nsteps`).
`cdsamplesize(#)`	Draws per coefficient vector (automatic: 128 times the number of free coefficients; `CD.samplesize.per_theta`).
`saburnin(#)`, `sainterval(#)`	Burn-in and interval of stochastic approximation (16384, 1024; `SA.burnin`, `SA.interval`).
`saphase3(#)`, `sagain(#)`, `sansub(#)`	Phase-3 sample size, initial gain and number of phase-2 sub-phases (1000, 0.1, 4; `SA.phase3_n`, `SA.initial_gain`, `SA.nsubphases`).

4.3.3 The Markov chain, reproducibility and performance

Option	Description (default)
`mcmcburnin(#)`	Burn-in proposals per simulation (16 times the interval, 16384; `MCMLE.burnin`). Given `mcmcinterval()` without `mcmcburnin()`, the burn-in is 16 times the interval. In the adaptive sample the burn-in is estimated from the sample and discarded, and only one interval is run explicitly before each run; the option is the burn-in of fixed-size samples (`mcmleeffectivesize(0)`) and of simulation.
`mcmcinterval(#)`	Proposals between recorded draws (1024; `MCMLE.interval`). The interval of the first iteration. Each later iteration starts from half the interval of the previous sample; thinning by two doubles it when the sample has doubled (Section 2.2.2). In a fixed-size fit it grows when the effective sample size stays low.
`mcmcsamplesize(#)`	Base size of the sample of each iteration (the larger of 256 and 32 times the number of coefficients; `MCMLE.samplesize`); the adaptive sample is extended beyond it until the target effective size is reached, and the number of draws kept is `e(mcmc_samplesize)`. With `mcmleeffectivesize(0)`, the exact number of draws per iteration (3000 by default; `MCMC.samplesize`).
`proposal(uniform|random|tnt)`	Proposal weights (`tnt`; `MCMC.prop.weights`). `random` is a synonym of `uniform`. Masked variants are used when dyads are constrained. With `tnt` on a one-mode network the default hints of `mcmcprop()` give SPDyad.
`seed(#)`	Seed of the sampler; without it the current Stata seed is used (Section 2.3).

Option	Description (default)
`mcmcprop(hints)`	R's `MCMC.prop` hint formula (`sparse+.triadic`, R's default: the SPDyad proposal with a share 0.25 of triadic steps on one-mode networks). `sparse` gives the tie/no-tie proposal alone; `triadic(triFocus=#)` another share of triadic steps (Chapter 2).
`mcmcmaxedges(#)`	R's `MCMC.maxedges`: stop sampling when the network has more ties.
`verbose`	Shows iteration detail.
`spcache`	Enables the incremental shared-partner cache for the shared-partner and triangle terms on an undirected network. Off by default: direct benchmarks found it a net loss below an average degree of roughly 30–40 and a net win above it. No effect on a directed network.
`nonative`	Forces the pure Mata backend where it exists (for testing and comparison; much slower).
`nomcmcsample`	Does not post `e(mcmcsample)`. Filling the matrix is the slowest step for a very large `mcmcsamplesize()`; `estat mcmcdiag` needs it.

The defaults follow R. The defaults of the Markov chain are those of `ergm`'s `control.ergm()` (version 4.12.0). These are an interval of 1024 proposals between draws (`MCMLE.interval`) and a burn-in of 16 times the interval, 16384 (`MCMLE.burnin`); `nwergm simulate` uses the same numbers, as `ergm`'s `control.simulate()` does. The sample size is adaptive, as in `ergm`: a base sample of the larger of 256 and 32 times the number of coefficients (`MCMLE.samplesize`) is extended until the effective sample size of the estimating equations reaches 64 (`MCMLE.effectiveSize`), the interval halves from one iteration to the next, the convergence test is `ergm`'s test with a tolerance region, and the covariance and the log-likelihood reuse the sample of the fit (Chapter 2). A fit with default settings therefore draws a few hundred networks per iteration (`e(mcmc_samplesize)`); Chapter 9 compares the estimates with `ergm`'s. Fits are several times faster than with a fixed 3000 draws (Section 11.5). `mcmleeffectivesize(0)` draws a fixed sample of `mcmcsamplesize()` draws per iteration, as with the fixed-size algorithm; a short fixed chain is `mcmleeffectivesize(0) mcmcburnin(3000) mcmcinterval(50)`. The default proposal is R's SPDyad, and the effective sample size, the burn-in test and the covariance of the convergence test are computed as in `ergm`, from the multivariate autoregressive spectral density of the estimating equations (Appendix C). SPDyad keeps a count of the shared partners of every dyad; the terms `gwesp`, `gwdsp`, `esp` and `dsp` (undirected, or directed with the types OTP and ITP) read their counts from it, as `ergm`'s terms do, which gives the same draws as counting the shared partners anew and takes less time. The difference from `ergm` that remains is that the target and the base size are capped at 32 times their starting values. The random-number streams differ in any case.

4.3.4 Log-likelihood by bridge sampling

Option	Description (default)
`nologlik`	Does not evaluate the log-likelihood of a Monte Carlo fit (`eval.loglik=FALSE`); log-likelihood, AIC and BIC are then missing.
`bridgesteps(#)`	Number of bridges (16; `bridge.nsteps`).
`bridgesamplesize(#)`	Draws per bridge (the number of draws of the fit's last iteration, as in `ergm`; with `mcmleeffectivesize(0)`, the value of `mcmcsamplesize()`).
`bridgeinterval(#)`	Thinning interval within the bridges (the interval of the fit's own sample, divided by the number of bridges).

4.3.5 Constraints

Option	Description
`constraints(formula)`	Sample-space constraints as in R's `constraints=`, several joined by `+` and `-` (Section 6.4). Needs the plugin.
`obsconstraints(formula)`	Observation-process constraints (R's `obs.constraints`). Needs the plugin and Monte Carlo maximum likelihood.
`freedyads(netname)`	Only the dyads that are ties in *netname* may vary (R's `fixallbut()`).
`blockdiag(varname)`	Only dyads within the same value of the variable may vary (R's `blockdiag()`). Cannot be combined with `freedyads()`.
`fixdensity`	Holds the number of ties fixed (R's `constraints=~edges`); `edges` is then not estimated. Requires Monte Carlo maximum likelihood and at least one term besides `edges`.

4.4 `nwergm simulate`

4.4.1 From given coefficients

```
nwergm simulate nodes, edges [terms] theta(numlist) [directed bipartite(#) nsim(#) ...]
```

draws networks from a fully specified model on *nodes* nodes; no network needs to be in memory. `theta()` gives one coefficient per resulting coefficient, in the order in which `nwergm` processes the terms (Chapter 3); a term that expands into several coefficients uses that many consecutive values. The terms are the options of the model, with fixed decay for the geometrically weighted terms; node covariates are read from the current dataset, which must have *nodes* observations. The result does not carry the covariates forward.

Option	Description (default)
`theta(numlist)`	Coefficients. Required.
`directed`	Simulate directed networks (default undirected).
`bipartite(#)`	Number of first-mode nodes of a two-mode network.
`nsim(#)`	Number of networks (1), each with a fresh burn-in. Named `generate()_1`, `generate()_2`, ... for $\# > 1$.
`generate(stub)`	Name of the networks (`ergmsim`).
`mcmcburnin(#)`, `mcmcinterval(#)`	Burn-in and interval (16384 and 1024, as in R's `control.simulate`; the burn-in is 16 times an interval given alone).
`proposal(uniform|tnt)`	Proposal (`tnt`).
`mcmcprop(hints)`	As for the fit (default SPDyad on one-mode networks). After a fit, `nwergm simulate` uses the proposal of the fit.
`seed(#)`	Seed.
`basis(netname)`, `constraints(formula)`	Starting network and sample-space constraints (one chain from the basis; needs the plugin).
`type()`, `spcache`	As for the model.

4.4.2 From the last fit

```
nwergm simulate, [nsim(#) output(network|stats|edgelist) basis(netname) nosequential
                  seed(#) burnin(#) interval(#) coef(matname) generate(name) clear]
```

Without a number of nodes, `nwergm simulate` draws from the model of the last `nwergm` fit (R's `simulate.ergm()`), starting from the last network of the fit or from `basis()`, at the fitted coefficients or at `coef()` (a row vector with one value per coefficient). The chain is sequential (burn-in once, `interval()` proposals between draws) unless `nosequential` restarts every draw from the starting network. The defaults of `burnin()` and `interval()` are those of the fit, or 16384 and 1024 for a fit without a Monte Carlo sample. `output(network)` (default) creates the networks *name*`_1`, *name*`_2`, ... (`generate()`, default `sim`); `output(stats)` returns the statistics in `r(stats)`; `output(edgelist)` replaces the data with the variables `_sim`, `_tail`, `_head` (specify `clear`). The constraints of the fit stay in force. Simulating a two-mode fit with `output(network)`

creates one-mode undirected networks on all nodes (not declared two-mode); use `output(stats)` or `output(edgelist)`.

4.5 Postestimation options

Command and option	Description (default)
`estat gof gof(statistics)`	Statistics to compare: `degree idegree odegree b1degree b2degree espartners dspartners distance triadcensus model`; a minus sign drops `model`. The default is that of R for the type of network.
`nsim(#)`, `seed(#)`	Number of simulated networks (100) and seed.
`gofburnin(#)`, `gofinterval(#)`	Burn-in and interval; default those of the fit (`e(mcmc_burnin)`, `e(mcmc_interval_final)`; 16384 and 1024 after a pseudolikelihood fit).
`plot`, `maxdeg(#)`, `maxdist(#)`, `name(string)`	Graph of the degree and geodesic distributions (maximum degree 15, maximum distance 6, name `gof`).
`estat mcmcdiag heidel`	Adds the Heidelberger–Welch test.
`report`	Adds the report of R's `mcmc.diagnostics()`.
`plot`, `name(string)`	Trace and density plots (name `mcmcdiag`).
`pvalue(#)`, `eps(#)`	Level of the Heidelberger–Welch test (one of 0.10, 0.05, 0.025, 0.01; default 0.05) and the relative half-width (0.1).
`estat ic`	Log-likelihood, AIC and BIC.
`predict pr`, `xb`	Conditional probability (default) or conditional log odds of each informative dyad.
`matrix(name)`, `frame(name)`	Result as an $n \times n$ matrix (missing dyads are missing) or as a frame with one row per dyad (`tail`, `head`, `p`). Available for the last fit only.

Chapter 5

The terms

A *term* is one statistic of the network with one coefficient (or, for a term such as `degree(1 2 3)`, one coefficient per listed value). In `nwergm` you request a term by naming it as an option, in the same way as you name a term in the formula of R's `ergm()`. This chapter lists every term of `ergm` 4.12.0 that `nwergm` implements, with the syntax, the restrictions and the evidence that it agrees with `ergm`, and then the terms that it does not implement and the reason.

5.1 Counts and how to read the tables

The package `ergm`, version 4.12.0, documents 139 terms. `nwergm` implements 125 of the 139 terms: 123 completely and 2 in part (89.9%). Of the 139 terms, 13 are valued-network terms that `ergm` itself refuses on a binary network, so that 126 terms can be used on a binary network at all, and 125 of those are implemented. The 14 terms that are not implemented are the 13 valued-network terms and the operator `B` (Section 5.9). The packages built on `ergm` document 32 further terms (15 of `tergm`, 15 of `ergm.multi`, 2 of `ergm.count`), none of which is implemented (Section 5.10). The tables below are generated from a table that records, for every term, its status and how it was validated.

Each implemented term is a row of one of the tables of Sections 5.2 to 5.8, with four columns.

ergm term
: The name of the term in `ergm`.

nwergm syntax and remarks
: The option of `nwergm` that requests it. A form that begins with `structural(` or `operators(` is written inside that option (Sections 6.1 and 6.2); where two forms are given, the first is the plain option and the second the form with the further arguments of the `ergm` term. Arguments of the `ergm` term that are not available are named in the remarks. "Partial" marks the two terms that are implemented in part.

Types, class
: The network types on which `ergm` accepts the term (`u` undirected, `d` directed, `b` two-mode), narrowed where `nwergm` accepts fewer, and the dependence class: `DI` dyad-independent (a model of such terms alone is fitted by pseudolikelihood, which is exact), `DD` dyad-dependent (Monte Carlo maximum likelihood), `curved` (curved by default in `ergm`).

Validation against ergm
: How the term was compared with `ergm` 4.12.0, in three levels of evidence: *statistic equals R's*, the observed statistic of test networks equals the value of `summary()`; *MPLE to* 10^{-k}, the pseudolikelihood coefficients are within 10^{-k} of those of `ergm(estimate="MPLE")` on test networks (standard errors agree to a relative difference of about 10^{-4}); *MCMLE within noise*, Monte Carlo maximum likelihood fits agree with those of `ergm` within the spread of the Monte Carlo error over several seeds. Chapter 9 describes the comparisons. Where an observed statistic is not listed, the term was compared through its change statistics, which

determine the pseudolikelihood.

Because the statistics of the observed network and the change statistics are compared directly, an agreement of the pseudolikelihood coefficients verifies the term *as a function of the network*; the Monte Carlo fits additionally verify the sampler. A term can agree with `ergm` and still be useless on a given network: a statistic that does not vary in the sample (no triangles in the data), or a collinear set of terms, makes the model non-identifiable in `nwergm` just as in `ergm`.

Terms that exist twice. The geometrically weighted terms have a fixed-decay form and a curved form (Section 2.6); in the tables both forms are listed in the row of the `ergm` term. A few `ergm` terms have a different option name in `nwergm`: `absdiff` is `absdist()`, `nodematch` with `diff=TRUE` is `nodematchdiff()`, and the two-mode terms move the digit of the mode (`b1degree` is `bdegree1()`, `gwb1degree` is `bgwdegree1()`).

Statistics, MPLE and the sampler. The pseudolikelihood is built from the change statistics of the term, and the sampler uses the same change statistics at every proposal; the design matrix of the plugin is identical to that of the Mata code, and the sampler is checked separately (Chapter 9).

5.2 Dyad-level and basic terms

Terms of single dyads and of the whole network: density, reciprocity, Hamming distance to another network. (5 terms)

ergm term	nwergm syntax and remarks	Types, class	Validation against ergm
`density`	`structural(density)`	u d b, DI	statistic equals R's; MPLE to 10^{-8}
`edges`	`edges` Required in every model, unless density or meandeg (collinear with it) stands in for it.	u d b, DI	statistic equals R's; MPLE to 10^{-6}
`hamming`	`hamming(netname)` Reference must be a loaded network; R's cov=/attrname=/defaultweight= not supported.	u d b, DI	MPLE to 10^{-9}
`meandeg`	`structural(meandeg)`	u d b, DI	statistic equals R's; MPLE to 10^{-8}

`mutual`	`mutual`
	`structural(mutual[(same=attr [ diff])]), structural(mutual(by= attr))`
	Directed networks only. R's keep= and levels= are not available.
		d, DD	statistic equals R's; MPLE to 10^{-5}

5.3 Degree and star terms

Terms of the degree distribution of the nodes and of the stars centred on them. (20 terms)

ergm term	nwergm syntax and remarks	Types, class	Validation against ergm
`concurrent`	`concurrent` `structural(concurrent[(by= attr)])` Undirected networks only.	u b, DD	statistic equals R's; MPLE to 10^{-9}
`concurrentties`	`structural(concurrentties[(by= attr)])`	u b, DD	statistic equals R's; MPLE to 10^{-6}
`degcor`	`structural(degcor)`	u b, DD	statistic equals R's; MPLE to 10^{-7}
`degcrossprod`	`structural(degcrossprod)`	u b, DD	statistic equals R's; MPLE to 10^{-8}
`degrange`	`degrange(numlist) degrangeto( numlist)` `structural(degrange(from=numlist [to=numlist] [by=attr] [ homophily]))` Undirected networks only.	u b, DD	statistic equals R's; MPLE to 10^{-5}
`degree`	`degree(numlist)` `structural(degree(d=numlist [by= attr] [homophily]))` Undirected networks only, as in R.	u b, DD	statistic equals R's; MPLE to 10^{-7}
`degree1.5`	`structural(degree1.5)`	u b, DD	statistic equals R's; MPLE to 10^{-7}; MCMLE within noise
`idegrange`	`idegrange(numlist) idegrangeto( numlist)` `structural(idegrange(...))` Directed only.	d, DD	statistic equals R's; MPLE to 10^{-8}
`idegree`	`idegree(numlist)` `structural(idegree(d=numlist [ by=attr] [homophily]))` Directed networks only. A level with no node that R reports as -Inf or NA is reported as 0.	d, DD	statistic equals R's; MPLE to 10^{-7}
`idegree1.5`	`structural(idegree1.5)`	d, DD	statistic equals R's; MPLE to 10^{-7}
`isolatededges`	`structural(isolatededges)`	u b, DD	statistic equals R's; MPLE to 10^{-9}
`isolates`	`structural(isolates)`	u b, DD	statistic equals R's; MPLE to 10^{-5}
`istar`	`istar(numlist)` `structural(istar(k=numlist [ attr=attr]))` Directed only.	d, DD	statistic equals R's; MPLE to 10^{-5}
`kstar`	`kstar(numlist)` `structural(kstar(k=numlist [ attr=attr]))` Undirected networks only.	u b, DD	statistic equals R's; MPLE to 10^{-6}
`m2star`	`structural(m2star)`	d, DD	statistic equals R's; MPLE to 10^{-8}; MCMLE within noise
`odegrange`	`odegrange(numlist) odegrangeto( numlist)` `structural(odegrange(...))` Directed only.	d, DD	statistic equals R's; MPLE to 10^{-6}

ergm term	nwergm syntax and remarks	Types, class	Validation against ergm
`odegree`	`odegree(numlist)` `structural(odegree(d=numlist [ by=attr] [homophily]))` Directed networks only.	d, DD	statistic equals R's; MPLE to 10^{-6}
`odegree1.5`	`structural(odegree1.5)`	d, DD	statistic equals R's; MPLE to 10^{-7}; MCMLE within noise
`ostar`	`ostar(numlist)` `structural(ostar(k=numlist [ attr=attr]))` Directed only.	d, DD	statistic equals R's; MPLE to 10^{-6}
`twopath`	`structural(twopath)`	u d b, DD	statistic equals R's; MPLE to 10^{-6}

5.4 Triad, cycle, path and shared-partner terms

Terms of triangles, cycles, two-paths and shared partners. (21 terms)

ergm term	nwergm syntax and remarks	Types, class	Validation against ergm
`asymmetric`	`structural(asymmetric[(attr=attr [diff])])`	d, DD	statistic equals R's; MPLE to 10^{-6}
`balance`	`structural(balance)` A triad-count term (types 102 and 300 of the triad census; an odd number of ties in an undirected triad), not a signed-network term.	u d b, DD	statistics, MPLE and MCMLE
`ctriple`	`ctriple` `structural(ctriple[(attr=attr [ diff])])` Directed networks only.	d, DD	statistic equals R's; MPLE to 10^{-7}
`cycle`	`structural(cycle(k=numlist [ semi]))`	d, DD	statistic equals R's; MPLE to 10^{-6}; MCMLE within noise
`cyclicalties`	`cyclicalties` Directed only (R also accepts undirected).	d, DD	MPLE to 10^{-9}
`dsp`	`dsp(numlist) [type()]` R's type= -> type() option (applies to the whole model). An exhaustive d-range is collinear (as in R).	u d b, DD	MPLE to 10^{-8}
`esp`	`esp(numlist) [type()]` R's type= -> type() option (applies to the whole model).	u d b, DD	MPLE to 10^{-8}
`intransitive`	`structural(intransitive)`	d, DD	statistic equals R's; MPLE to 10^{-8}
`localtriangle`	`structural(localtriangle(x= netname))`	u d b, DD	statistic equals R's on two networks
`nearsimmelian`	`structural(nearsimmelian)`	d, DD	statistic equals R's; MPLE to 10^{-10}
`nsp`	`structural(nsp(d=numlist [type= OTP|ITP|OSP|ISP|RTP]))`	u d b, DD	statistic equals R's; MPLE to 10^{-5}

ergm term	nwergm syntax and remarks	Types, class	Validation against ergm
`opentriad`	`structural(opentriad)`	u b, DD	statistic equals R's; MPLE equals that of ergm 4.13.0-8255 (development version)
`simmelian`	`structural(simmelian)`	d, DD	statistic equals R's; MPLE to 10^{-8}
`simmelianties`	`structural(simmelianties)`	d, DD	statistic equals R's; MPLE to 10^{-8}
`threetrail`	`structural(threetrail[(keep=numlist)])`	u d b, DD	statistic equals R's; MPLE to 10^{-7}
`transitive`	`structural(transitive)`	d, DD	statistic equals R's; MPLE to 10^{-6}; MCMLE within noise
`transitiveties`	`transitiveties` Directed only (R also accepts undirected).	d, DD	MPLE to 10^{-7}
`triadcensus`	`structural(triadcensus[(levels=numlist)])`	u d b, DD	statistic equals R's; MPLE to 10^{-7}
`triangle`	`triangle` `structural(triangle[(attr=attr [diff])])`	u d b, DD	statistic equals R's; MPLE to 10^{-5}; MCMLE within noise
`tripercent`	`structural(tripercent[(attr=attr [diff])])`	u b, DD	statistic equals R's (to 10^{-6}); MPLE to 10^{-7}
`ttriple`	`structural(ttriple[(attr=attr [diff])])`	d, DD	statistic equals R's; MPLE to 10^{-6}; MCMLE within noise

5.5 Curved terms: geometrically weighted and alternating statistics

Terms with a decay parameter. The fixed-decay form (decay as a number in the option) is an ordinary term; the form with an estimated decay (option names ending in `free`) is a curved exponential family and reports a weight and a decay. (7 terms)

ergm term	nwergm syntax and remarks	Types, class	Validation against ergm

`altkstar`	`structural(altkstar(lambda=# fixed))`
	`structural(altkstar) with init()`
	partial. Estimated lambda (R: fixed = FALSE, the default) is available for method(mple) with init() only: R cannot fit it with its default starting values and its MCMLE does not converge on the test networks, so nothing exists to compare an MCMLE with.
		u b, curved	fixed form: statistic and MPLE; curved form: map, gradient and MPLE from given starting values (to 10^{-5})

`gwdegree`	`gwdegree(#)`
	`gwdegreefree(#)`
	Fixed decay gwdegree(), curved gwdegreefree() (undirected). nwergm also accepts gwdegree() on a directed network, which R refuses.
		u, curved	statistic equals R's; MPLE to 10^{-5} except the cases named in the remark

ergm term	nwergm syntax and remarks	Types, class	Validation against ergm

`gwdsp`	`gwdsp(#)`
	`gwdspfree(#)`
	`type()`
	Fixed decay gwdsp(), curved gwdspfree(). The curved fit on a boundary solution can end elsewhere than R.
		u d b, curved	MPLE to 10^{-8} except the cases named in the remark

`gwesp`	`gwesp(#)`
	`gwespfree(#)`
	`type(OTP|ITP|OSP|ISP|RTP)`
	R's default is fixed=FALSE; in nwergm gwesp() has a fixed decay and gwespfree() estimates it. RTP agrees with R 4.12.0 only when R is run with cache.sp=FALSE (a defect of that release). R's cutoff= is not available. The curved fit on a boundary solution (decay near 0) can end elsewhere than R.
		u d b, curved	statistic equals R's; MPLE to 10^{-6} except the cases named in the remark

`gwidegree`	`gwidegree(#)`
	`gwidegreefree(#)`
	Directed networks only. Fixed decay gwidegree(), curved gwidegreefree(). The curved fit on a boundary solution can end elsewhere than R.
		d, curved	statistic equals R's; MPLE to 10^{-9} except the cases named in the remark

`gwnsp`	`gwnsp(#)`
	`gwnspfree(#)`
	`type()`
	Fixed decay gwnsp(), curved gwnspfree(). The curved fit on a boundary solution can end elsewhere than R.
		u d b, curved	MPLE to 10^{-7} except the cases named in the remark

`gwodegree`	`gwodegree(#)`
	`gwodegreefree(#)`
	Directed networks only. Fixed decay gwodegree(), curved gwodegreefree().
		d, curved	statistic equals R's; MPLE to 10^{-3}

5.6 Covariate terms

Terms of node attributes and of dyadic covariates. (25 terms)

ergm term	nwergm syntax and remarks	Types, class	Validation against ergm
`absdiff`	`absdist(varlist [, pow(#)])` R's name is absdiff; the exponent is pow().	u d b, DI	MPLE to 10^{-7}
`absdiffcat`	`absdiffcat(varlist [, levels() levelvalues() base()])`	u d b, DI	statistic equals R's; MPLE to 10^{-4}
`attrcov`	`attrcov(varlist, mat(matname))`	u d, DI	statistic equals R's; MPLE to 10^{-7} (compared for this manual)
`diff`	`diff(varlist [, pow() dir() signaction()])`	u d b, DI	statistic and change statistics equal R's; MPLE to 10^{-4}

ergm term	nwergm syntax and remarks	Types, class	Validation against ergm
`dyadcov`	`dyadcov(netname | matname)`	u d, DI	statistic equals R's; MPLE to 10^{-9} (compared for this manual)
`edgecov`	`edgecov(netname | matname)` The covariate is a network in memory (possibly valued) or an n by n Stata matrix.	u d, DI	statistic equals R's; MPLE to 10^{-9}
`mm`	`mm(var [, levels() levels2()])`	u d b, DI	statistic equals R's; MPLE to 10^{-9} (compared for this manual)
`nodecov`	`nodecov(varlist)`	u d b, DI	statistic equals R's; MPLE to 10^{-6}
`nodecovrange`	`nodecovrange(varlist)`	u d b, DD	statistic equals R's; MPLE to 10^{-4}
`nodefactor`	`nodefactor(varlist [, levels() levelvalues() base()])` The lowest level is omitted (R's default base=1).	u d b, DI	MPLE to 10^{-7}
`nodefactordistinct`	`nodefactordistinct(varlist [, levels() levelvalues()])`	u d b, DD	statistic equals R's; MPLE to 10^{-4}
`nodeicov`	`nodeicov(varlist)` Directed only.	d, DI	statistic equals R's; MPLE to 10^{-7}
`nodeicovrange`	`nodeicovrange(varlist)`	d, DD	statistic equals R's; MPLE to 10^{-10} (compared for this manual)
`nodeifactor`	`nodeifactor(varlist [, levels() levelvalues() base()])` Directed networks only.	d, DI	MPLE to 10^{-7}
`nodeifactordistinct`	`nodeifactordistinct(varlist [, levels() levelvalues()])`	d, DD	statistic equals R's; MPLE to 10^{-4}

`nodematch`	`nodematch(varlist [, diff keep() levels() levelvalues()])`
	`nodematchdiff(varlist)`
	Exact match on a numeric variable (encode strings first); one coefficient per variable, or per level with diff.
		u d b, DI	statistic equals R's; MPLE to 10^{-7}

`nodemix`	`nodemix(varlist [, levels() levelvalues() levels2() base()])`	u d b, DI	statistic and change statistics equal R's; MPLE to 10^{-4}
`nodeocov`	`nodeocov(varlist)` Directed only.	d, DI	statistic equals R's; MPLE to 10^{-6}
`nodeocovrange`	`nodeocovrange(varlist)`	d, DD	statistic equals R's; MPLE to 10^{-4}
`nodeofactor`	`nodeofactor(varlist [, levels() levelvalues() base()])` Directed networks only.	d, DI	MPLE to 10^{-7}
`nodeofactordistinct`	`nodeofactordistinct(varlist [, levels() levelvalues()])`	d, DD	statistic equals R's; MPLE to 10^{-9} (compared for this manual)
`receiver`	`receiver` Directed only.	d, DI	MPLE to 10^{-8}
`sender`	`sender` Directed only. First node dropped as in R. (R/nwergm differ on separated nodes: R -Inf, nwergm 0.)	d, DI	MPLE to 10^{-9}

ergm term	nwergm syntax and remarks	Types, class	Validation against ergm
`smalldiff`	`smalldiff(varlist, cutoff(#))`	u d b, DI	statistic equals R's; MPLE to 10^{-4}
`sociality`	`sociality(varlist [, nodes() levels()])`	u b, DI	statistic equals R's; MPLE to 10^{-4}

5.7 Two-mode (bipartite) terms

Terms for two-mode networks. R's mode digit moves to the end of the option name (`b1degree` is `bdegree1()`). (33 terms)

ergm term	nwergm syntax and remarks	Types, class	Validation against ergm
`b1concurrent`	`bconcurrent1(all) | bconcurrent1(by(var) [levels( numlist)])`	b, DD	statistic and change statistics equal R's; MPLE to 10^{-4}; MCMLE within noise
`b1cov`	`bcov1(varlist)`	b, DI	MPLE to 10^{-6}
`b1covrange`	`bcovrange1(varname)`	b, DD	statistic and change statistics equal R's; MPLE to 10^{-4}; MCMLE within noise
`b1degrange`	`bdegrange1(numlist, to(numlist) [by() levels() homophily])`	b, DD	statistic and change statistics equal R's; MPLE to 10^{-4}; MCMLE within noise
`b1degree`	`bdegree1(numlist [, by() levels( ) homophily])` R's by=, levels=, homophily= are suboptions.	b, DD	MPLE to 10^{-7}
`b1dsp`	`bdsp1(numlist)`	b, DD	statistic and change statistics equal R's; MPLE to 10^{-4}; MCMLE within noise
`b1factor`	`bfactor1(varlist)`	b, DI	MPLE to 10^{-7}
`b1factordistinct`	`bfactordistinct1(var [, levels( )])`	b, DD	statistic and change statistics equal R's; MPLE to 10^{-4}; MCMLE within noise
`b1mindegree`	`bmindegree1(numlist)`	b, DD	statistic and change statistics equal R's; MPLE to 10^{-4}
`b1nodematch`	`bnodematch1(var [, diff alpha(#) beta(#) byattr(var) keep() levels()])`	b, DD	statistic and change statistics equal R's; MPLE to 10^{-4}; MCMLE within noise
`b1sociality`	`bsociality1(numlist)`	b, DI	statistic and change statistics equal R's; MPLE to 10^{-4}; MCMLE within noise
`b1star`	`bstar1(numlist [, attr() levels( )])` R's attr= and levels= are suboptions.	b, DD	MPLE to 10^{-7}

ergm term	nwergm syntax and remarks	Types, class	Validation against ergm
b1starmix	bstarmix1(k [, attr(var) nodiff base()])	b, DD	statistic and change statistics equal R's; MPLE to 10^{-4}; MCMLE within noise
b1twostar	btwostar1(var [, attr2(var) levels1() levels2() levels3() base()])	b, DD	statistic and change statistics equal R's; MPLE to 10^{-4}; MCMLE within noise
b2concurrent	bconcurrent2(all) | bconcurrent2(by(var) [levels(numlist)])	b, DD	statistic and change statistics equal R's; MPLE to 10^{-4}; MCMLE within noise
b2cov	bcov2(varlist)	b, DI	MPLE to 10^{-6}
b2covrange	bcovrange2(varname)	b, DD	statistic and change statistics equal R's; MPLE to 10^{-4}; MCMLE within noise
b2degrange	bdegrange2(numlist, to(numlist) [by() levels() homophily])	b, DD	statistic and change statistics equal R's; MPLE to 10^{-4}; MCMLE within noise
b2degree	bdegree2(numlist [, by() levels( ) homophily]) R's by=, levels=, homophily= are suboptions.	b, DD	MPLE to 10^{-7}
b2dsp	bdsp2(numlist)	b, DD	statistic and change statistics equal R's; MPLE to 10^{-4}; MCMLE within noise
b2factor	bfactor2(varlist)	b, DI	MPLE to 10^{-7}
b2factordistinct	bfactordistinct2(var [, levels( )])	b, DD	statistic and change statistics equal R's; MPLE to 10^{-4}; MCMLE within noise
b2mindegree	bmindegree2(numlist)	b, DD	statistic and change statistics equal R's; MPLE to 10^{-4}
b2nodematch	bnodematch2(var [, diff alpha(#) beta(#) byattr(var) keep() levels()])	b, DD	statistic and change statistics equal R's; MPLE to 10^{-4}; MCMLE within noise
b2sociality	bsociality2(numlist)	b, DI	statistic and change statistics equal R's; MPLE to 10^{-4}; MCMLE within noise
b2star	bstar2(numlist [, attr() levels( )]) R's attr= and levels= are suboptions.	b, DD	MPLE to 10^{-7}
b2starmix	bstarmix2(k [, attr(var) nodiff base()])	b, DD	statistic and change statistics equal R's; MPLE to 10^{-4}; MCMLE within noise
b2twostar	btwostar2(var [, attr2(var) levels1() levels2() levels3() base()])	b, DD	statistic and change statistics equal R's; MPLE to 10^{-4}; MCMLE within noise

ergm term	nwergm syntax and remarks	Types, class	Validation against ergm
`coincidence`	`coincidence(all) | coincidence( active(#))`	b, DD	statistic and change statistics equal R's; MPLE to 10^{-4}
`gwb1degree`	`bgwdegree1(# [, attr() levels( )])` `bgwdegreefree1(#)` Fixed decay bgwdegree1(), curved bgwdegreefree1().	b, curved	MPLE to 10^{-6}; MCMLE within noise
`gwb1dsp`	`bgwdsp1(#)` `bgwdspfree1(#)` Fixed decay bgwdsp1(), curved bgwdspfree1().	b, DD	MPLE to 10^{-6}
`gwb2degree`	`bgwdegree2(# [, attr() levels( )])` `bgwdegreefree2(#)` Fixed decay bgwdegree2(), curved bgwdegreefree2().	b, curved	MPLE to 10^{-6}; MCMLE within noise
`gwb2dsp`	`bgwdsp2(#)` `bgwdspfree2(#)` Fixed decay bgwdsp2(), curved bgwdspfree2().	b, DD	MPLE to 10^{-6}

5.8 Operators

Operators build a term from other terms or from a derived network; they are written inside `operators()` (Section 6.2). (14 terms)

ergm term	nwergm syntax and remarks	Types, class	Validation against ergm
`Curve`	`operators(Curve(terms, params() map(rep | matrix(name))))` *partial.* Linear maps only (map(rep) and map(matrix(name)), estimated as the statistics L'g(y)); R map functions, gradient, minpar, maxpar and cov are not available.	u d b, op.	statistics, MPLE (to 10^{-9}), MCMLE
`Exp`	`operators(Exp(terms))`	u d b, op.	statistics, MPLE and MCMLE (several models)
`F`	`operators(F(terms, filter()))`	u d b, op.	statistics, MPLE and MCMLE (several models)
`For`	`operators(For(name=values, terms))`	u d b, op.	statistics, MPLE and MCMLE (several models)
`I`	`operators(I(terms))`	u d b, op.	statistics, MPLE and MCMLE (several models)
`Label`	`operators(Label(terms, label() pos()))`	u d b, op.	statistics, MPLE and MCMLE (several models)
`Log`	`operators(Log(terms, log0()))`	u d b, op.	statistics, MPLE and MCMLE (several models)
`NodematchFilter`	`operators(NodematchFilter(terms, attrname()))`	u d b, op.	statistics, MPLE and MCMLE (several models)
`Offset`	`operators(Offset(terms, coef() which())) and offset(coefname #)`	u d b, op.	statistics, MPLE and MCMLE (several models)

ergm term	nwergm syntax and remarks	Types, class	Validation against ergm
`Prod`	`operators(Prod(terms | terms, label()))` R computes Prod as Exp(Sum(Log)) with log(0) = -1/sqrt(eps); its statistics are exact to about 1e-8 only. Nested Sum/Prod inside Prod is not available.	u d b, op.	statistics, MPLE and MCMLE (several models)
`Project`	`operators(Project(terms, mode( )))`	b, op.	statistics, MPLE and MCMLE (several models)
`S`	`operators(S(terms, attrs()))`	u d, op.	statistics, MPLE and MCMLE (several models)
`Sum`	`operators(Sum(terms | terms, label()))`	u d b, op.	statistics, MPLE and MCMLE (several models)
`Symmetrize`	`operators(Symmetrize(terms, rule()))` rule(weak) on a network with reciprocated ties: R 4.12.0 starts from a symmetrized network that lacks the reciprocated pairs, so its estimation differs from its summary(); nwergm uses the exact change statistics.	d, op.	statistics, MPLE and MCMLE (several models)

5.9 Terms that are not implemented

Of the 139 terms of `ergm` 4.12.0, 14 are not implemented: the 13 terms that are defined only for *valued* networks, and the operator `B`, which wraps binary terms for use in a valued model. `nwergm` models binary ties; the values of a valued network are not used as an outcome, and a valued network is rejected with an error. Valued ERGMs are a different statistical framework (Krivitsky [15]), the count and valued models of the package `ergm.count`, and are not a small extension. Every other term of `ergm` is implemented, completely or in part. The two terms that are implemented in part are the operator `Curve` (linear maps of the coefficients only: R's map functions, gradient, `minpar`, `maxpar` and `cov` are not available) and `altkstar` (the curved form with an estimated λ is available for the pseudolikelihood with starting values from `init()`; R cannot fit it with its default starting values, and its Monte Carlo maximum likelihood does not converge on the test networks, so there is nothing to compare a Monte Carlo fit with).

ergm term	What it does in ergm	Why nwergm has no counterpart
`atleast`	Number of dyads with values greater than or equal to a threshold	term for valued networks (ergm.count framework); nwergm models binary ties only
`atmost`	Number of dyads with values less than or equal to a threshold	term for valued networks (ergm.count framework); nwergm models binary ties only
`B`	Wrap binary terms for use in valued models	wraps binary terms for use in a valued model
`cdf`	Empirical cumulative distribution function (unnormalized) of the network's dyad values	term for valued networks (ergm.count framework); nwergm models binary ties only
`cyclicalweights`	Cyclical weights	term for valued networks (ergm.count framework); nwergm models binary ties only
`equalto`	Number of dyads with values equal to a specific value (within tolerance)	term for valued networks (ergm.count framework); nwergm models binary ties only
`greaterthan`	Number of dyads with values strictly greater than a threshold	term for valued networks (ergm.count framework); nwergm models binary ties only
`ininterval`	Number of dyads whose values are in an interval	term for valued networks (ergm.count framework); nwergm models binary ties only

ergm term	What it does in ergm	Why nwergm has no counterpart
`nodecovar`	Covariance of undirected dyad values incident on each actor	term for valued networks (ergm.count framework); nwergm models binary ties only
`nodeicovar`	Covariance of in-dyad values incident on each actor	term for valued networks (ergm.count framework); nwergm models binary ties only
`nodeocovar`	Covariance of out-dyad values incident on each actor	term for valued networks (ergm.count framework); nwergm models binary ties only
`smallerthan`	Number of dyads with values strictly smaller than a threshold	term for valued networks (ergm.count framework); nwergm models binary ties only
`sum`	Sum of dyad values (optionally taken to a power)	term for valued networks (ergm.count framework); nwergm models binary ties only
`transitiveweights`	Transitive weights	term for valued networks (ergm.count framework); nwergm models binary ties only

5.10 Terms of the packages built on ergm

The table also records the terms that the packages built on `ergm` document. None of them is implemented. The terms of `tergm` model dynamic networks (separable temporal ERGMs, Krivitsky and Handcock [6]); the terms of `ergm.multi` model several networks at once; the terms of `ergm.count` are the valued-network terms of the count framework. For longitudinal network data the `nwcommands` family has other models, which are not ERGMs: `nwsaom` for stochastic actor-oriented models, `nwdynam` and `nwrem` for relational event data.

Term	Package	What it does
`CMB`	ergm.count	Conway-Maxwell-Binomial Distribution
`CMP`	ergm.count	Conway-Maxwell-Poisson Distribution
`b1dspL`	ergm.multi	Dyadwise shared partners for dyads in the first bipartition on layers
`b2dspL`	ergm.multi	Dyadwise shared partners for dyads in the second bipartition on layers
`CMBL`	ergm.multi	Conway–Maxwell-Binomial dependence among layers
`ddspL`	ergm.multi	Dyadwise shared partners on layers
`despL`	ergm.multi	Edgewise shared partners on layers
`dgwdspL`	ergm.multi	Geometrically weighted dyadwise shared partner distribution on layers
`dgwespL`	ergm.multi	Geometrically weighted edgewise shared partner distribution on layers
`dgwnspL`	ergm.multi	Geometrically weighted non-edgewise shared partner distribution on layers
`dnspL`	ergm.multi	Non-edgewise shared partners and paths on layers
`gwb1dspL`	ergm.multi	Geometrically weighted dyadwise shared partner distribution for dyads in the first bipartition on layers
`gwb2dspL`	ergm.multi	Geometrically weighted dyadwise shared partner distribution for dyads in the second bipartition on layers
`L`	ergm.multi	Evaluation on layers
`mutualL`	ergm.multi	Mutuality
`N`	ergm.multi	Evaluation on multiple networks
`twostarL`	ergm.multi	Multilayer two-star
`Change`	tergm	The Change Operator Term
`Cross`	tergm	The Crossection Operator Term
`degrange.mean.age`	tergm	Average age of ties incident on nodes having degree in a given range
`degree.mean.age`	tergm	Average age of ties incident on nodes having a given degree
`Diss`	tergm	The Dissolution Operator Term
`edge.ages`	tergm	Sum of ages of extant ties
`EdgeAges`	tergm	The EdgeAges Operator Term
`edgecov.ages`	tergm	Weighted sum of ages of extant ties
`edgecov.mean.age`	tergm	Weighted average age of an extant tie
`edges.ageinterval`	tergm	Number of edges with age falling into a specified range
`Form`	tergm	The Formation Operator Term
`mean.age`	tergm	Average age of an extant tie

Term	Package	What it does
`nodefactor.mean.age`	tergm	Average ages of extant half-ties incident on nodes of specified attribute levels
`nodemix.mean.age`	tergm	Average ages of extant ties of specified mixing types
`Persist`	tergm	The Persistence Operator Term

Chapter 6

Structural terms, operators, constraints and simulation

This chapter describes the parts of `nwergm` that go beyond the plain options of the terms: the terms written like R's terms inside `structural()`, the operators of `ergm` inside `operators()`, fixed coefficients, constraints and observation processes, and simulation. Examples with output are in Chapter 8.

6.1 Structural terms: `structural()`

```
nwergm [netname], edges structural(termlist) [other options]
nwergm simulate nodes, edges structural(termlist) theta() [other options]
```

termlist is a list of terms written like R's terms, separated by blanks: *term* or *term*`(`*arguments*`)`. Arguments have R's names and defaults (*name*`=`*value*); values are numbers, variable names, network names, or lists of numbers separated by blanks (R's `c(1,2,3)` is written `d=1 2 3`), and R's logical arguments are written as bare words (`diff`, `fixed`, `semi`, `homophily`) or *name*`=TRUE`. The first argument may be given without its name, as in R. For example,

```
. nwergm mynet, edges structural(twopath triangle(attr=sex diff) cycle(k=3 4) altkstar(lambda
    =0.8 fixed))
```

A node attribute is a numeric variable of the current dataset, one row per node. The coefficients are named after R's coefficient names with `_` for the period, for example `triangle_sex_1`, `cycle3`, `odeg1_sex_2`, `threetrail_RRR`. Terms that depend on more than the covariates of a dyad make the model dyad-dependent, so that Monte Carlo maximum likelihood is the default. `density` and `meandeg` are collinear with `edges`; `edges` may be left out when one of them is used instead. `structural()` runs in the native plugin and has no slower fallback.

The terms (their definition is that of `ergm`):

Undirected networks only

`isolatededges`, `degree1.5`, `degcrossprod`, `degcor`, `concurrentties`[`(by=`*attr*`)`], `concurrent`[`(by=`*attr*`)`], `altkstar(lambda=`*#* `fixed)`, `tripercent`[`(attr=`*attr*[`diff`]`)`], `opentriad`, `degree(d=`*numlist*[`by=`*attr*][`homophily`]`)`, `degrange(from=`*numlist*[`to=`*numlist*][`by=`*attr*][`homophily`]`)`, `kstar(k=`*numlist*[`attr=`*attr*]`)`.

Directed networks only

`m2star`, `odegree1.5`, `idegree1.5`, `asymmetric`[`(attr=`*attr*[`diff`]`)`], `mutual`[`(same=`*attr*[`diff`]`)`], `mutual(by=`*attr*`)`, `transitive`, `intransitive`, `ttriple`[`(attr=`*attr*[`diff`]`)`], `ctriple`[`(attr=`*attr*[`diff`]`)`], `nearsimmelian`, `simmelian`, `simmelianties`, `odegree(d=...)`, `idegree(...)`, `odegrange(...)`, `idegrange(...)`, `ostar(k=`*numlist*[`attr=`*attr*]`)`, `istar(...)`.

Both kinds of network

`density`, `meandeg`, `twopath`, `isolates`, `cycle(k=`*numlist*`[ semi])` (semicycles for directed networks), `triangle`[`(attr=`*attr*`[ diff])`], `threetrail`[`(keep=`*numlist*`)`] (the four types RRR, RRL, LRR, LRL for directed networks), `balance` (balanced triads: of type 102 or 300 in a directed network, with an odd number of ties in an undirected one), `triadcensus`[`(levels=`*numlist or labels*`)`], `localtriangle(x=`*netname*`)` (a binary network with the same nodes), `nsp(d=`*numlist*`[ type=`*OTP/ITP/OSP/ISP/RTP*`])`.

Where the result is not simply R's. (1) `degcor` and `degcrossprod` are scaled by quantities of the observed network, as in R, and are therefore not available in `nwergm simulate`. (2) `altkstar` without `fixed` is R's default, the curved form with an estimated λ. It is estimated by the pseudolikelihood with starting values given by `init(`*edges weight lambda*`)`, as R needs them to fit this term, and has the coefficients `altkstar` and `altkstar_lambda`; `altkstar(lambda=# fixed)` is the fixed form. (3) The `levels=` argument of the attribute terms is not available: all levels of the attribute are used.

6.2 Operators: `operators()`

```
nwergm [netname], edges operators(oplist) [other options]
```

oplist is a list of operator calls separated by blanks, written like R's operator terms: *Name*(*terms*[`,` *suboptions*]); the names are not case sensitive. The *terms* inside an operator are written as options of `nwergm`, for example `triangle nodematch(sex) gwesp(0.5) kstar(2)` or `structural(twopath)`; they give the statistics of the operator in the order in which they are written, as in an R formula. An operator can be used inside another operator, for example `Sum(F(triangle, filter(nodematch(sex))) | S(triangle, attrs(age>30)), label(a))`. The coefficient names are those of R (`Sum~a`, `S(age>30)~triangle`, `Symmetrize(strong)~triangle`, `Proj1~edges`) with the period written as an underscore. An operator makes the model dyad-dependent unless its terms are dyad-independent and the operator itself introduces no dependence; Monte Carlo maximum likelihood is then the default. All operators run in the native plugin; there is no slower fallback.

Operator	What it does
`Sum(`f_1 [`|` f_2 ...], `label())`	Sum of the statistics of one or more formulas, element by element. A formula can start with its weights and a colon: a number (`2: triangle`), one weight per statistic, `matrix(`*name*`):`, or `sum:` / `mean:`, which combine all statistics of the formula into one. `label()` is required: one name, to which the index is appended, or one name per resulting statistic.
`Prod(`f_1 [`|` f_2 ...], `label())`	Product of the statistics of the formulas; weights are exponents; the presets are `prod:` and `geomean:`. The statistics must not be negative. Formulas inside `Prod()` cannot contain `Sum()` or `Prod()`.
`S(`*terms*`, attrs(`*expr*`))`	Terms evaluated on the subgraph induced by the nodes for which *expr* (a variable, or a condition such as `age>30`) is true (R's one-sided `attrs`; the two-sided form that selects different node sets for tails and heads is not available). One-mode networks only.
`F(`*terms*`, filter(`*term*`))`	Terms evaluated on the network from which the ties are removed for which the filter term is zero. The filter is one dyad-independent term with one statistic; `!` in front of it, or a comparison after it (`absdist(age) > 3`, with `== != > < >= <=`), selects the ties kept.
`NodematchFilter(`*terms*`, attrname(`*varlist*`))`	Terms evaluated on the network without the ties between nodes that differ in one of the attributes.
`Symmetrize(`*terms*`, rule(`*rule*`))`	Terms evaluated on the undirected network obtained from a directed one; *rule* is `weak` (default, or `max`), `strong` (`min`), `upper` or `lower`. Directed networks only.

Operator	What it does
`Project(`*terms*`, mode(#))`	Terms evaluated on the one-mode network of the nodes of mode # (1 or 2), in which two nodes are tied if they have a partner in common; the terms see this network of ties and non-ties (R: `Proj1(~B(`*terms*`, "nonzero"))`), not the counts of common partners. `Proj1()` and `Proj2()` are short forms. Two-mode networks only.
`I(`*terms*`)`	The terms themselves; useful to paste a list of terms kept in a local macro.
`For(`*name*`=`*values*`[, ...], `*terms*`)`	The terms repeated for each value; `{`*name*`}` in the terms is replaced by the value. With several loops the first is the outermost. Example: `For(v = sex region, nodematch({v}))`.
`Label(`*terms*`, label() [pos()])`	The terms with other coefficient names; *pos* is `paren` (default: `label(coefficient)`), `prepend`, `append` or `replace`.
`Log(`*terms*`[, log0(#)])`	Natural logarithm of the statistics; `log0()` (default $-1/\sqrt{\epsilon}$) stands in for the logarithm of zero.
`Exp(`*terms*`)`	Exponential function of the statistics.
`Curve(`*terms*`, params(`*names*`) map(`*map*`))`	The coefficients of the terms are a linear function of the parameters, $\eta = L\theta$, where *map* is `rep` (recycle the parameters over the coefficients) or `matrix(`*name*`)` (a matrix L with one row per coefficient and one column per parameter). The model is estimated for the parameters, which carry the names given in `params( )`. `Parametrise()` and `Parametrize()` are the same operator. Map functions, `gradient()`, `minpar()`, `maxpar()` and `cov()` of R are not available.
`Offset(`*terms*`[, coef(`*numlist*`) which(`*selection*`)])`	Coefficients held at fixed values (default 0) instead of being estimated; `which()` selects them by position or coefficient name, default all. The fixed coefficients are reported with standard error 0.

Remarks. The operators work with the pseudolikelihood, with Monte Carlo maximum likelihood, with `nwergm simulate` from a fit and with `estat gof`. They cannot be combined with `constraints()`, `obsconstraints()`, `freedyads()`, `blockdiag()` or `fixdensity`, and a curved term cannot be used inside an operator. Dyadic covariates (`edgecov()`, `dyadcov()`, ...) cannot be used inside `S()` or `Project()`, which change the set of nodes. For `Symmetrize()` with `rule(weak)` on a network with reciprocated ties, the pseudolikelihood is formed from the exact change statistics of the symmetrized network, which is not what `ergm` 4.12.0 does (Appendix B).

6.3 Offsets: fixed coefficients

`offset(`*coefname* `# [`*coefname* `# ...])` holds one or more named coefficients at known values instead of estimating them. It is useful for imposing a theoretically motivated effect size, for reproducing a published model exactly, or for holding a nuisance term constant. The *coefname* must be a coefficient that the model would otherwise report; the term still has to be requested by its option (`nodecov(wealth)`), and `offset(nodecov_wealth 0.01)` then fixes its coefficient: `offset()` names an existing coefficient, it does not add a second copy of the term. Several coefficients may be fixed, `offset(mutual 2 gwesp_0.5 0.5)`. Both estimators accept offsets (Section 2.2); at least one coefficient must remain free; offsets are not supported for the weight and decay of a curved term. The fixed coefficient is reported at exactly the given value with standard error 0 and a zero row and column in `e(V)`. The operator `Offset()` (Section 6.2) does the same for the terms inside it. Section 8.1.1 has an example.

6.4 Constraints and observation processes

The formula of `constraints()` is a list of the following constraint terms joined by `+` or `-` (Section 2.5). Arguments are written as in R, by position or as *name*=*value*; node attributes are variables of the current dataset, and networks are names of networks in memory. The constraints refer to the network being fitted (for `nwergm simulate`, to `basis()`). The constraints need the native plugin.

Constraint	Effect
`edges`	Keep the number of ties.
`degrees` (`nodedegrees`)	Keep each node's degree (undirected) or in- and out-degree (directed).
`odegrees`, `idegrees`	Keep out-degrees, in-degrees (directed networks).
`b1degrees`, `b2degrees`	Keep the degrees of mode-1, mode-2 nodes (two-mode networks).
`degreedist`, `idegreedist`	Keep the degree distribution, the in-degree distribution.
`bd(`*attribs, maxout, maxin, minout, minin*`)`	Bound degrees. *attribs* is a list of 0/1 node variables defining the alter groups that are counted (default: all nodes); each bound is a number or node variable(s); defaults are 0 (minimum) and the number of nodes minus one (maximum).
`blocks(`*attr, b1levels, b2levels, levels, levels2*`)`	Fix blocks of dyads defined by the mixing of a categorical node variable. *levels2* chooses the fixed mixing types: `TRUE` (all), `FALSE` (none, the default), or their positions `c(1,2)`, `-c(1,2)` in R's ordering.
`blockdiag(`*attr, noncontig*`)`	Only dyads within the same value of *attr* may vary; *noncontig* is `merge` (default; the values must form runs, as in R) or `split`.
`fixedas(`*fixed.dyads, present, absent*`)`	Fix the dyads that are ties in the given networks; the present and absent dyads must match the network being fitted.
`fixallbut(`*free.dyads*`)`	Only the dyads that are ties in the given network may vary.
`observed`	Only the missing dyads may vary. With no missing dyad the constraint is empty and the model is fitted without it, as in R. In `constraints()` R itself fails on a network with missing dyads; use `obsconstraints(observed)` there.
`egocentric(`*attr, direction*`)`	Fix the dyads of nodes for which the 0/1 node variable *attr* is nonzero; *direction* is `both` (default), `out` or `in`.
`dyadnoise(`*p01, p10*`)`	Soft constraint: the sampled network is an observation of the given one with independent dyad errors of those probabilities.

The constraint `hamming` stops with the same message as R's current release (it is broken there). `ChangeStats()`, `Dyads()` and `I()` are not available. `odegreedist` and the degree-distribution constraints on two-mode networks are refused, because R's released proposals for them fail.

Examples of the formula.

```
constraints(edges)
constraints(odegrees)
constraints(blockdiag(g) + bd(maxout=5))
constraints(blocks(g, levels2=TRUE) - blockdiag(g))
obsconstraints(fixallbut(unobserved))
```

The examples of Sections 8.7 and 8.7.1 run some of them.

6.5 Simulation

`nwergm simulate` draws networks from a model, in two ways (syntax in Chapter 4). From *given coefficients*, `nwergm simulate` *nodes*`, edges ... theta()`, it needs no network in memory and covers the whole term library; covariates are read from the dataset in memory, which must have *nodes* observations, and the simulated networks are new networks (`generate()_1`, ...) without the covariates. From the *last fit*, `nwergm simulate, nsim()` ..., it starts from the last network of the fit (or `basis()`) and uses the fitted coefficients (or `coef()`), the proposal and the constraints of the fit; with `output(stats)` it returns only the statistics of the draws, in `r(stats)`. This is R's `simulate.ergm()`. A check against R: for the Lazega model of Section 8.3, 4000 draws at the fitted coefficients reproduce the means and standard deviations of the statistics of R's `simulate()` at the same coefficients (Chapter 9). Examples are in Sections 8.3 and 8.8.

The draws of the last fit form one chain: a burn-in once, then `interval()` proposals between draws, so that successive draws are autocorrelated; use `nosequential` for draws that restart from the starting network, or increase `interval()`. The simulated statistics of two draws from one chain are not independent replicates.

Chapter 7

Diagnostics, goodness of fit and postestimation

A fitted ERGM raises two questions. Did the estimation work, that is, did the Monte Carlo fit converge and is the Markov chain trustworthy? And does the model describe the network, that is, does it reproduce features of the data that were not built into it? This chapter describes the tools for both, and the other postestimation commands. All output was produced with the Lazega model of Section 8.3.

7.1 Diagnosing the Markov chain: `estat mcmcdiag`

```
estat mcmcdiag [, heidel report plot name(string) pvalue(#) eps(#)]
```

`estat mcmcdiag` reports diagnostics of the *final* Markov chain sample that the Monte Carlo fit simulated at its last coefficients. It is available after Monte Carlo maximum likelihood (and contrastive divergence and stochastic approximation), not after a pure pseudolikelihood fit, which involves no sampling; and it needs `e(mcmcsample)`, so it is not available after `nomcmcsample`. By default it prints a summary of the sample and one table with a row per statistic; `heidel` adds the Heidelberger–Welch test, and `report` adds the report of R's `mcmc.diagnostics()`. The results in `r()` are the same whichever options are given.

1. A summary per statistic. The header gives the size of the sample, the burn-in, the interval and the acceptance rate of the sampler; the table gives the mean, standard deviation, lag-1 autocorrelation, effective sample size (ESS) and Geweke z of each model statistic. The interval shown is that of the final sample, `e(mcmc_interval_final)`: in an adaptive fit the sampler may have lengthened it beyond the requested `e(mcmc_interval)`, which a note under the table then names; the lags of the autocorrelations in the report (option `report`) are multiples of the interval shown (Section 8.3). The ESS is computed as `coda`'s `effectiveSize()` does, $n\,\mathrm{var}(x)$ divided by the spectral density at frequency zero estimated from an autoregressive model; the formula $n(1-\rho_1)/(1+\rho_1)$ would overstate the ESS severalfold for the slowly decaying autocorrelation typical of ERGM chains. The fit samples until the effective sample size of the estimating equations reaches a target (64 by default, `mcmleeffectivesize()`), so that a final sample of a few hundred draws with an ESS of 64 to 100 is normal and is what `ergm` produces; an ESS far below that, or a small fraction of the number of draws, means that the chain mixes slowly and that the estimates and standard errors carry a large Monte Carlo error.

2. Two convergence tests. *Geweke's diagnostic* (the last column of the table) compares the mean of the first 10% of the retained sample with the mean of the last 50%, each corrected for autocorrelation by the same autoregressive spectral estimate that enters the standard errors;

$|z| > 1.96$ (flagged with an asterisk) rejects stationarity of that statistic at the 5% level. The *Heidelberger–Welch test* (option `heidel`) first checks stationarity (a Cramér–von Mises test on the cumulative-sum path of the chain, discarding an increasing initial part of up to 50%; *Start* is the first retained draw once the test is passed and *failed* means no part achieved it), and only then, for the retained part, whether the half-width of the 95% confidence interval of the mean is within `eps()` (default 10%) of the mean. `pvalue()` sets the level and must be 0.10, 0.05 (default), 0.025 or 0.01, because the test is evaluated against a table of critical values. Both tests are those of the `coda` package, which `ergm` uses.

3. The report of R's diagnostics. Option `report` adds the report of R's `mcmc.diagnostics()`: the mean, standard deviation, naive and time-series standard error of the sample statistics centered at the observed statistics; their quantiles; a test of whether the sample statistics differ from the observed ones, with Hotelling's omnibus test; the cross-correlations; the autocorrelations at lags that are multiples of the interval; and the Geweke statistic with individual and joint p-values. (The multivariate spectral estimate follows `ergm` 4.12.0; the development version of `ergm` changes the estimator, so the omnibus p-values can differ in the second or third digit from those of a later release.) The matrices are returned in `r(mc_summary)`, `r(mc_quantiles)`, `r(mc_obstest)`, `r(mc_crosscorr)`, `r(mc_autocorr)` and `r(mc_geweke)`, and the first block in `r(ess)`, `r(geweke)`, `r(heidel)` and `r(acceptrate)`.

What to conclude. `estat mcmcdiag` never claims that a fit converged merely because estimation stopped. The convergence test of the fit is a necessary check, not a proof: a low ESS, a low acceptance rate, a strongly autocorrelated chain, a large Geweke z or a failed stationarity test are all reasons to refit with a higher target effective sample size (`mcmleeffectivesize()`, the advice of the last note for an adaptive fit) or, for a fit with fixed-size samples (`mcmleeffectivesize(0)`), a larger `mcmcsamplesize()` or a longer `mcmcinterval()`, even when `e(converged)` is 1. Section 8.3 shows the output for the default settings and for a fit with a larger target. `plot` draws a trace plot and a kernel density for each statistic in one figure (Figure 7.1); a trace that drifts or shows long runs at one level, rather than a stationary "fuzzy caterpillar", indicates poor mixing even when the numbers look acceptable. The diagnostics are those of the last round of simulation, before the final estimates were computed, and may understate the performance of the final model; to assess the final model on its own statistics, use `estat gof`.

7.2 Goodness of fit: `estat gof`

```
estat gof [, gof(statistics) nsim(#) seed(#) gofburnin(#) gofinterval(#) plot maxdeg(#) maxdist
    (#) name(string)]
```

A model is not validated by a significant coefficient; it is validated by whether networks simulated from it resemble the observed network on features that the model does not contain (Hunter, Goodreau and Handcock [4]). `estat gof` follows R's `gof()`. It simulates the fitted model `nsim()` times (default 100) — continuing the Markov chain from the last network of the fit, with the burn-in and interval of the fit (the interval rescaled by sample size over `nsim()` as in R; 3000 and 50 after a pseudolikelihood fit) — and for each statistic reports the observed value, the minimum, mean and maximum over the simulations, and the Monte Carlo p-value $2\min\{P(\text{sim} \geq \text{obs}), P(\text{sim} \leq \text{obs})\}$. Rows that are zero in every simulation and in the data are left out. `gof(statistics)` chooses among

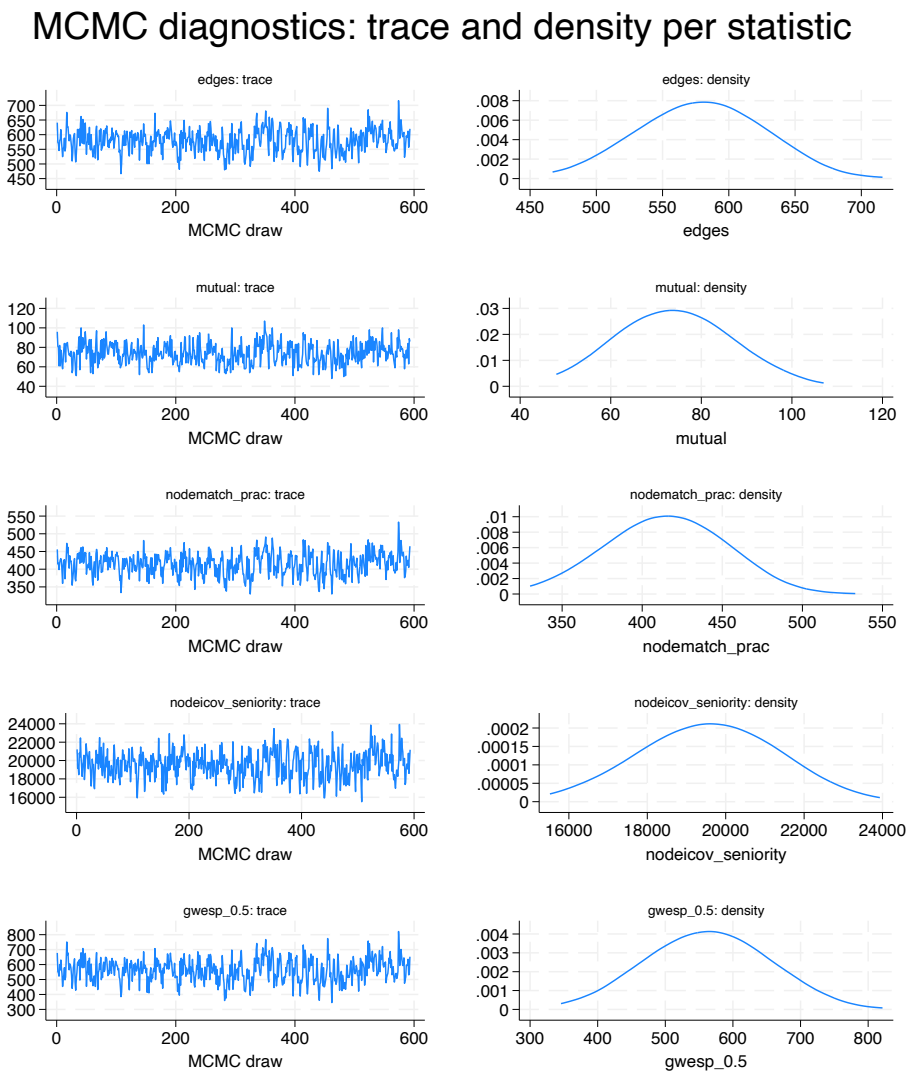


Figure 7.1: `estat mcmcdiag, plot` for the Lazega model with the default settings: trace (left) and density (right) of the five model statistics in the final Monte Carlo sample.

statistic	what is compared
`model`	the statistics of the model itself
`degree`	degree distribution (undirected networks)
`idegree`, `odegree`	in- and out-degree distributions (directed networks)
`b1degree`, `b2degree`	degree distributions of the two modes (two-mode networks)
`espartners`	edgewise shared partners
`dspartners`	dyadwise shared partners
`distance`	minimum geodesic distances (`Inf` for unreachable pairs)
`triadcensus`	triad census

A minus sign drops the model block (`gof(idegree -model)`). The default is R's default for the type of network: for an undirected network `degree espartners distance model`, for a directed network `idegree odegree espartners distance model`, and for a two-mode network `b1degree b2degree dspartners distance model`. The tables are returned as matrices `r(gof_`*statistic*`)`, with the columns observed, minimum, mean, maximum and p-value; the older summary values (`r(obs_meandeg)`, `r(sim_avgpath)`, `r(obs_triad_`*cat*`)`, ...) are still returned. The observed columns of these tables equal those of R's `gof()` for the Florentine (undirected) and Sampson (directed) networks.

How to read it. A large systematic gap between the observed value and the range of the simulations is evidence against the model on that feature; a small p-value on one row among many rows is expected by chance, and the Monte Carlo p-values of neighboring rows are not independent. A network in which the observed count is 0 and the simulations are rarely 0 (or the reverse) at the end of a distribution is a typical sign of a missing term. The comparison is as noisy as the Markov chain: each simulated network is `gofinterval()` proposals after the previous one, so that for a slowly mixing model even 100 draws can be too few, and refitting at different seeds can move the simulated mean degree from below the observed value to above it purely from Monte Carlo noise. If the final sample of the fit has a high autocorrelation, increase `nsim()` (and consider `gofinterval()`) before treating one run as a stable estimate; R's `gof()` has the same limitation. `plot` draws the degree and geodesic-distance distributions: a whisker (minimum–maximum), a box (interquartile range) and a median marker summarize the simulated draws at each value, and the observed proportion is overlaid as a red line with circle markers (Figure 7.2). Degrees above `maxdeg()` are pooled in one category and unreached pairs in a category *NR*, as in statnet.

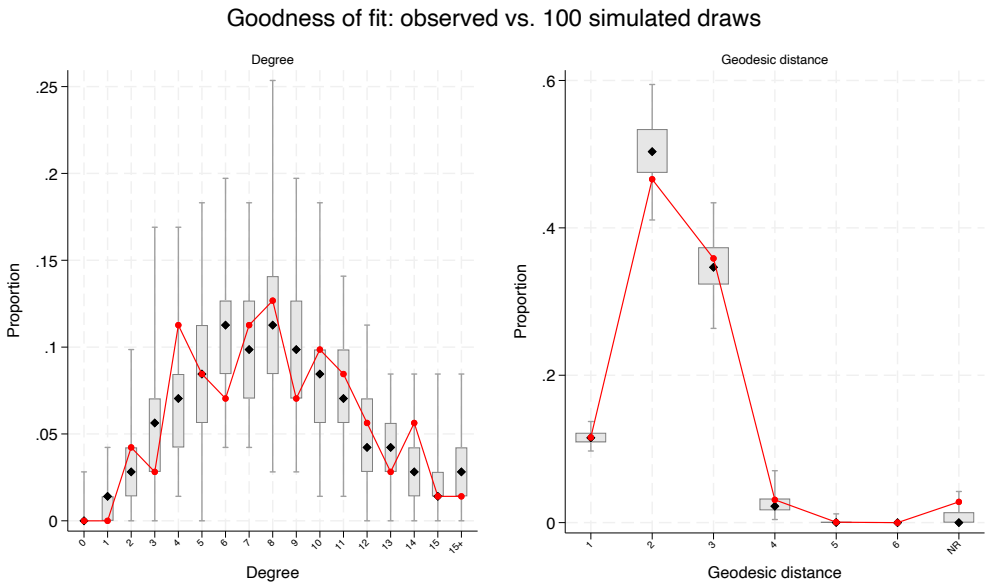


Figure 7.2: `estat gof, plot` for the Lazega model: degree and geodesic-distance distributions of 100 simulated networks (boxes) and of the observed network (line).

7.3 Information criteria and likelihood-ratio tests

After every fit `nwergm` stores the log-likelihood as R's `logLik()` (Section 2.2.3). `estat ic` displays the number of informative dyads, the log-likelihood of the null model and of the fitted model, the degrees of freedom, AIC and BIC; it refuses when `e(ll)` is missing (a curved Monte Carlo fit, or `nologlik`). `estimates stats` lists the criteria of several stored models; note that Stata's `estimates stats` uses the number of observations N for the BIC, which for an ERGM is the number of informative dyads, as `estat ic` does. `lrtest` works between nested fits of the same network with the same number of informative dyads, for example

```
. estimates store m1
. nwergm lazega_fr, edges mutual nodematch(prac) gwesp(0.5) ...
. estimates store m2
. lrtest m2 m1
```

The test is exact only to the extent that the log-likelihoods are: for Monte Carlo fits they carry

the Monte Carlo standard error `e(ll_se)`, which should be small compared with the differences that matter; for a pseudolikelihood fit of a dyad-dependent model the "log-likelihood" is the pseudolikelihood and a likelihood-ratio test is not valid. Wald tests of coefficients (`test`, `lincom`) work on `e(b)` and `e(V)`. Section 8.6 has an example.

7.4 Prediction: `predict`

`predict, matrix(`*name*`)` [`pr`|`xb`] and `predict, frame(`*name*`)` [`pr`|`xb`] give the conditional probability (`pr`, the default) or the conditional log odds (`xb`) of each informative dyad given the rest of the observed network, as R's `predict.ergm()` (equation (2.2)). The frame has one row per dyad (`tail`, `head`, `p`); the matrix is $n \times n$ with missing dyads set to missing. The prediction is available for the most recent `nwergm` fit only. R uses the last network of the Monte Carlo sample as the basis for a dyad-dependent fit; `nwergm` always uses the observed network. For a dyad-independent fit the two coincide, and the result equals R's to 10^{-7}.

7.5 Stored results

After a fit `nwergm` stores the following in `e()`. The listings in Section 8.1.3 show them for a pseudolikelihood fit and for a Monte Carlo fit.

Scalars.

`e(N)`	Number of informative dyads (observed, not missing, not constrained).
`e(nodes)`, `e(ties)`	Number of nodes and of observed ties.
`e(nmissing)`	Number of missing dyads.
`e(ll)`, `e(ll_0)`	Log-likelihood and null log-likelihood (Section 2.2.3). Missing after a curved Monte Carlo fit and with `nologlik`.
`e(ll_se)`	Monte Carlo standard error of `e(ll)` (0 when exact).
`e(df_m)`, `e(rank)`, `e(df_resid)`	Estimated coefficients (offsets excluded), rank, and $N-$`df_m`.
`e(deviance)`, `e(deviance_0)`	Residual and null deviance, -2 times the log-likelihood.
`e(aic)`, `e(bic)`	AIC and BIC (BIC with N the number of informative dyads).
`e(curved)`	1 if the model has a curved term.
`e(native)`	1 if the native plugin was used (the sampler for Monte Carlo fits, the design-matrix build for pseudolikelihood fits), 0 for Mata. Informational only.
`e(spcache)`	1 if `spcache` was used.
`e(converged)`	1 if the convergence test of the Monte Carlo iteration was satisfied (Monte Carlo fits only).
`e(mcmle_iterations)`	Number of outer iterations (for stochastic approximation: of phase 2).
`e(mcmc_acceptrate)`	Acceptance rate of the final simulation.
`e(mcmc_burnin)`, `e(mcmc_interval)`, `e(mcmc_samplesize)`	Burn-in and interval as requested, and the number of draws kept by the last iteration (in the adaptive sample, more than the base size `mcmcsamplesize()` if the effective sample size target needed it).
`e(mcmc_interval_final)`	The interval of the sample of the last iteration. In the adaptive sample (the default) it follows from the halving of the interval between iterations and the thinning of the sample (Section 2.2.2) and can be shorter or longer than `e(mcmc_interval)`; in a fixed-size fit it is larger than `e(mcmc_interval)` if the interval was grown.

Macros.

`e(cmd)`	`nwergm`
`e(title)`	Title of the estimation.
`e(depvar)`	Name of the fitted network.
`e(method)`	`mple` or `mcmle` (the simulation-based estimators share `mcmle`).
`e(estimate)`	`MLE`, `CD` or `SA` for a simulation-based fit (empty after a pseudolikelihood fit).
`e(directed)`	`true` or `false`.
`e(proposal)`	Proposal weights used (simulation-based fits).

`e(trifocus)`	Share of triadic steps of the SPDyad proposal (0.25 by default; missing when SPDyad was not used, for example with `mcmcprop(sparse)` or on a two-mode network).
`e(ll_method)`	How `e(ll)` was obtained: `exact`, `pseudolikelihood` or `bridge sampling`.
`e(version)`	The release of `nwergm` that produced the results.
`e(predict)`, `e(estat_cmd)`, `e(properties)`	Postestimation dispatch.

Matrices. `e(b)` (coefficients) and `e(V)` (covariance matrix); `e(mcmcsample)`, the final simulation's sample of the model statistics, size × number of coefficients (simulation-based fits, unless `nomcmcsample`); `e(obs_stats)`, the observed statistics (pseudolikelihood fits).

`r()`. `estat mcmcdiag`, `estat gof`, `nwergm simulate` and `nwergm version` return the results named in the sections above: `r(ess)`, `r(geweke)`, `r(heidel)`, `r(acceptrate)` and `r(mc_...)`; `r(gof_`*statistic*`)`, `r(nsim)`, `r(burnin)`, `r(interval)`; `r(stats)` (`nwergm simulate`, also `r(nsim)`, `r(burnin)`, `r(interval)`); `r(version)`, `r(date)`, `r(core)`, `r(plugin)`, `r(reference)`, `r(license)`.

Chapter 8

Worked examples

All output in this chapter was produced by the do-files in the directory `manual/ex` of the repository, run with `nwergm` 1.5.9 and the `nwcommands` core 3.17.2 in Stata/BE 18 on macOS, with the native plugin. Every Monte Carlo fit has a `seed()`, so that the output can be reproduced on the same platform. The data are example networks of `nwcommands`, which are loaded from the web with `nwwebuse`; the Florentine marriage network, the Mesa High School network and the second Kapferer network are the networks `flomarriage`, `faux.mesa.high` and `kapferer2` of `ergm`. Nothing in this chapter is a substantive analysis; the models are chosen to show the commands.

8.1 The Florentine families: estimation methods and stored results

Chapter 3 fitted `edges nodecov(wealth)` by pseudolikelihood (exact) and added `gwesp(0.5)` by Monte Carlo maximum likelihood. This section uses the second model to show the estimation options. The network is small (16 nodes, 20 ties), so that every fit takes a few seconds.

8.1.1 The same model by different estimators

The pseudolikelihood fit of the dyad-dependent model, with `estimate(mple)`, is a starting value and not a maximum likelihood estimate; `nwergm` says so and labels the deviances "pseudo-deviance":

```
. nwergm flomarriage, edges nodecov(wealth) gwesp(0.5) estimate(mple)
Note: method(mple) requested for a dyad-dependent model - reporting pseudolikelihood, NOT full
    ERGM maximum likelihood.

Exponential-family random graph model

Network:               =  flomarriage
Nodes:                 =  16
Ties:                  =  20
Directed:              =  No
Estimation:            =  MPLE

------------------------------------------------------------------------------
   flomarriage | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
---------------+--------------------------------------------------------------
         edges |  -2.614769   .5469954    -4.78   0.000     -3.68686   -1.542677
nodecov_wealth |   .0103724   .0047631     2.18   0.029     .0010369    .0197079
     gwesp_0.5 |   .0333115   .1724709     0.19   0.847    -.3047253    .3713482
------------------------------------------------------------------------------

Maximum pseudolikelihood estimate; this is the maximum likelihood estimate only if the model is
    dyad-independent.
```

```
      Null Pseudo-deviance:     166.4  on 120 degrees of freedom
  Residual Pseudo-deviance:     103.1  on 117 degrees of freedom

Log-likelihood:   -51.5358 (df = 3)   AIC:     109.1   BIC:     117.4
(For a dyad-dependent model fitted by MPLE, this is the pseudolikelihood.)
```

Contrastive divergence (`estimate(cd)`) is the cheap estimator for starting values; it ran six iterations here and reports the covariance of the short-run sample:

```
. nwergm flomarriage, edges nodecov(wealth) gwesp(0.5) estimate(cd) seed(1)

Exponential-family random graph model

Network:               =  flomarriage
Nodes:                 =  16
Ties:                  =  20
Directed:              =  No
Estimation:            =  Contrastive divergence

------------------------------------------------------------------------------
   flomarriage | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
---------------+--------------------------------------------------------------
         edges |  -2.672972   1.037071    -2.58   0.010    -4.705594   -.6403512
nodecov_wealth |   .0122082   .0101107     1.21   0.227    -.0076085    .0320249
     gwesp_0.5 |  -.0278411   .4066446    -0.07   0.945    -.8248499    .7691676
------------------------------------------------------------------------------

Contrastive divergence estimate (6 iteration(s)); standard errors come from the covariance of
    the short-run CD sample.
Evaluating log-likelihood at the estimate (bridge sampling with 16 bridges) ...

             Null Deviance:     166.4  on 120 degrees of freedom
         Residual Deviance:     103.2  on 117 degrees of freedom

Log-likelihood:   -51.6057 (df = 3)   AIC:     109.2   BIC:     117.6
Monte Carlo standard error of the log-likelihood:  .01698
```

Stochastic approximation (`mainmethod(sa)`) is the alternative to Monte Carlo maximum likelihood (462 phase-2 iterations here):

```
. nwergm flomarriage, edges nodecov(wealth) gwesp(0.5) mainmethod(sa) seed(1)

Exponential-family random graph model

Network:               =  flomarriage
Nodes:                 =  16
Ties:                  =  20
Directed:              =  No
Estimation:            =  Stochastic approximation

------------------------------------------------------------------------------
   flomarriage | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
---------------+--------------------------------------------------------------
         edges |  -2.605002   .5547047    -4.70   0.000    -3.692203   -1.517801
nodecov_wealth |   .0109453   .0050857     2.15   0.031     .0009775    .0209131
     gwesp_0.5 |  -.0268843   .2714457    -0.10   0.921    -.5589081    .5051395
------------------------------------------------------------------------------

Stochastic approximation estimate (462 phase-2 iteration(s)).
Evaluating log-likelihood at the estimate (bridge sampling with 16 bridges) ...

             Null Deviance:     166.4  on 120 degrees of freedom
         Residual Deviance:     103.1  on 117 degrees of freedom
```

```
Log-likelihood:   -51.5565 (df = 3)   AIC:     109.1   BIC:     117.5
Monte Carlo standard error of the log-likelihood:  .00059
```

All three agree with the Monte Carlo maximum likelihood fit of Chapter 3 within what the standard errors allow: the edges coefficient is -2.61 (pseudolikelihood), -2.67 (contrastive divergence) and -2.61 (stochastic approximation) against -2.58, and the wealth coefficient is 0.0104, 0.0122 and 0.0109 against 0.0104. The Firth-penalized pseudolikelihood (`mpletype(penalized)`) shrinks the coefficients slightly; on this network, where the ordinary estimate exists, the difference is small:

```
. nwergm flomarriage, edges nodecov(wealth) estimate(mple) mpletype(penalized)

Exponential-family random graph model

Network:               =  flomarriage
Nodes:                 =  16
Ties:                  =  20
Directed:              =  No
Estimation:            =  MPLE

------------------------------------------------------------------------------
   flomarriage | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
---------------+--------------------------------------------------------------
         edges |  -2.543743    .528982    -4.81   0.000    -3.580529   -1.506957
nodecov_wealth |   .0103689   .0046355     2.24   0.025     .0012835    .0194543
------------------------------------------------------------------------------

Maximum pseudolikelihood estimate; this is the maximum likelihood estimate only if the model is
    dyad-independent.

             Null Deviance:     166.4  on 120 degrees of freedom
         Residual Deviance:     103.1  on 118 degrees of freedom

Log-likelihood:   -51.5639 (df = 2)   AIC:     107.1   BIC:     112.7
```

A coefficient can be fixed with `offset()`; the fixed coefficient is reported at its value with no standard error, and the rest of the model adapts:

```
. nwergm flomarriage, edges nodecov(wealth) gwesp(0.5) offset(nodecov_wealth 0.01) seed(1)

Exponential-family random graph model

Network:               =  flomarriage
Nodes:                 =  16
Ties:                  =  20
Directed:              =  No
Estimation:            =  MCMLE
MCMC sample size:      =  243

------------------------------------------------------------------------------
   flomarriage | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
---------------+--------------------------------------------------------------
         edges |  -2.520162   .3759037    -6.70   0.000    -3.256919   -1.783404
nodecov_wealth |        .01          .        .       .           .           .
     gwesp_0.5 |  -.0098068   .2417939    -0.04   0.968    -.4837142    .4641005
------------------------------------------------------------------------------

MCMLE converged after 1 iteration(s).
Evaluating log-likelihood at the estimate (bridge sampling with 16 bridges) ...

             Null Deviance:     166.4  on 120 degrees of freedom
         Residual Deviance:     103.1  on 118 degrees of freedom
```

```
Log-likelihood:   -51.5612 (df = 2)   AIC:     107.1   BIC:     112.7
Monte Carlo standard error of the log-likelihood:  .00134
```

`init()` gives starting values; here the fit starts from the values below instead of from the pseudolikelihood estimate and ends in the same place up to Monte Carlo noise:

```
. nwergm flomarriage, edges nodecov(wealth) gwesp(0.5) init(-2.6 0.01 0) seed(1)

Exponential-family random graph model

Network:               =  flomarriage
Nodes:                 =  16
Ties:                  =  20
Directed:              =  No
Estimation:            =  MCMLE
MCMC sample size:      =  486

------------------------------------------------------------------------------
   flomarriage | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
---------------+--------------------------------------------------------------
         edges |  -2.604571   .5328454    -4.89   0.000    -3.648929   -1.560214
nodecov_wealth |   .0109055   .0049508     2.20   0.028     .0012021    .0206089
     gwesp_0.5 |  -.0149075   .2823989    -0.05   0.958    -.5683991    .5385842
------------------------------------------------------------------------------

MCMLE converged after 1 iteration(s).
Evaluating log-likelihood at the estimate (bridge sampling with 16 bridges) ...

             Null Deviance:     166.4  on 120 degrees of freedom
         Residual Deviance:     103.1  on 117 degrees of freedom

Log-likelihood:   -51.5531 (df = 3)   AIC:     109.1   BIC:     117.5
Monte Carlo standard error of the log-likelihood:  .00238
```

Finally the curved form of the shared-partner term, `gwespfree(0.5)`, estimates a weight and a decay. On this small network the decay is not identified: the fit ends at a boundary solution, with a decay of about 10^{-9} and a standard error of 9, which is what a "large standard error on the decay" (Section 2.6) looks like:

```
. nwergm flomarriage, edges nodecov(wealth) gwespfree(0.5) estimate(mple)
Note: method(mple) requested for a dyad-dependent model - reporting pseudolikelihood, NOT full
    ERGM maximum likelihood.

Exponential-family random graph model

Network:               =  flomarriage
Nodes:                 =  16
Ties:                  =  20
Directed:              =  No
Estimation:            =  MPLE

------------------------------------------------------------------------------
   flomarriage | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
---------------+--------------------------------------------------------------
         edges |  -2.621176    .547741    -4.79   0.000    -3.694729   -1.547624
nodecov_wealth |   .0103079   .0047691     2.16   0.031     .0009605    .0196552
  gwesp_weight |   .0496566   .2045734     0.24   0.808    -.3512999    .4506131
   gwesp_decay |   6.46e-10   9.038386     0.00   1.000    -17.71491    17.71491
------------------------------------------------------------------------------

Maximum pseudolikelihood estimate; this is the maximum likelihood estimate only if the model is
    dyad-independent.
```

```
      Null Pseudo-deviance:     166.4  on 120 degrees of freedom
  Residual Pseudo-deviance:     103.0  on 116 degrees of freedom

Log-likelihood:   -51.5184 (df = 4)   AIC:     111.0   BIC:     122.2
(For a dyad-dependent model fitted by MPLE, this is the pseudolikelihood.)
(decay is an estimated curved parameter here - see help nwergm, gw_effects for how this differs
    from R ergm's own BFGS-ba
> sed fit)
```

8.1.2 Reproducibility with `seed()`

The same call with the same seed gives the same coefficients; a different seed gives a different, statistically equivalent fit. Below the model is fitted twice with `seed(7)` and once with `seed(8)`:

```
. quietly nwergm flomarriage, edges nodecov(wealth) gwesp(0.5) seed(7)

. matrix b1 = e(b)

. quietly nwergm flomarriage, edges nodecov(wealth) gwesp(0.5) seed(7)

. matrix b2 = e(b)

. quietly nwergm flomarriage, edges nodecov(wealth) gwesp(0.5) seed(8)

. matrix b3 = e(b)

. matrix list b1, format(%10.6f)

b1[1,3]
           edges  nodecov_we~h     gwesp_0.5
y1     -2.590426      0.011175     -0.035132

. matrix list b2, format(%10.6f)

b2[1,3]
           edges  nodecov_we~h     gwesp_0.5
y1     -2.590426      0.011175     -0.035132

. matrix list b3, format(%10.6f)

b3[1,3]
           edges  nodecov_we~h     gwesp_0.5
y1     -2.593350      0.010514      0.011883
```

The first two coefficient vectors are identical to the digit; the third differs by 0.003 in the edges coefficient and by 0.047 in the shared-partner coefficient, which is the Monte Carlo error of this fit (on a network of 16 nodes and 20 ties, where the sample of an iteration has a few hundred draws). Reporting two or three seeds is a cheap way to judge the size of that error; Chapter 9 does this for a larger model.

8.1.3 Stored results

The stored results of the pseudolikelihood fit and of the Monte Carlo fit (the output is that of `ereturn list`; Section 7.5 describes each item):

```
. quietly nwergm flomarriage, edges nodecov(wealth)

. ereturn list

scalars:
           e(trifocus) =  .25
```

```
                e(N) =  120
            e(nodes) =  16
             e(ties) =  20
         e(nmissing) =  0
           e(curved) =  0
           e(native) =  1
               e(ll) =  -51.55428619540994
             e(ll_0) =  -83.17766166719343
            e(ll_se) =  0
             e(df_m) =  2
             e(rank) =  2
         e(df_resid) =  118
         e(deviance) =  103.1085723908199
       e(deviance_0) =  166.3553233343869
              e(aic) =  107.1085723908199
              e(bic) =  112.683555876384

macros:
          e(predict) : "nwergm_p"
        e(ll_method) : "exact"
        e(estat_cmd) : "nwergm_estat"
         e(proposal) : "tnt"
         e(directed) : "false"
           e(method) : "mple"
           e(depvar) : "flomarriage"
            e(title) : "Exponential-family random graph model (MPLE)"
          e(version) : "1.5.7"
              e(cmd) : "nwergm"
       e(properties) : "b V"

matrices:
                e(b) :  1 x 2
                e(V) :  2 x 2
         e(obs_stats) :  1 x 2
```

```
. ereturn list

scalars:
          e(trifocus) =  .25
                e(N) =  120
            e(nodes) =  16
          e(converged) =  1
   e(mcmle_iterations) =  1
    e(mcmc_acceptrate) =  .659462
        e(mcmc_burnin) =  16384
      e(mcmc_interval) =  1024
  e(mcmc_interval_fin
    al)               =  1024
           e(native) =  1
          e(spcache) =  0
           e(curved) =  0
    e(mcmc_samplesize) =  243
             e(ties) =  20
         e(nmissing) =  0
               e(ll) =  -51.5534343475783
             e(ll_0) =  -83.17766166719343
            e(ll_se) =  .000861690410224
             e(df_m) =  3
             e(rank) =  3
              e(bic) =  117.4693439235027
              e(aic) =  109.1068686951566
       e(deviance_0) =  166.3553233343869
         e(deviance) =  103.1068686951566
```

```
           e(df_resid) =  117

macros:
            e(predict) : "nwergm_p"
          e(ll_method) : "bridge sampling"
          e(estat_cmd) : "nwergm_estat"
           e(proposal) : "tnt"
           e(directed) : "false"
           e(estimate) : "MLE"
             e(method) : "mcmle"
             e(depvar) : "flomarriage"
              e(title) : "Exponential-family random graph model (MCMLE)"
            e(version) : "1.5.7"
                e(cmd) : "nwergm"
         e(properties) : "b V"

matrices:
                  e(b) :  1 x 3
                  e(V) :  3 x 3
         e(mcmcsample) :  243 x 3
```

```
. display "converged: " e(converged) "   iterations: " e(mcmle_iterations) "   native: " e(
    native)
converged: 1   iterations: 1   native: 1

. display "sample size: " e(mcmc_samplesize) "   burn-in: " e(mcmc_burnin) "   interval: " e(
    mcmc_interval_final)
sample size: 243   burn-in: 16384   interval: 1024

. display "version: " e(version)
version: 1.5.7

. matrix list e(b)

e(b)[1,3]
           edges  nodecov_we~h     gwesp_0.5
y1    -2.5769751     .01044623    -.00450087
```

8.2 Mesa High School: dyad-independent models

The Mesa High School network is the friendship network of 205 students of a US school (`ergm`'s `faux.mesa.high`, a simulated network that mimics survey data on adolescent friendships) with the grade, race and sex of each student. The ties are undirected. All the terms in this section depend only on the attributes of the two students, so that the models are dyad-independent: they are fitted by pseudolikelihood, which is exact, and no random numbers are used. The attributes are strings and must be encoded:

```
. nwwebuse mesa, nwclear

. nwset
(1 networks)
--------------------
      mesa

. describe, short

Contains data from https://raw.githubusercontent.com/thomas-u-grund/nwcommands/main/data/mesa.
    nwdta
 Observations:           205
    Variables:             5                  1 Sep 2026 21:41
```

```
Sorted by:
     Note: Dataset has changed since last saved.

. tabulate grade

      grade |      Freq.     Percent        Cum.
------------+-----------------------------------
          7 |         62       30.24       30.24
          8 |         40       19.51       49.76
          9 |         42       20.49       70.24
         10 |         25       12.20       82.44
         11 |         24       11.71       94.15
         12 |         12        5.85      100.00
------------+-----------------------------------
      Total |        205      100.00

. tabulate race

       race |      Freq.     Percent        Cum.
------------+-----------------------------------
      Black |          6        2.93        2.93
       Hisp |        109       53.17       56.10
      NatAm |         68       33.17       89.27
      Other |          4        1.95       91.22
      White |         18        8.78      100.00
------------+-----------------------------------
      Total |        205      100.00

. tabulate sex

        sex |      Freq.     Percent        Cum.
------------+-----------------------------------
          F |         99       48.29       48.29
          M |        106       51.71      100.00
------------+-----------------------------------
      Total |        205      100.00
```

```
. encode race, generate(race_n)

. encode sex, generate(sex_n)

. label list race_n
race_n:
           1 Black
           2 Hisp
           3 NatAm
           4 Other
           5 White

. label list sex_n
sex_n:
           1 F
           2 M
```

The intercept-only model estimates the density; the edges coefficient is the log odds of a tie between two randomly chosen students ($\exp(-4.63) \approx 0.01$):

```
. nwergm mesa, edges

Exponential-family random graph model

Network:               =  mesa
Nodes:                 =  205
```

```
Ties:                  =  203
Directed:              =  No
Estimation:            =  MPLE

------------------------------------------------------------------------------
        mesa | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
-------------+----------------------------------------------------------------
       edges |  -4.625021   .0705294   -65.58   0.000    -4.763256   -4.486786
------------------------------------------------------------------------------

Maximum pseudolikelihood estimate; this is the maximum likelihood estimate only if the model is
    dyad-independent.

             Null Deviance:   28987.4  on 20910 degrees of freedom
         Residual Deviance:    2285.7  on 20909 degrees of freedom

Log-likelihood: -1142.8711 (df = 1)   AIC:    2287.7   BIC:    2295.7
```

Adding homophily on the grade, `nodematch(grade)`, shows that students in the same grade are $\exp(2.83) \approx 17$ times as likely, in odds, to be tied, and the log-likelihood rises by 173:

```
. nwergm mesa, edges nodematch(grade)

Exponential-family random graph model

Network:               =  mesa
Nodes:                 =  205
Ties:                  =  203
Directed:              =  No
Estimation:            =  MPLE

------------------------------------------------------------------------------
           mesa | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
----------------+-------------------------------------------------------------
          edges |  -6.034045   .1583032   -38.12   0.000    -6.344314   -5.723776
nodematch_grade |   2.830999   .1773251    15.97   0.000     2.483449     3.17855
------------------------------------------------------------------------------

Maximum pseudolikelihood estimate; this is the maximum likelihood estimate only if the model is
    dyad-independent.

             Null Deviance:   28987.4  on 20910 degrees of freedom
         Residual Deviance:    1939.5  on 20908 degrees of freedom

Log-likelihood:  -969.7742 (df = 2)   AIC:    1943.5   BIC:    1959.4
```

`nodematch()` takes several variables, each with its own coefficient:

```
. nwergm mesa, edges nodematch(grade race_n sex_n)

Exponential-family random graph model

Network:               =  mesa
Nodes:                 =  205
Ties:                  =  203
Directed:              =  No
Estimation:            =  MPLE

------------------------------------------------------------------------------
            mesa | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
-----------------+------------------------------------------------------------
           edges |  -6.600575   .1998753   -33.02   0.000    -6.992324   -6.208827
 nodematch_grade |   2.830693    .177456    15.95   0.000     2.482886    3.178501
```

```
nodematch_race_n |   .4331562   .1429951     3.03   0.002     .1528911    .7134214
 nodematch_sex_n |   .6447389    .149475     4.31   0.000     .3517733    .9377045
------------------------------------------------------------------------------

Maximum pseudolikelihood estimate; this is the maximum likelihood estimate only if the model is
    dyad-independent.

             Null Deviance:   28987.4  on 20910 degrees of freedom
         Residual Deviance:    1911.2  on 20906 degrees of freedom

Log-likelihood:  -955.5870 (df = 4)   AIC:    1919.2   BIC:    1951.0
```

Differential homophily, `nodematchdiff()`, gives one coefficient per level of the attribute: homophily is stronger among the students of sex level 1 (female) than among those of level 2 (male); the levels are the values of the encoded variable, whose labels `label list` shows above:

```
. nwergm mesa, edges nodematch(grade) nodematchdiff(sex_n)

Exponential-family random graph model

Network:               =  mesa
Nodes:                 =  205
Ties:                  =  203
Directed:              =  No
Estimation:            =  MPLE

------------------------------------------------------------------------------
             mesa | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
------------------+-----------------------------------------------------------
            edges |  -6.392904   .1856467   -34.44   0.000    -6.756765   -6.029043
  nodematch_grade |   2.820284   .1774895    15.89   0.000     2.472411    3.168157
nodematch_sex_n_1 |   .8880277   .1651607     5.38   0.000     .5643186    1.211737
nodematch_sex_n_2 |    .333064   .1871779     1.78   0.075     -.033798     .699926
------------------------------------------------------------------------------

Maximum pseudolikelihood estimate; this is the maximum likelihood estimate only if the model is
    dyad-independent.

             Null Deviance:   28987.4  on 20910 degrees of freedom
         Residual Deviance:    1910.8  on 20906 degrees of freedom

Log-likelihood:  -955.4147 (df = 4)   AIC:    1918.8   BIC:    1950.6
```

Finally `nodefactor()` (one coefficient per level of race but the first, each counting the ties of the students at that level) and `nodemix()` (a mixing matrix of the sex of the two students, the first cell dropped) in the same model:

```
. nwergm mesa, edges nodematch(grade) nodefactor(race_n) nodemix(sex_n)

Exponential-family random graph model

Network:               =  mesa
Nodes:                 =  205
Ties:                  =  203
Directed:              =  No
Estimation:            =  MPLE

------------------------------------------------------------------------------
              mesa | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
-------------------+----------------------------------------------------------
             edges |  -3.303633   .4342755    -7.61   0.000    -4.154797   -2.452468
   nodematch_grade |   2.874633   .1785757    16.10   0.000     2.524631    3.224635
nodefactor_race_n_2 |  -1.335929   .2233276    -5.98   0.000    -1.773643   -.8982147
```

```
nodefactor_race_n_3 |   -.924473   .2260387    -4.09   0.000    -1.367501   -.4814453
nodefactor_race_n_4 |  -3.214314   1.025584    -3.13   0.002    -5.224422   -1.204206
nodefactor_race_n_5 |  -.8606332   .2582152    -3.33   0.001    -1.366726   -.3545407
  nodemix_sex_n_1_2 |  -.9352662   .1663521    -5.62   0.000     -1.26131   -.6092221
  nodemix_sex_n_2_2 |  -.6284435   .1849762    -3.40   0.001    -.9909901   -.2658968
------------------------------------------------------------------------------

Maximum pseudolikelihood estimate; this is the maximum likelihood estimate only if the model is
    dyad-independent.

            Null Deviance:   28987.4  on 20910 degrees of freedom
        Residual Deviance:    1863.4  on 20902 degrees of freedom

Log-likelihood:  -931.6931 (df = 8)   AIC:    1879.4   BIC:    1943.0
```

8.3 The Lazega lawyers: a directed network, diagnostics and goodness of fit

The data are the friendship ties among the 71 attorneys of a New England corporate law firm (Lazega [16]): a directed network with 575 ties and, for each lawyer, the seniority, the age, the practice (corporate or litigation) and the office, among others. The seniority is a number from 1 to 71.

```
. nwwebuse lazega, nwclear

. nwset
(3 networks)
--------------------
      lazega_adv
      lazega_cow
      lazega_fr

. describe, short

Contains data from https://raw.githubusercontent.com/thomas-u-grund/nwcommands/main/data/lazega.
> nwdta
 Observations:            71
    Variables:            10                  1 Sep 2026 21:38
Sorted by:
     Note: Dataset has changed since last saved.

. summarize seniority age

    Variable |        Obs        Mean    Std. dev.       Min        Max
-------------+---------------------------------------------------------
   seniority |         71          36    20.63977          1         71
         age |         71    41.84507    10.25343         26         67

. tabulate practice

   practice |      Freq.     Percent        Cum.
------------+-----------------------------------
  corporate |         30       42.25       42.25
 litigation |         41       57.75      100.00
------------+-----------------------------------
      Total |         71      100.00
```

The practice is encoded before it is used in `nodematch()`:

```
. encode practice, generate(prac)
```

The model has the density (`edges`), reciprocity (`mutual`), homophily on the practice (`nodematch(prac)`), a main effect of the seniority of the receiver (`nodeicov(seniority)`) and a geometrically weighted shared-partner term (`gwesp(0.5)`), which captures transitive closure. The model is dyad-dependent, so that the Monte Carlo maximum likelihood estimator is used:

```
. nwergm lazega_fr, edges mutual nodematch(prac) nodeicov(seniority) gwesp(0.5) seed(1)

Exponential-family random graph model

Network:               =  lazega_fr
Nodes:                 =  71
Ties:                  =  575
Directed:              =  Yes
Estimation:            =  MCMLE
MCMC sample size:      =  593

------------------------------------------------------------------------------
         lazega_fr | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
-------------------+----------------------------------------------------------------
             edges |  -3.318445   .1335225   -24.85   0.000    -3.580144   -3.056746
            mutual |   .8058073   .1547052     5.21   0.000     .5025907    1.109024
    nodematch_prac |   .7113241   .0808705     8.80   0.000     .5528208    .8698274
nodeicov_seniority |  -.0044112   .0017983    -2.45   0.014    -.0079358   -.0008866
         gwesp_0.5 |   .4896686   .0639934     7.65   0.000     .3642438    .6150933
------------------------------------------------------------------------------

MCMLE converged after 6 iteration(s).
Evaluating log-likelihood at the estimate (bridge sampling with 16 bridges) ...

             Null Deviance:    6889.9  on 4970 degrees of freedom
         Residual Deviance:    3335.1  on 4965 degrees of freedom

Log-likelihood: -1667.5431 (df = 5)   AIC:    3345.1   BIC:    3377.6
Monte Carlo standard error of the log-likelihood:  .16479
```

Friendship ties are reciprocated (`mutual` is 0.81, so that a tie that reciprocates an existing one has about 2.2 times the odds), among lawyers of the same practice (the odds are $\exp(0.71) \approx 2$ times higher), and cluster (the shared-partner coefficient is 0.49, with a standard error of 0.06). The coefficient of the seniority of the receiver is small and negative (-0.004 per unit of a variable that runs from 1 to 71).

8.3.1 Is the chain trustworthy?

The fit says it converged after six iterations. The default sample is adaptive (Section 2.2.2): the last iteration kept 593 draws, and the interval of that sample is not the requested interval of 1024 but the one that the sampler arrived at (`e(mcmc_interval_final)`); `estat mcmcdiag` shows the interval of the final sample and, in a note under the table, the requested one when the two differ:

```
. display "requested interval " e(mcmc_interval) ", interval of the last sample " e(mcmc_interva
> l_final) ", draws kept " e(mcmc_samplesize)
requested interval 1024, interval of the last sample 4096, draws kept 593
```

`estat mcmcdiag` looks at the final sample; option `heidel` adds the Heidelberger–Welch test. The last note advises a higher target effective sample size (`mcmleeffectivesize()`) for an adaptive fit and a larger `mcmcsamplesize()` or `mcmcinterval()` for a fit with fixed-size samples:

```
. estat mcmcdiag, heidel

MCMC diagnostics, final sample            Sample size      =        593
                                          Burn-in          =     16,384
                                          Interval         =      4,096
```

```
                                     Acceptance rate  =     0.5765

------------------------------------------------------------------------
         Statistic |      Mean        SD  Autocorr       ESS   Geweke z
-------------------+----------------------------------------------------
             edges |    579.02     39.63     0.496     201.1      0.17
           mutual |     73.96     10.62     0.367     185.4      0.23
    nodematch_prac |    414.53     30.84     0.491     203.8      0.13
nodeicov_seniority |  19563.64   1467.93     0.439     232.1      0.14
         gwesp_0.5 |    562.29     75.43     0.484     207.4      0.33
------------------------------------------------------------------------
The adaptive sampler raised the interval from 1,024 to 4,096.

Heidelberger-Welch test           pvalue           =      0.050
                                  eps              =       0.10
-------------------------------------------------------------
         Statistic | Stationary    Start  Halfwidth  Halfwidth
                   |                                     test
-------------------+-----------------------------------------
             edges |     passed        1       5.48     passed
           mutual |     passed        1       1.53     passed
    nodematch_prac |     passed        1       4.23     passed
nodeicov_seniority |     passed        1     188.84     passed
         gwesp_0.5 |     passed        1      10.27     passed
-------------------------------------------------------------
Stationary: a Cramer-von-Mises test on the cumulative sums passes after discarding up to half
of the sample (Heidelberger and Welch 1983); Start is the first draw kept. Halfwidth test: the
half-width of the 95% interval of the mean is within eps() of the mean.

The convergence test of MCMLE was satisfied after 6 iterations. This is necessary but not
sufficient for convergence; option plot draws the trace plots.
If the ESS is low, raise mcmleeffectivesize(); after mcmleeffectivesize(0), raise
mcmcsamplesize() or mcmcinterval().
```

With the default settings, which are those of `ergm`, the chain mixes acceptably: the lag-1 autocorrelation of the five statistics is 0.37 to 0.50, and the effective sample size is between 185 and 232 out of 593 draws. That is well above the target of 64, which refers to the estimating equations jointly (a smaller number than that of any single statistic) and which the iterations whose convergence test failed had raised. Geweke's test does not reject stationarity for any statistic ($|z|$ is at most 0.33), and the Heidelberger–Welch test passes. A fit with a burn-in of 3000, an interval of 50 proposals and fixed-size samples of 3000 draws (`mcmleeffectivesize(0) mcmcburnin(3000) mcmcinterval(50)`) shows what a chain that is too short looks like:

```
. nwergm lazega_fr, edges mutual nodematch(prac) nodeicov(seniority) gwesp(0.5) seed(1) mcmleeff
> ectivesize(0) mcmcburnin(3000) mcmcinterval(50)

Exponential-family random graph model

Network:               =  lazega_fr
Nodes:                 =  71
Ties:                  =  575
Directed:              =  Yes
Estimation:            =  MCMLE
MCMC sample size:      =  3000

------------------------------------------------------------------------------
         lazega_fr | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
-------------------+----------------------------------------------------------
             edges |  -3.299282    .161064   -20.48   0.000    -3.614962   -2.983602
           mutual |   .8312222   .1500579     5.54   0.000     .5371141     1.12533
    nodematch_prac |   .7169036   .0846234     8.47   0.000     .5510448    .8827624
nodeicov_seniority |  -.0052748   .0020911    -2.52   0.012    -.0093732   -.0011764
```

```
         gwesp_0.5 |   .4898494   .0651695     7.52   0.000     .3621196    .6175792
------------------------------------------------------------------------------

MCMLE converged after 1 iteration(s).
Evaluating log-likelihood at the estimate (bridge sampling with 16 bridges) ...

             Null Deviance:    6889.9  on 4970 degrees of freedom
         Residual Deviance:    3335.1  on 4965 degrees of freedom

Log-likelihood: -1667.5558 (df = 5)   AIC:    3345.1   BIC:    3377.7
Monte Carlo standard error of the log-likelihood:  .03905
```

The coefficients move by less than their standard errors, and the standard errors are of the same size. The diagnostics of the short chain:

```
. estat mcmcdiag, heidel

MCMC diagnostics, final sample             Sample size      =      3,000
                                           Burn-in          =      3,000
                                           Interval         =         50
                                           Acceptance rate  =     0.5716

------------------------------------------------------------------------
         Statistic |      Mean        SD  Autocorr       ESS   Geweke z
-------------------+----------------------------------------------------
             edges |    579.88     37.80     0.990      16.0      1.16
           mutual |     75.96     11.21     0.984      22.7      1.30
    nodematch_prac |    416.38     32.76     0.991      13.0      1.36
nodeicov_seniority |  19186.31   1434.38     0.989      17.5      1.77
         gwesp_0.5 |    568.80     73.55     0.988      20.0      1.36
------------------------------------------------------------------------

Heidelberger-Welch test           pvalue           =      0.050
                                  eps              =       0.10
-------------------------------------------------------------
         Statistic | Stationary    Start  Halfwidth  Halfwidth
                   |                                     test
-------------------+-----------------------------------------
             edges |     passed        1      18.54     passed
           mutual |     passed        1       4.61     passed
    nodematch_prac |     passed        1      17.82     passed
nodeicov_seniority |     passed        1     671.95     passed
         gwesp_0.5 |     passed        1      32.25     passed
-------------------------------------------------------------
```

With the short chain the lag-1 autocorrelation of every statistic is 0.98 to 0.99 and the effective sample size is between 13 and 23 out of 3000 draws, whereas Geweke's test does not reject stationarity ($|z|$ is at most 1.77, $p = 0.077$) and the Heidelberger–Welch test passes. The stationarity tests do not reveal the problem here; the autocorrelation and the effective sample size do, and the coefficients carry a large Monte Carlo error. The adaptive sample is protected against this. With the short interval and burn-in as starting values, but the adaptive sample of the default, the fit does not stay at the short interval; it thins the sample until the effective sample size is reached:

```
. quietly nwergm lazega_fr, edges mutual nodematch(prac) nodeicov(seniority) gwesp(0.5) seed(1)
> mcmcburnin(3000) mcmcinterval(50)

. quietly estat mcmcdiag

. display "adaptive sample, starting interval 50: " e(mcmle_iterations) " iterations, " e(mcmc_s
> amplesize) " draws, interval of the last sample " e(mcmc_interval_final)
adaptive sample, starting interval 50: 10 iterations, 948 draws, interval of the last sample 320
```

```
> 0

. matrix list r(ess), format(%9.1f)

r(ess)[1,5]
           edges        mutual  nodematch_~c  nodeicov_s~y     gwesp_0.5
r1         280.7         396.4         284.2         305.3         261.7
```

The fit takes ten iterations, the interval of the last sample is 3200 proposals, and the effective sample size of the 948 draws kept is between 262 and 396. These settings are a bad choice for the *fixed* chain, not of the starting values of the adaptive one. For the default fit, option `report` adds the report of R's `mcmc.diagnostics()`; one of its tables tests whether the sample statistics differ from the observed ones, with Hotelling's omnibus test:

```
Difference between the sample and the observed statistics

---------------------------------------------------
         Statistic |     Diff.       Test    p-value
-------------------+-------------------------------
             edges |     4.022      1.437      .1508
            mutual |   -.04216    -.05385      .9571
    nodematch_prac |     2.535      1.171      .2415
nodeicov_seniority |     184.6      1.913     .05572
         gwesp_0.5 |     6.675      1.272      .2032
           Omnibus |          .      14.18     .01676
---------------------------------------------------
```

None of the five individual tests rejects (the smallest p-value is 0.056), but Hotelling's omnibus test, which takes the very high cross-correlations of the five statistics into account, does ($p = 0.017$). The omnibus test is sensitive to a small joint displacement along the direction in which the statistics move together, and the sample was drawn at the coefficients before the last update of the iteration (the note at the end of the report; Section 7.1). It is a reason for caution, for example refitting from the estimate with `init()` and a larger `mcmleeffectivesize()`, and not a verdict on the final model, whose check is `estat gof`. Figure 7.1 shows the trace and density plots of this sample.

8.3.2 Goodness of fit

`estat gof` compares the observed network with 100 networks simulated from the fitted model. For a directed network the default statistics are the model statistics, the geodesic distances, the out-degrees, the in-degrees and the edgewise shared partners; the first three blocks are shown:

```
. estat gof, nsim(100) seed(2)

Goodness of fit: model statistics (100 simulated networks)
-------------------------------------------------------------------------
         Statistic |  Observed       Min      Mean       Max  MC p-value
-------------------+-----------------------------------------------------
             edges |       575       483    572.94       682        0.96
            mutual |        74        47     74.90       104        1.00
    nodematch_prac |       412       343    409.70       488        0.96
nodeicov_seniority |     19379     15591  19349.15     23426        0.98
         gwesp_0.5 |    555.62    386.57    551.32    734.20        1.00
-------------------------------------------------------------------------

Goodness of fit: minimum geodesic distance (100 simulated networks)
---------------------------------------------------------------------
    Distance |  Observed       Min      Mean       Max  MC p-value
-------------+-------------------------------------------------------
           1 |       575       483    572.94       682        0.96
```

```
           2 |      2316      2041   2497.67      2955        0.36
           3 |      1782      1310   1735.35      2157        0.84
           4 |       154        21    131.41       350        0.58
           5 |         3         0      3.87        59        0.62
           6 |         0         0      0.11         5        1.00
         Inf |       140         0     28.65       210        0.12
-------------------------------------------------------------------

Goodness of fit: out-degree (100 simulated networks)
-------------------------------------------------------------------
  Out-degree |  Observed       Min      Mean       Max  MC p-value
-------------+-----------------------------------------------------
           0 |         2         0      0.22         3        0.06
           1 |         1         0      0.81         3        1.00
           2 |         2         0      1.71         7        1.00
           3 |         7         0      3.26        12        0.12
           4 |         5         0      4.78        12        1.00
           5 |         5         1      6.30        12        0.86
           6 |         6         3      7.84        17        0.72
           7 |         6         1      8.08        16        0.56
           8 |         9         4      7.79        16        0.62
           9 |         3         1      7.13        13        0.16
          10 |         7         1      6.19        12        0.80
          11 |         4         0      5.27        12        0.72
          12 |         3         0      3.94        11        0.88
          13 |         3         0      2.81         9        1.00
          14 |         1         0      1.96         6        0.86
          15 |         1         0      1.22         7        1.00
          16 |         2         0      0.82         4        0.34
          17 |         1         0      0.36         2        0.64
          18 |         1         0      0.29         2        0.48
          19 |         0         0      0.12         2        1.00
          20 |         1         0      0.06         1        0.12
          21 |         0         0      0.02         1        1.00
          22 |         1         0      0.02         1        0.04
-------------------------------------------------------------------
```

The model statistics are reproduced (the observed value is inside the range of the simulations, with p-values between 0.94 and 1.00). The geodesic distances show a hint of a discrepancy: in the observed network 140 ordered pairs of lawyers cannot reach each other (distance `Inf`), whereas the simulated networks have 29 such pairs on average (Monte Carlo $p = 0.12$ on that row). A few small p-values among some eighty rows, which are not independent, are not surprising (the smallest in the blocks shown is 0.04, for an out-degree of 22), but this row points to a feature, a few lawyers who are reached by few others, that the model may not capture. Figure 7.2 shows the same comparison as a plot. The tables are matrices; for example the edgewise shared partners, with the statistics chosen explicitly:

```
. estat gof, gof(idegree odegree espartners) nsim(100) seed(2)

Goodness of fit: model statistics (100 simulated networks)
-------------------------------------------------------------------------
         Statistic |  Observed       Min      Mean       Max  MC p-value
-------------------+-----------------------------------------------------
             edges |       575       456    573.20       671        0.98
            mutual |        74        52     73.92        97        0.98
    nodematch_prac |       412       334    409.14       489        0.96
nodeicov_seniority |     19379     16529  19346.53     22358        0.98
         gwesp_0.5 |    555.62    361.07    554.14    760.38        0.98
-------------------------------------------------------------------------

Goodness of fit: out-degree (100 simulated networks)
-------------------------------------------------------------------
```

```
  Out-degree |  Observed       Min      Mean       Max  MC p-value
-------------+-----------------------------------------------------
           0 |         2         0      0.24         2        0.06
           1 |         1         0      0.89         4        1.00
           2 |         2         0      1.59         6        0.98
           3 |         7         0      3.31        11        0.16
           4 |         5         0      4.91        12        1.00
           5 |         5         1      6.46        13        0.64
           6 |         6         1      7.49        13        0.66
           7 |         6         3      8.29        16        0.52
           8 |         9         2      7.36        16        0.58
           9 |         3         1      7.17        14        0.16
          10 |         7         2      6.52        14        0.88
          11 |         4         1      5.33        11        0.72
          12 |         3         0      3.77         9        0.98
          13 |         3         0      2.61         7        0.96
          14 |         1         0      2.03         7        0.82
          15 |         1         0      1.23         4        1.00
          16 |         2         0      0.69         4        0.22
          17 |         1         0      0.52         3        0.86
          18 |         1         0      0.31         3        0.48
          19 |         0         0      0.14         2        1.00
          20 |         1         0      0.07         1        0.14
          21 |         0         0      0.05         1        1.00
          22 |         1         0      0.02         1        0.04
-------------------------------------------------------------------

Goodness of fit: in-degree (100 simulated networks)
-------------------------------------------------------------------
   In-degree |  Observed       Min      Mean       Max  MC p-value
-------------+-----------------------------------------------------
           0 |         0         0      0.21         2        1.00
           1 |         0         0      0.85         4        0.80
           2 |         3         0      1.95         7        0.60
           3 |         2         0      3.85         9        0.52
           4 |         8         0      4.85        11        0.30
           5 |         6         2      6.15        15        1.00
           6 |         5         3      7.58        15        0.48
           7 |         8         2      7.38        19        0.98
           8 |         9         1      7.69        15        0.74
           9 |         5         3      7.32        15        0.42
          10 |         7         0      6.08        11        0.86
          11 |         6         1      5.14        10        0.78
          12 |         4         0      3.89         8        1.00
          13 |         2         0      2.66         7        1.00
          14 |         4         0      2.18         7        0.42
          15 |         1         0      1.24         4        1.00
          16 |         0         0      0.78         3        0.94
          17 |         1         0      0.49         3        0.76
          18 |         0         0      0.28         3        1.00
          19 |         0         0      0.18         2        1.00
          20 |         0         0      0.12         2        1.00
          21 |         0         0      0.07         1        1.00
          22 |         0         0      0.05         1        1.00
          25 |         0         0      0.01         1        1.00
-------------------------------------------------------------------

Goodness of fit: edgewise shared partner (100 simulated networks)
-------------------------------------------------------------------
         ESP |  Observed       Min      Mean       Max  MC p-value
-------------+-----------------------------------------------------
           0 |       138        88    131.53       165        0.76
           1 |       190       163    201.20       238        0.40
           2 |       131        79    140.21       198        0.66
```

```
          3 |        69        29     66.33       113        0.86
          4 |        26         5     24.16        57        0.80
          5 |        11         0      7.11        17        0.50
          6 |         7         0      1.93         6        0.00
          7 |         2         0      0.61         4        0.30
          8 |         1         0      0.09         2        0.16
          9 |         0         0      0.01         1        1.00
         10 |         0         0      0.02         1        1.00
-------------------------------------------------------------------
```

```
. matrix list r(gof_espartners)

r(gof_espartners)[70,5]
           obs         min        mean         max  MC_p-value
 0         138          88      131.53         165         .76
 1         190         163       201.2         238          .4
 2         131          79      140.21         198         .66
 3          69          29       66.33         113         .86
 4          26           5       24.16          57          .8
 5          11           0        7.11          17          .5
 6           7           0        1.93           6           0
 7           2           0         .61           4          .3
 8           1           0         .09           2         .16
 9           0           0         .01           1           1
10           0           0         .02           1           1
11           0           0           0           0           1
12           0           0           0           0           1
13           0           0           0           0           1
14           0           0           0           0           1
15           0           0           0           0           1
16           0           0           0           0           1
17           0           0           0           0           1
18           0           0           0           0           1
19           0           0           0           0           1
20           0           0           0           0           1
21           0           0           0           0           1
22           0           0           0           0           1
23           0           0           0           0           1
24           0           0           0           0           1
25           0           0           0           0           1
26           0           0           0           0           1
27           0           0           0           0           1
28           0           0           0           0           1
29           0           0           0           0           1
30           0           0           0           0           1
31           0           0           0           0           1
32           0           0           0           0           1
33           0           0           0           0           1
34           0           0           0           0           1
35           0           0           0           0           1
36           0           0           0           0           1
37           0           0           0           0           1
38           0           0           0           0           1
39           0           0           0           0           1
40           0           0           0           0           1
41           0           0           0           0           1
42           0           0           0           0           1
43           0           0           0           0           1
44           0           0           0           0           1
45           0           0           0           0           1
46           0           0           0           0           1
47           0           0           0           0           1
48           0           0           0           0           1
```

```
49          0          0          0          0          1
50          0          0          0          0          1
51          0          0          0          0          1
52          0          0          0          0          1
53          0          0          0          0          1
54          0          0          0          0          1
55          0          0          0          0          1
56          0          0          0          0          1
57          0          0          0          0          1
58          0          0          0          0          1
59          0          0          0          0          1
60          0          0          0          0          1
61          0          0          0          0          1
62          0          0          0          0          1
63          0          0          0          0          1
64          0          0          0          0          1
65          0          0          0          0          1
66          0          0          0          0          1
67          0          0          0          0          1
68          0          0          0          0          1
69          0          0          0          0          1
```

8.3.3 Simulation from the fit

`nwergm simulate` without a number of nodes draws from the last fit. The statistics of four draws (with `output(stats)`), and two networks that are created as `nwcommands` networks:

```
. nwergm simulate, nsim(4) output(stats) seed(3)

. matrix list r(stats)

r(stats)[4,5]
           edges        mutual  nodematch_~c  nodeicov_s~y     gwesp_0.5
r1           559            63           392         18272     534.23323
r2           584            73           431         19942     600.66471
r3           583            79           439         20271      569.6952
r4           557            62           384         18387     489.90956
```

```
. nwergm simulate, nsim(2) output(network) generate(lzsim) seed(3)
(2 simulated network(s) created: lzsim_1 ... lzsim_2)

. nwset
(5 networks)
--------------------
      lazega_adv
      lazega_cow
      lazega_fr
      lzsim_1
      lzsim_2

. nwsummarize lzsim_1
-------------------------------------------------
   Network name:  lzsim_1
   Network id:  4
   Directed: true
   Valued: false
   Two-mode: false
   Nodes: 71
   Selfloop: false
   Arcs: 559
   Minimum value:  0
   Maximum value:  1
```

```
   Density:  .112
   Temporal: false
```

Networks can also be drawn from a model that was never fitted, by giving the coefficients; this draws three directed networks with 20 nodes, $\theta_{\texttt{edges}} = -2$ and $\theta_{\texttt{mutual}} = 1$:

```
. nwergm simulate 20, edges mutual theta(-2 1) directed nsim(3) seed(1) generate(toy)

. nwset
(8 networks)
--------------------
      lazega_adv
      lazega_cow
      lazega_fr
      lzsim_1
      lzsim_2
      toy_1
      toy_2
      toy_3
```

8.4 Kapferer's tailor shop: a degenerate model

The network is the tailor-shop network of Kapferer [18] at the second time point: 43 workers and 190 ties, undirected. A model with edges and triangles is a classical example of a *degenerate* ERGM: the triangle statistic grows so fast with the number of ties that the model puts nearly all its probability on empty and complete networks, the sampler cannot produce a sample whose statistics vary, and the fit stops:

```
. nwwebuse kapferer, nwclear

. nwset
(2 networks)
--------------------
      kapferer_t1
      kapferer_t2

. nwsummarize kapferer_t2
-------------------------------------------------
   Network name:  kapferer_t2
   Network id:  2
   Directed: false
   Valued: false
   Two-mode: false
   Nodes: 43
   Selfloop: false
   Edges: 190
   Minimum value:  0
   Maximum value:  1
   Density:  .21
   Temporal: false
```

```
. capture noisily nwergm kapferer_t2, edges triangle seed(1) mcmcsamplesize(1000)
Warning: MCMLE iter 1 - statistic(s) at column(s) 1 2 not varying this iteration - this may indi
> cate an extreme point or dead-end configuration.
Unconstrained MCMC sampling did not mix at all. Optimization cannot continue.

. display "return code: " _rc
return code: 498
```

The message *unconstrained MCMC sampling did not mix at all* is the error of `ergm` for this situation, and the return code is 498. The remedy is the geometrically weighted version of the triangle term, which discounts additional shared partners:

```
. nwergm kapferer_t2, edges gwesp(0.5) seed(1)

Exponential-family random graph model

Network:               =  kapferer_t2
Nodes:                 =  43
Ties:                  =  190
Directed:              =  No
Estimation:            =  MCMLE
MCMC sample size:      =  497

------------------------------------------------------------------------------
 kapferer_t2 | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
-------------+----------------------------------------------------------------
       edges |  -4.282998   .4164744   -10.28   0.000    -5.099272   -3.466723
   gwesp_0.5 |   1.518992   .2358945     6.44   0.000     1.056648    1.981337
------------------------------------------------------------------------------

MCMLE converged after 6 iteration(s).
Evaluating log-likelihood at the estimate (bridge sampling with 16 bridges) ...

             Null Deviance:    1251.8  on 903 degrees of freedom
         Residual Deviance:     859.4  on 901 degrees of freedom

Log-likelihood:  -429.6927 (df = 2)   AIC:     863.4   BIC:     873.0
Monte Carlo standard error of the log-likelihood:   .1643
```

The model converges (after six iterations). The goodness-of-fit table shows, however, that the fitted model is not far from the degenerate region: the simulated networks range from 135 to 227 ties against the observed 190, and many simulated networks leave pairs unconnected (49 pairs on average), which the observed network does not.

```
. estat gof, nsim(100) seed(2)

Goodness of fit: model statistics (100 simulated networks)
-------------------------------------------------------------------
   Statistic |  Observed       Min      Mean       Max  MC p-value
-------------+-----------------------------------------------------
       edges |       190       135    189.22       227        0.86
   gwesp_0.5 |    275.77    184.80    274.60    343.08        0.88
-------------------------------------------------------------------

Goodness of fit: minimum geodesic distance (100 simulated networks)
-------------------------------------------------------------------
    Distance |  Observed       Min      Mean       Max  MC p-value
-------------+-----------------------------------------------------
           1 |       190       135    189.22       227        0.86
           2 |       535       390    527.63       613        0.88
           3 |       163        49    131.90       242        0.40
           4 |        15         0      4.76        39        0.20
           5 |         0         0      0.27        20        1.00
           6 |         0         0      0.01         1        1.00
         Inf |         0         0     49.21       200        0.60
-------------------------------------------------------------------
```

Such a model should be checked with a longer chain and with the diagnostics before it is interpreted.

8.5 Missing dyads

Missing dyads are tie values that are missing in the network. Here three marriages of the Florentine network are blanked out (set to missing in both directions), by editing the adjacency matrix in Mata and declaring a new network:

```
. nwwebuse florentine, nwclear

. nwtomata flomarriage, mat(A)

. mata: A[1,2] = A[2,1] = .

. mata: A[3,5] = A[5,3] = .

. mata: A[4,9] = A[9,4] = .

. mata: st_matrix("Am", A)

. tempfile nodes

. save `nodes'
file /var/folders/hx/2zp2k5ps2y7d_6hx015fcr1h0000gn/T//S_61211.000001 saved as .dta format

. nwset, mat(Am) undirected name(flomiss)

. use `nodes', clear
```

The dyad-independent model drops the missing dyads from the likelihood: the 120 dyads of the complete network become 117 informative dyads, and the coefficients change:

```
. nwergm flomarriage, edges nodecov(wealth)

Exponential-family random graph model

Network:               =  flomarriage
Nodes:                 =  16
Ties:                  =  20
Directed:              =  No
Estimation:            =  MPLE

------------------------------------------------------------------------------
   flomarriage | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
---------------+--------------------------------------------------------------
         edges |  -2.594929   .5360561    -4.84   0.000     -3.64558   -1.544278
nodecov_wealth |   .0105459   .0046743     2.26   0.024     .0013845    .0197073
------------------------------------------------------------------------------

Maximum pseudolikelihood estimate; this is the maximum likelihood estimate only if the model is
    dyad-independent.

            Null Deviance:     166.4  on 120 degrees of freedom
        Residual Deviance:     103.1  on 118 degrees of freedom

Log-likelihood:   -51.5543 (df = 2)   AIC:     107.1   BIC:     112.7

. nwergm flomiss, edges nodecov(wealth)
Note: network flomiss has 3 missing dyad(s); they are left out of the likelihood, as in ergm.

Exponential-family random graph model

Network:               =  flomiss
Nodes:                 =  16
Ties:                  =  19
```

```
Directed:              =  No
Estimation:            =  MPLE

------------------------------------------------------------------------------
       flomiss | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
---------------+--------------------------------------------------------------
         edges |  -2.692927    .552001    -4.88   0.000    -3.774829   -1.611025
nodecov_wealth |   .0111924   .0047613     2.35   0.019     .0018604    .0205244
------------------------------------------------------------------------------

Maximum pseudolikelihood estimate; this is the maximum likelihood estimate only if the model is
    dyad-independent.

             Null Deviance:     162.2  on 117 degrees of freedom
         Residual Deviance:      98.3  on 115 degrees of freedom

Log-likelihood:   -49.1649 (df = 2)   AIC:     102.3   BIC:     107.9

. display "missing dyads: " e(nmissing) "   informative dyads: " e(N)
missing dyads: 3   informative dyads: 117
```

For a dyad-dependent model, the missing dyads are handled by the method of Handcock and Gile (Section 2.4); the output notes that the missing dyads are handled as in `ergm`:

```
. nwergm flomiss, edges nodecov(wealth) gwesp(0.5) seed(1)
Note: network flomiss has 3 missing dyad(s); they are left out of the likelihood, as in ergm.

Exponential-family random graph model

Network:               =  flomiss
Nodes:                 =  16
Ties:                  =  19
Directed:              =  No
Estimation:            =  MCMLE
MCMC sample size:      =  486

------------------------------------------------------------------------------
       flomiss | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
---------------+--------------------------------------------------------------
         edges |  -2.678908   .5691385    -4.71   0.000    -3.794399   -1.563417
nodecov_wealth |   .0105619   .0054018     1.96   0.051    -.0000255    .0211493
     gwesp_0.5 |   .0452257   .2927755     0.15   0.877    -.5286037    .6190551
------------------------------------------------------------------------------

MCMLE converged after 1 iteration(s).
Evaluating log-likelihood at the estimate (bridge sampling with 16 bridges) ...

             Null Deviance:     162.2  on 117 degrees of freedom
         Residual Deviance:      98.3  on 114 degrees of freedom

Log-likelihood:   -49.1518 (df = 3)   AIC:     104.3   BIC:     112.6
Monte Carlo standard error of the log-likelihood:  .00288

. display "missing dyads: " e(nmissing) "   informative dyads: " e(N)
missing dyads: 3   informative dyads: 117
```

A missing *covariate* value is refused, as in `ergm`:

```
. preserve

. replace wealth = . in 3
(1 real change made, 1 to missing)
```

```
. capture noisily nwergm flomarriage, edges nodecov(wealth)
In term nodecov: attribute wealth has missing data, which is not currently supported.

. display "return code: " _rc
return code: 416

. restore
```

8.6 Comparing models and predicting ties

Two models of the Lazega network, with and without the shared-partner term, are stored and compared. The log-likelihoods come from bridge sampling and carry a small Monte Carlo error (`e(ll_se)`); the fits use a long interval and burn-in (`mcmcinterval(1000) mcmcburnin(20000)`) as starting values of the adaptive sample.

```
. nwergm lazega_fr, edges mutual nodematch(prac) seed(1) mcmcinterval(1000) mcmcburnin(20000)

Exponential-family random graph model

Network:               =  lazega_fr
Nodes:                 =  71
Ties:                  =  575
Directed:              =  Yes
Estimation:            =  MCMLE
MCMC sample size:      =  745

------------------------------------------------------------------------------
     lazega_fr | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
---------------+--------------------------------------------------------------
         edges |  -2.739039   .0787711   -34.77   0.000    -2.893428   -2.584651
        mutual |   .9977665    .166355     6.00   0.000     .6717168    1.323816
nodematch_prac |   .9018349   .0939423     9.60   0.000     .7177113    1.085958
------------------------------------------------------------------------------

MCMLE converged after 5 iteration(s).
Evaluating log-likelihood at the estimate (bridge sampling with 16 bridges) ...

             Null Deviance:    6889.9  on 4970 degrees of freedom
         Residual Deviance:    3404.6  on 4967 degrees of freedom

Log-likelihood: -1702.3120 (df = 3)   AIC:    3410.6   BIC:    3430.2
Monte Carlo standard error of the log-likelihood:    .109

. estimates store m1
```

```
. nwergm lazega_fr, edges mutual nodematch(prac) gwesp(0.5) seed(1) mcmcinterval(1000)
    mcmcburnin(20000)

Exponential-family random graph model

Network:               =  lazega_fr
Nodes:                 =  71
Ties:                  =  575
Directed:              =  Yes
Estimation:            =  MCMLE
MCMC sample size:      =  1085

------------------------------------------------------------------------------
     lazega_fr | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
---------------+--------------------------------------------------------------
```

```
         edges |  -3.473913   .1137593   -30.54   0.000    -3.696878   -3.250949
        mutual |   .8059472   .1588423     5.07   0.000      .494622    1.117272
nodematch_prac |    .702382   .0806238     8.71   0.000     .5443623    .8604016
     gwesp_0.5 |   .4927435   .0650009     7.58   0.000      .365344    .6201429
------------------------------------------------------------------------------

MCMLE converged after 7 iteration(s).
Evaluating log-likelihood at the estimate (bridge sampling with 16 bridges) ...

             Null Deviance:    6889.9  on 4970 degrees of freedom
         Residual Deviance:    3344.1  on 4966 degrees of freedom

Log-likelihood: -1672.0729 (df = 4)   AIC:    3352.1   BIC:    3378.2
Monte Carlo standard error of the log-likelihood:  .07806

. estimates store m2
```

`estimates stats` and `lrtest` compare them:

```
. estimates stats m1 m2

Akaike's information criterion and Bayesian information criterion

------------------------------------------------------------------------------
       Model |          N   ll(null)  ll(model)      df        AIC        BIC
-------------+----------------------------------------------------------------
          m1 |      4,970  -3444.941  -1702.312       3   3410.624   3430.157
          m2 |      4,970  -3444.941  -1672.073       4   3352.146    3378.19
------------------------------------------------------------------------------
Note: BIC uses N = number of observations. See [R] IC note.
```

```
. lrtest m2 m1

Likelihood-ratio test
Assumption: m1 nested within m2

 LR chi2(1) =  60.48
Prob > chi2 = 0.0000
```

The shared-partner term improves the log-likelihood by 30.2, so that the likelihood-ratio statistic is 60.5 on one degree of freedom; AIC and BIC both prefer the model with `gwesp(0.5)`. `predict` gives the conditional probability of each dyad, as a frame (one row per dyad) or a matrix:

```
. predict, frame(pp) pr
(frame pp contains one row per informative dyad: tail, head, p = conditional tie probability)

. frame pp: summarize p

    Variable |        Obs        Mean    Std. dev.       Min        Max
-------------+---------------------------------------------------------
           p |      4,970    .1151156    .0914372   .0300637   .7969506

. frame pp: list in 1/5

     +-------------------------+
     | tail   head           p |
     |-------------------------|
  1. |    1      2   .03006366 |
  2. |    1      3   .35609851 |
  3. |    1      4   .10199767 |
  4. |    1      5    .3038023 |
  5. |    1      6   .03626253 |
```

```
     +-------------------------+
```

```
. predict, matrix(P) pr
(matrix P contains the conditional tie probabilities; missing dyads are .)

. matrix P5 = P[1..5, 1..5]

. matrix list P5, format(%7.4f)

P5[5,5]
        c1      c2      c3      c4      c5
r1  0.0000  0.0301  0.3561  0.1020  0.3038
r2  0.0483  0.0000  0.0301  0.0929  0.0558
r3  0.1311  0.0301  0.0000  0.1434  0.1401
r4  0.0580  0.0589  0.0845  0.0000  0.0917
r5  0.1267  0.0580  0.1437  0.2850  0.0000
```

The average predicted probability, 0.115, is close to the density of the network, 0.116.

8.7 Constraints and observation processes

`constraints(edges)` holds the number of ties fixed at the observed number; the density term is then not estimated (it is reported as omitted, with a note), and the other coefficients are those of a model conditional on the density. Florentine, with and without the constraint:

```
. nwergm flomarriage, edges nodecov(wealth) gwesp(0.5) constraints(edges) seed(1)
Note: the constraints hold edges constant; they are dropped from the model.

Exponential-family random graph model

Network:               =  flomarriage
Nodes:                 =  16
Ties:                  =  20
Directed:              =  No
Estimation:            =  MCMLE
MCMC sample size:      =  243

------------------------------------------------------------------------------
   flomarriage | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
---------------+--------------------------------------------------------------
         edges |          0  (omitted)
nodecov_wealth |   .0097449   .0049592     1.97   0.049      .000025    .0194649
     gwesp_0.5 |   .0487482   .2489059     0.20   0.845    -.4390985    .5365949
------------------------------------------------------------------------------

MCMLE converged after 1 iteration(s).
Evaluating log-likelihood at the estimate (bridge sampling with 16 bridges) ...

             Null Deviance:     166.4  on 120 degrees of freedom
         Residual Deviance:     110.9  on 118 degrees of freedom

Log-likelihood:   -55.4304 (df = 2)   AIC:     114.9   BIC:     120.4
```

`constraints(odegrees)` holds the out-degree of every node fixed, which fixes the number of ties as well. In the Lazega network the density is again omitted, with a note:

```
. nwergm lazega_fr, edges mutual nodematch(prac) constraints(odegrees) seed(1)
Note: the constraints hold edges constant; they are dropped from the model.

Exponential-family random graph model
```

```
Network:               =  lazega_fr
Nodes:                 =  71
Ties:                  =  575
Directed:              =  Yes
Estimation:            =  MCMLE
MCMC sample size:      =  243

------------------------------------------------------------------------------
     lazega_fr | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
---------------+--------------------------------------------------------------
         edges |          0  (omitted)
        mutual |   1.011941   .1496752     6.76   0.000     .7185833    1.305299
nodematch_prac |   .9500744   .0940187    10.11   0.000     .7658011    1.134348
------------------------------------------------------------------------------

MCMLE converged after 1 iteration(s).
Evaluating log-likelihood at the estimate (bridge sampling with 16 bridges) ...

             Null Deviance:    6889.9  on 4970 degrees of freedom
         Residual Deviance:    4783.1  on 4968 degrees of freedom

Log-likelihood: -2391.5270 (df = 2)   AIC:    4787.1   BIC:    4800.1
Monte Carlo standard error of the log-likelihood:  .38296
```

When a constraint fixes the sample space, `nwergm simulate` from the fit keeps it: the three simulated networks have exactly 575 ties, the number of ties of the data, and the out-degree of every node is that of the data.

```
. nwergm simulate, nsim(3) output(stats) seed(1)

. matrix list r(stats)

r(stats)[3,3]
           edges        mutual  nodematch_~c
r1           575            74           422
r2           575            68           409
r3           575            66           404
```

A constraint can also confine the model to part of the network. With `constraints(fixallbut(boston))`, where `boston` is a network that has a tie for every pair of lawyers in the Boston office, only the dyads among the Boston lawyers are modeled and all other dyads are held at their observed values. The network `boston` is built from the variable `office` in Mata:

```
. nwwebuse lazega, nwclear

. encode practice, generate(prac)

. encode office, generate(off)

. tempfile lnodes

. save `lnodes'
file /var/folders/hx/2zp2k5ps2y7d_6hx015fcr1h0000gn/T//S_61299.000001 saved as .dta format

. mata: o = st_data(., "off"); F = (o :== 1) * (o :== 1)'; _diag(F, 0); st_matrix("F", F)

. nwset, mat(F) directed name(boston)

. use `lnodes', clear
```

```
. nwergm lazega_fr, edges mutual nodematch(prac) constraints(fixallbut(boston)) seed(1)
```

```
Exponential-family random graph model

Network:               =  lazega_fr
Nodes:                 =  71
Ties:                  =  575
Directed:              =  Yes
Estimation:            =  MCMLE
MCMC sample size:      =  373

------------------------------------------------------------------------------
     lazega_fr | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
---------------+--------------------------------------------------------------
         edges |  -2.467572   .1052988   -23.43   0.000    -2.673954    -2.26119
        mutual |   .7726361   .1882545     4.10   0.000      .403664    1.141608
nodematch_prac |   1.007996   .1286134     7.84   0.000     .7559185    1.260074
------------------------------------------------------------------------------

MCMLE converged after 2 iteration(s).
Evaluating log-likelihood at the estimate (bridge sampling with 16 bridges) ...

             Null Deviance:    3127.5  on 2256 degrees of freedom
         Residual Deviance:    1839.1  on 2253 degrees of freedom

Log-likelihood:  -919.5451 (df = 3)   AIC:    1845.1   BIC:    1862.3
Monte Carlo standard error of the log-likelihood:  .07357
```

The number of informative dyads is $48 \times 47 = 2256$.

8.7.1 An observation process

`obsconstraints()` describes how the network was observed. The three Florentine marriages of Section 8.5 are now declared as unobserved by a network `unobs` that has a tie for each of the three pairs, and the observed network `flo0` has a 0 there:

```
. nwwebuse florentine, nwclear

. nwtomata flomarriage, mat(A)

. mata: A[1,2] = A[2,1] = 0

. mata: A[3,5] = A[5,3] = 0

. mata: A[4,9] = A[9,4] = 0

. mata: U = J(16, 16, 0)

. mata: U[1,2] = U[2,1] = U[3,5] = U[5,3] = U[4,9] = U[9,4] = 1

. mata: st_matrix("A0", A); st_matrix("U", U)

. tempfile nodes

. save ‘nodes’
file /var/folders/hx/2zp2k5ps2y7d_6hx015fcr1h0000gn/T//S_61299.000002 saved as .dta format

. nwset, mat(A0) undirected name(flo0)

. nwset, mat(U) undirected name(unobs)

. use ‘nodes’, clear
```

```
. nwergm flo0, edges nodecov(wealth) gwesp(0.5) obsconstraints(fixallbut(unobs)) seed(1)

Exponential-family random graph model

Network:               =  flo0
Nodes:                 =  16
Ties:                  =  19
Directed:              =  No
Estimation:            =  MCMLE
MCMC sample size:      =  3000

------------------------------------------------------------------------------
          flo0 | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
---------------+--------------------------------------------------------------
         edges |  -2.695638   .5630754    -4.79   0.000    -3.799245    -1.59203
nodecov_wealth |    .010867    .005151     2.11   0.035     .0007713    .0209627
     gwesp_0.5 |   .0363212   .2771956     0.13   0.896    -.5069722    .5796147
------------------------------------------------------------------------------

MCMLE converged after 2 iteration(s).
Evaluating log-likelihood at the estimate (bridge sampling with 16 bridges) ...

             Null Deviance:     162.2  on 117 degrees of freedom
         Residual Deviance:      98.3  on 114 degrees of freedom

Log-likelihood:   -49.1485 (df = 3)   AIC:     104.3   BIC:     112.6
Monte Carlo standard error of the log-likelihood:  .00084
```

The result agrees with the missing-dyad fit of Section 8.5 (edges −2.6956 against −2.6789, wealth 0.0109 against 0.0106, shared partners 0.0363 against 0.0452, with standard errors of the same size; the difference is Monte Carlo error: the missing-dyad fit uses the adaptive sample, whereas a fit with `obsconstraints()` draws fixed samples of 3000, as the sample-size line of the listing shows): in both cases the likelihood of the network is the sum over the values of the three unobserved dyads, which `obsconstraints(fixallbut(unobs))` expresses directly.

8.8 A two-mode network, structural terms, operators and labels

Two-mode (bipartite) networks connect two sets of nodes. The network of this section is simulated for the manual: 30 people and 12 clubs, in which people and clubs have a binary type, and ties between the same type are more likely (30% against 8%). The network is built from the matrix `B`, transposed because `nwset` takes the first mode from the columns, so the 30 people are the first mode:

```
. clear

. set seed 20261024

. nwclear

. mata: g1 = (runiform(30,1) :> 0.5) :+ 1

. mata: g2 = (runiform(12,1) :> 0.5) :+ 1

. mata: P = 0.08 :+ 0.22 :* ((g1 * J(1,12,1)) :== (J(30,1,1) * rowshape(g2,1)))

. mata: B = (runiform(30,12) :< P)

. mata: st_matrix("B", transposeonly(B))

. nwset, mat(B) bipartite name(clubs)
```

```
. set obs 42
Number of observations (_N) was 42, now 42.

. generate byte kind = .
(42 missing values generated)

. mata: st_store(., "kind", (g1 \ g2))

. nwsummarize clubs
-------------------------------------------------
   Network name:  clubs
   Network id:  1
   Directed: false
   Valued: false
   Two-mode: true
   Nodes: 42
      Level 1: 30 ()
      Level 2: 12 ()
   Selfloop: false
   Edges: 71
   Minimum value:  0
   Maximum value:  1
   Density:  .197
   Temporal: false
```

A model with the density and the mixing of the type of person and club (`nodemix(kind)`; the cell names give the type of the first-mode node and of the second-mode node, and the first cell, both of type 1, is omitted) is dyad-independent:

```
. nwergm clubs, edges nodemix(kind)

Exponential-family random graph model

Network:               =  clubs
Nodes:                 =  42
Ties:                  =  71
Directed:              =  No
Estimation:            =  MPLE

------------------------------------------------------------------------------
           clubs | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
-----------------+------------------------------------------------------------
           edges |  -.7563261   .1787266    -4.23   0.000    -1.106624   -.4060283
nodemix_kind_2_1 |  -1.512357   .3931267    -3.85   0.000    -2.282872   -.7418434
nodemix_kind_1_2 |  -2.379168   .6162542    -3.86   0.000    -3.587004   -1.171332
nodemix_kind_2_2 |  -.2340726   .3707259    -0.63   0.528    -.9606821    .4925369
------------------------------------------------------------------------------

Maximum pseudolikelihood estimate; this is the maximum likelihood estimate only if the model
is dyad-independent.

             Null Deviance:     499.1  on 360 degrees of freedom
         Residual Deviance:     321.2  on 356 degrees of freedom

Log-likelihood:  -160.5841 (df = 4)   AIC:     329.2   BIC:     344.7
```

Ties between people and clubs of the same type are more likely than ties between different types: the coefficients of the two cells of different types are -1.5 and -2.4 relative to the cell in which both are of type 1, against -0.23 for the cell in which both are of type 2. A geometrically weighted degree term for the second mode (`bgwdegree2(0.5)`) makes the model dyad-dependent:

```
. nwergm clubs, edges nodemix(kind) bgwdegree2(0.5) seed(1)
```

```
Exponential-family random graph model

Network:               =  clubs
Nodes:                 =  42
Ties:                  =  71
Directed:              =  No
Estimation:            =  MCMLE
MCMC sample size:      =  243

------------------------------------------------------------------------------
           clubs | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
-----------------+------------------------------------------------------------
           edges |  -.7414179   .1930383    -3.84   0.000    -1.119766   -.3630697
nodemix_kind_2_1 |  -1.506756   .4068944    -3.70   0.000    -2.304254   -.7092575
nodemix_kind_1_2 |  -2.212356   .5936993    -3.73   0.000    -3.375985   -1.048727
nodemix_kind_2_2 |  -.0559743   .4063925    -0.14   0.890    -.8524889    .7405403
  bgwdegree2_0.5 |  -1.690794   1.394711    -1.21   0.225    -4.424378     1.04279
------------------------------------------------------------------------------

MCMLE converged after 2 iteration(s).
Evaluating log-likelihood at the estimate (bridge sampling with 16 bridges) ...

             Null Deviance:     499.1  on 360 degrees of freedom
         Residual Deviance:     320.2  on 355 degrees of freedom

Log-likelihood:  -160.1196 (df = 5)   AIC:     330.2   BIC:     349.7
Monte Carlo standard error of the log-likelihood:   .0943
```

The goodness of fit of a two-mode model compares the statistics of the model, the degree distributions of both modes, the dyadwise shared partners and the distances. The listing shows the first three blocks (the other blocks follow in the same layout):

```
. estat gof, gof(b1degree b2degree dspartners distance model) nsim(100) seed(2)

Goodness of fit: model statistics (100 simulated networks)
------------------------------------------------------------------------
       Statistic |  Observed       Min      Mean       Max  MC p-value
-----------------+------------------------------------------------------
           edges |        71        56     71.27        93        1.00
nodemix_kind_2_1 |         9         3      9.00        17        1.00
nodemix_kind_1_2 |         3         0      3.10         8        1.00
nodemix_kind_2_2 |        13         5     13.36        23        1.00
  bgwdegree2_0.5 |     18.78     16.31     18.88     19.72        0.60
------------------------------------------------------------------------

Goodness of fit: minimum geodesic distance (100 simulated networks)
------------------------------------------------------------------
    Distance |  Observed       Min      Mean       Max  MC p-value
-------------+----------------------------------------------------
           1 |        71        56     71.27        93        1.00
           2 |       213       137    206.81       308        0.84
           3 |       213       134    205.64       254        0.90
           4 |       238       123    200.77       268        0.30
           5 |        60         6     43.57       108        0.42
           6 |        21         0     21.24        92        0.78
           7 |         4         0      2.31        29        0.32
           8 |         0         0      1.17        28        1.00
           9 |         0         0      0.17        10        1.00
          10 |         0         0      0.04         4        1.00
         Inf |        41         0    108.01       333        0.60
------------------------------------------------------------------
```

```
Goodness of fit: bipartition 1 degree (100 simulated networks)
------------------------------------------------------------------
      Degree |  Observed       Min      Mean       Max  MC p-value
-------------+----------------------------------------------------
           0 |         1         0      2.39         8        0.64
           1 |         6         2      6.09        12        1.00
           2 |        12         2      8.32        14        0.26
           3 |         6         2      6.97        13        0.98
           4 |         2         0      4.09         9        0.50
           5 |         3         0      1.65         5        0.52
           6 |         0         0      0.42         3        1.00
           7 |         0         0      0.06         1        1.00
           8 |         0         0      0.01         1        1.00
------------------------------------------------------------------
```

Operators evaluate terms on a derived network. `Proj1(edges triangle)` evaluates the number of ties and triangles of the one-mode projection on the first mode (two people are tied if they belong to a common club):

```
. nwergm clubs, edges nodemix(kind) operators(Proj1(edges triangle)) seed(1)

Exponential-family random graph model

Network:               =  clubs
Nodes:                 =  42
Ties:                  =  71
Directed:              =  No
Estimation:            =  MCMLE
MCMC sample size:      =  509

------------------------------------------------------------------------------
           clubs | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
-----------------+------------------------------------------------------------
           edges |  -1.105045   .3662216    -3.02   0.003    -1.822826   -.3872639
nodemix_kind_2_1 |  -1.776232   .5313858    -3.34   0.001    -2.817729   -.7347348
nodemix_kind_1_2 |   -2.40565   .6836645    -3.52   0.000    -3.745607   -1.065692
nodemix_kind_2_2 |  -.2850296    .459771    -0.62   0.535    -1.186164     .616105
     Proj1~edges |   .1944617   .1914415     1.02   0.310    -.1807567    .5696801
  Proj1~triangle |  -.0139922   .0263967    -0.53   0.596    -.0657289    .0377444
------------------------------------------------------------------------------

MCMLE converged after 3 iteration(s).
Evaluating log-likelihood at the estimate (bridge sampling with 16 bridges) ...

             Null Deviance:     499.1  on 360 degrees of freedom
         Residual Deviance:     319.0  on 354 degrees of freedom

Log-likelihood:  -159.5072 (df = 6)   AIC:     331.0   BIC:     354.3
Monte Carlo standard error of the log-likelihood:  .16144
```

On the directed Lazega network, `Symmetrize(triangle, rule(strong))` counts the triangles of the network of reciprocated ties:

```
. nwwebuse lazega, nwclear

. encode practice, generate(prac)

. nwergm lazega_fr, edges mutual nodematch(prac) operators(Symmetrize(triangle, rule(strong))) s
> eed(1)

Exponential-family random graph model

Network:               =  lazega_fr
```

```
Nodes:                 =  71
Ties:                  =  575
Directed:              =  Yes
Estimation:            =  MCMLE
MCMC sample size:      =  766

------------------------------------------------------------------------------------------
                  lazega_fr | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
----------------------------+-------------------------------------------------------------
                      edges |  -2.736795   .0810857   -33.75   0.000     -2.89572    -2.57787
                     mutual |   .9130825   .1676719     5.45   0.000     .5844516    1.241713
             nodematch_prac |   .8967159   .0964408     9.30   0.000     .7076954    1.085736
Symmetrize(strong)~triangle |   .6922598   .4560459     1.52   0.129    -.2015737    1.586093
------------------------------------------------------------------------------------------

MCMLE converged after 5 iteration(s).
Evaluating log-likelihood at the estimate (bridge sampling with 16 bridges) ...

             Null Deviance:    6889.9  on 4970 degrees of freedom
         Residual Deviance:    3401.6  on 4966 degrees of freedom

Log-likelihood: -1700.7989 (df = 4)   AIC:    3409.6   BIC:    3435.6
Monte Carlo standard error of the log-likelihood:  .10562
```

Further structural terms of `ergm` are available inside `structural()`, here the two-paths and the 3-cycles:

```
. nwergm lazega_fr, edges mutual nodematch(prac) structural(twopath cycle(k=3)) seed(1)

Exponential-family random graph model

Network:               =  lazega_fr
Nodes:                 =  71
Ties:                  =  575
Directed:              =  Yes
Estimation:            =  MCMLE
MCMC sample size:      =  525

-------------------------------------------------------------------------------
     lazega_fr | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
---------------+---------------------------------------------------------------
         edges |  -3.109937   .2456087   -12.66   0.000    -3.591321   -2.628552
        mutual |   .9958499   .1661045     6.00   0.000      .670291    1.321409
nodematch_prac |   .8724326   .0982727     8.88   0.000     .6798216    1.065044
       twopath |   .0195593   .0171992     1.14   0.255    -.0141504     .053269
        cycle3 |   .0695876   .0893497     0.78   0.436    -.1055345    .2447097
-------------------------------------------------------------------------------

MCMLE converged after 6 iteration(s).
Evaluating log-likelihood at the estimate (bridge sampling with 16 bridges) ...

             Null Deviance:    6889.9  on 4970 degrees of freedom
         Residual Deviance:    3400.2  on 4965 degrees of freedom

Log-likelihood: -1700.1227 (df = 5)   AIC:    3410.2   BIC:    3442.8
Monte Carlo standard error of the log-likelihood:  .10035
```

`coeflabel()` renames a coefficient in the output and in `e(b)`:

```
. nwergm lazega_fr, edges mutual nodematch(prac) coeflabel(nodematch_prac samepractice)

Exponential-family random graph model
```

```
Network:               =  lazega_fr
Nodes:                 =  71
Ties:                  =  575
Directed:              =  Yes
Estimation:            =  MCMLE
MCMC sample size:      =  703

------------------------------------------------------------------------------
   lazega_fr | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
-------------+----------------------------------------------------------------
       edges |  -2.734762   .0805179   -33.96   0.000    -2.892574   -2.576949
      mutual |   .9785789    .156804     6.24   0.000     .6712488    1.285909
samepractice |   .8947619   .0972677     9.20   0.000     .7041207    1.085403
------------------------------------------------------------------------------

MCMLE converged after 3 iteration(s).
Evaluating log-likelihood at the estimate (bridge sampling with 16 bridges) ...

             Null Deviance:    6889.9  on 4970 degrees of freedom
         Residual Deviance:    3403.9  on 4967 degrees of freedom

Log-likelihood: -1701.9427 (df = 3)   AIC:    3409.9   BIC:    3429.4
Monte Carlo standard error of the log-likelihood:  .05836
```

8.9 Network types: a valued network and a two-mode covariate

The type of a network is fixed when it is created (Section 3.1). The edge list of this section is simulated for the manual: 40 nodes and 134 undirected ties, of which about 30% carry the value 2 and the others the value 1. `nwfromedge` with the option `valued` makes a valued network of it:

```
. nwfromedge a b w, undirected name(val) valued

. nwsummarize val
-------------------------------------------------
   Network name:  val
   Network id:  1
   Directed: false
   Valued: true
   Two-mode: false
   Nodes: 40
   Selfloop: false
   Edges: 134
   Minimum value:  0
   Maximum value:  2
   Density:  .172
   Temporal: false

. nwergm val, edges
(note: tie values are ignored; the model is fitted to the binary network)

Exponential-family random graph model

Network:               =  val
Nodes:                 =  40
Ties:                  =  134
Directed:              =  No
Estimation:            =  MPLE

------------------------------------------------------------------------------
         val | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
-------------+----------------------------------------------------------------
```

```
       edges |   -1.57296   .0949246   -16.57   0.000    -1.759008   -1.386911
------------------------------------------------------------------------------

Maximum pseudolikelihood estimate; this is the maximum likelihood estimate only if the model is
    dyad-independent.

             Null Deviance:    1081.3  on 780 degrees of freedom
         Residual Deviance:     715.6  on 779 degrees of freedom

Log-likelihood:  -357.8022 (df = 1)   AIC:     717.6   BIC:     722.3
```

The summary shows the flag `Valued: true` and the maximum value 2. `nwergm` fits the graph in which a tie is present when its value is not zero, and the note in the first line says so; with `edges` alone the coefficient is the log odds of the density, $\log(134/646) = -1.57$, as for the binary network. When every value is 1 nothing is lost, and no note is printed:

```
. use ‘edges’, clear

. nwfromedge a b one, undirected name(val1) valued

. nwergm val1, edges

Exponential-family random graph model

Network:               =  val1
Nodes:                 =  40
Ties:                  =  134
Directed:              =  No
Estimation:            =  MPLE

------------------------------------------------------------------------------
        val1 | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
-------------+----------------------------------------------------------------
       edges |   -1.57296   .0949246   -16.57   0.000    -1.759008   -1.386911
------------------------------------------------------------------------------

Maximum pseudolikelihood estimate; this is the maximum likelihood estimate only if the model is
    dyad-independent.

             Null Deviance:    1081.3  on 780 degrees of freedom
         Residual Deviance:     715.6  on 779 degrees of freedom

Log-likelihood:  -357.8022 (df = 1)   AIC:     717.6   BIC:     722.3
```

A two-mode covariate network can be used in `edgecov()` on a two-mode model. Here the network `visits` is a second two-mode network on the same 30 people and 12 clubs as `clubs`; only the cells between the two modes enter the model:

```
. nwergm clubs, edges edgecov(visits)

Exponential-family random graph model

Network:               =  clubs
Nodes:                 =  42
Ties:                  =  72
Directed:              =  No
Estimation:            =  MPLE

------------------------------------------------------------------------------
         clubs | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
---------------+--------------------------------------------------------------
         edges |  -1.396876   .1629103    -8.57   0.000    -1.716175   -1.077578
edgecov_visits |   .0307848   .2770268     0.11   0.912    -.5121777    .5737473
```

```
------------------------------------------------------------------------------

Maximum pseudolikelihood estimate; this is the maximum likelihood estimate only if the model is
    dyad-independent.

             Null Deviance:     499.1  on 360 degrees of freedom
         Residual Deviance:     360.3  on 358 degrees of freedom

Log-likelihood:  -180.1387 (df = 2)   AIC:     364.3   BIC:     372.0
```

Chapter 9

Validation against ergm

`nwergm` is a reimplementation, and the only reason to trust its numbers is that they agree with the program that it reimplements. This chapter describes how the comparison with `ergm` 4.12.0 was done, shows a comparison that was run for this manual, and lists the places where the two programs differ. The status of every term is in Chapter 5. Appendix B lists the definitions of `ergm` that `nwergm` reproduces on purpose, and Appendix C shows that the two programs run the same estimator and explains where the difference in speed comes from.

9.1 How the comparison was done

The reference is `ergm` 4.12.0 (R 4.6). The comparisons are of four kinds, from the strictest to the loosest. The first three are scripted and rerun by the test suite of the package, 66 do-files in the directory `cscripts`; the generators of the R reference values are in the directory `dev`.

1. **Observed statistics.** For every term, the observed statistic of test networks is computed by `nwergm` and by `summary()` of `ergm`. The test networks are random undirected, directed and two-mode networks with 14 to 100 nodes and the real networks of the statnet examples (the Florentine marriages, Sampson's monastery, the Kapferer tailor shop, Mesa High School). The statistics agree exactly, or to the floating-point precision of the statistic (at most 2×10^{-7}, for the term that `ergm` computes in single precision).
2. **Change statistics and pseudolikelihood.** The change statistics determine the pseudolikelihood. The change statistic of every dyad is compared with the difference of the statistic computed by brute force with and without the tie, and, for the two-mode terms, with the design matrix of `ergmMPLE()`. The pseudolikelihood coefficients and standard errors are compared with `ergm(estimate="MPLE")` on the same networks: the coefficients agree to 10^{-4} or better for every term that was compared, and to 10^{-6} to 10^{-10} for most (the column *Validation* of Chapter 5 gives the figure per term), with the exceptions that Appendix B and Section 9.3 name. Where R's own fit is degenerate or separated (an infinite or missing coefficient, a standard error above 20), the comparison is not possible and the case is skipped.
3. **Monte Carlo maximum likelihood.** For dyad-dependent models, the fit by `nwergm` is repeated with several seeds and compared with several runs of `ergm` on the same network. The criterion is that the difference of the mean coefficients is small compared with the standard error and with the run-to-run spread of both programs: for the models of the structural terms every coefficient was within 0.1 posterior standard errors of R's and inside the spread over seeds; for two-mode terms the differences of the means were between 0.01 and 0.14 standard errors. Constrained fits and fits with missing dyads were compared in the same way.
4. **The sampler itself.** The Markov chains were checked against exact enumeration of the distribution on networks that are small enough (5-node undirected, 4-node directed and 3×3 two-mode networks; with every constraint that restricts the sample space) and against

`simulate()` of `ergm` (means and standard deviations of the statistics of long chains agree; Section 9.2 and Appendix C).

The estimation controls (the Hummel step length, the convergence test, the standard errors) were ported from the algorithms of `ergm` and checked against intermediate results of R: the step-length routine against `ergm`'s function to 10^{-7} (for models with more than 64 statistics the same linear program is solved by a revised simplex, which agrees with the original routine to 10^{-10}), the multivariate spectral variance estimate used by `mcmc.diagnostics()` to 10^{-8}, the effective sample size and the Geweke and Heidelberger–Welch statistics against `coda`.

9.2 A comparison run for this manual

To show what these comparisons look like, the main ones were repeated for this manual on the networks of Chapter 8. The Stata side is the do-file `manual/ex/ex91_compare_stata.do`, the R side `manual/ex/r/compare.R`; both read the same networks and attributes from files.

Observed statistics. Table 9.1 compares the observed statistics of the Lazega network. The statistics of the counts agree exactly; the geometrically weighted statistic, which is a sum of non-integer weights, agrees to 1.5e-08.

Pseudolikelihood. Table 9.2 compares the pseudolikelihood estimates of four models: a dyad-independent model of the Florentine network, a dyad-dependent model (with `mutual`) of the Lazega network, and two models of the simulated two-mode network (one dyad-independent with `nodemix`, one dyad-dependent with two-mode degree and star terms). The largest difference between the coefficients is 4.1e-08; the standard errors agree to a relative difference of at most 1.0e-04.

Monte Carlo maximum likelihood. Table 9.3 compares five runs of each program on the Lazega model of Section 8.3. With its default settings, which are those of `ergm` (burn-in 16384, interval 1024, adaptive sample size, `ergm`'s convergence test), `nwergm` has a spread over seeds of the order of R's (standard deviations of the coefficients over the five seeds between 0.000 and 0.006 for both programs), and the means differ from R's by at most 0.02 of a standard error; the standard errors of the two programs agree. The middle columns repeat the fits with a short chain and fixed-size samples (`mcmleeffectivesize(0) mcmcburnin(3000) mcmcinterval(50)`): the chains are then short relative to the autocorrelation (Section 8.3), the spread over seeds is up to nine times larger (0.053 for `edges`, against 0.006 with the default settings and 0.003 for `ergm`), and single fits are off by a larger part of a standard error. Five seeds of one model are a small sample. The eight benchmark models of Appendix C were also fitted five times each (seeds 1 to 5, default settings) by both programs: over the 27 coefficients the means differ by at most 0.09 of `ergm`'s standard error, and the standard errors of `nwergm` are 0.96 to 1.04 times those of `ergm`.

The sampler. Table 9.4 compares the means and standard deviations of the five model statistics in chains simulated at the coefficients of the Lazega fit, by `nwergm` and by `simulate()` of `ergm`. The means differ by at most 0.05 of a standard deviation and the standard deviations agree to the first digits.

9.3 Where nwergm differs from ergm

The differences that matter to a user are these.

- **Monte Carlo sampling.** Burn-in 16384, interval 1024, a base sample of the larger of 256 and 32 times the number of coefficients, an effective-sample-size target of 64 and the

Table 9.1: Observed statistics of the Lazega friendship network: nwergm (the first row of the model block of `estat gof`) and `summary()` of ergm.

term	nwergm	ergm	difference
`edges`	575	575	0.0e+00
`mutual`	74	74	0.0e+00
`nodematch_prac`	412	412	0.0e+00
`nodeicov_seniority`	19379	19379	0.0e+00
`gwesp_0.5`	555.6191	555.6191	1.5e-08

Table 9.2: Pseudolikelihood estimates: nwergm (`estimate(mple)`) and ergm (`estimate="MPLE"`) on the same data.

fit and term	nwergm	ergm	$\lvert\Delta\rvert$	SE nwergm	SE ergm
Florentine marriages					
`edges`	-2.59493	-2.59493	2.1e-10	0.53606	0.53606
`nodecov_wealth`	0.01055	0.01055	3.8e-12	0.00467	0.00467
Lazega friendship					
`edges`	-2.50763	-2.50763	4.1e-08	0.11080	0.11079
`mutual`	0.99645	0.99645	2.2e-08	0.11050	0.11050
`nodematch_prac`	0.91390	0.91390	1.1e-08	0.09903	0.09902
`nodeicov_seniority`	-0.00691	-0.00691	2.6e-10	0.00224	0.00224
simulated two-mode network, model 1					
`edges`	-0.75633	-0.75633	1.9e-11	0.17873	0.17873
`nodemix_kind_2_1`	-1.51236	-1.51236	2.8e-09	0.39313	0.39312
`nodemix_kind_1_2`	-2.37917	-2.37917	2.4e-08	0.61625	0.61620
`nodemix_kind_2_2`	-0.23407	-0.23407	4.6e-11	0.37073	0.37073
simulated two-mode network, model 2					
`edges`	-0.25048	-0.25048	4.1e-11	0.37254	0.37254
`nodemix_kind_2_1`	-1.60452	-1.60452	9.9e-11	0.42091	0.42091
`nodemix_kind_1_2`	-4.02396	-4.02396	1.1e-09	0.76838	0.76837
`nodemix_kind_2_2`	-1.68223	-1.68223	7.4e-10	0.52610	0.52610
`b2degree_5`	2.64228	2.64228	2.4e-10	0.51729	0.51728
`b2degree_6`	2.75054	2.75054	6.1e-10	0.62388	0.62387
`b1star_2`	-0.13954	-0.13954	1.7e-11	0.13553	0.13553

convergence test with a tolerance region are R's defaults. Both programs stop after at most 60 iterations. The default proposal is `ergm`'s SPDyad, and the effective sample size, the burn-in test and the convergence test are computed as in `ergm` (Appendix C). One difference remains: a cap of 32 times the starting values on the target and the base size, which `ergm` does not have and which matters only in a fit that never converges. The pseudo-random streams differ.

- **Curved fits on boundary solutions.** Where the data do not identify the decay of a curved term, both programs end at a boundary, and the end points can differ: in a comparison with random networks, the fits of `gwespfree` (directed, OTP), `gwidegreefree`, `gwdspfree` and `gwnspfree` ended at a decay of about 10^{-12} in R and at another small value, 10^{-6} or less, in `nwergm`. Where the data identify the curve the fits agree to 10^{-8} to 10^{-6}.

- **`gwdegree` on a directed network.** R refuses it; `nwergm` accepts it, with no counterpart in R to check the result against.

- **Controls not available.** `MCMLE.metric` (the log-normal approximation is the only one), `MCMLE.dampening`, `MCMLE.sequential`, `MCMLE.density.guard`, `parallel`, `monitor=`, `response=`, `esteq=`, and the constraint `constraints=`-arguments `ChangeStats()`, `Dyads()`

Table 9.3: Monte Carlo maximum likelihood, Lazega friendship network, five seeds each: mean and (in parentheses) standard deviation over the seeds of the coefficients, and mean standard error. nwergm with its default settings (adaptive sample), nwergm with a short fixed chain (fixed-size samples, `mcmleeffectivesize(0) mcmcburnin(3000) mcmcinterval(50)`: 3000 draws per iteration), and ergm with its default settings.

	nwergm, default		nwergm, short fixed chain		ergm	
term	mean (sd)	SE	mean (sd)	SE	mean (sd)	SE
`edges`	-3.312 (0.006)	0.131	-3.330 (0.053)	0.148	-3.315 (0.003)	0.136
`mutual`	0.806 (0.003)	0.161	0.804 (0.036)	0.161	0.804 (0.006)	0.161
`nodematch_prac`	0.715 (0.004)	0.081	0.710 (0.014)	0.085	0.715 (0.004)	0.081
`nodeicov_seniority`	-0.005 (0.000)	0.002	-0.004 (0.001)	0.002	-0.005 (0.000)	0.002
`gwesp_0.5`	0.488 (0.003)	0.063	0.495 (0.015)	0.067	0.488 (0.002)	0.065

Table 9.4: Simulation at given coefficients (the coefficients of the Lazega fit): mean and standard deviation of the model statistics over three chains of 4000 draws, nwergm (`nwergm simulate` at the fitted coefficients with `burnin(20000) interval(1000)`) and ergm (`simulate()` with its default burn-in 16384 and interval 1024).

term	mean nwergm	mean ergm	sd nwergm	sd ergm
`edges`	575.0	576.4	40.3	39.6
`mutual`	73.9	74.4	10.6	10.5
`nodematch_prac`	411.7	412.3	30.7	30.2
`nodeicov_seniority`	19389.1	19465.8	1488.2	1473.9
`gwesp_0.5`	556.3	558.8	75.6	74.5

and `I()`.

- **Proposals not available.** The Hamming proposals (reachable only through the broken `hamming` constraint), shared-partner proposals of other types than R's default (OTP on directed networks), `TNT10` (which R marks as flawed), the bipartite degree-distribution proposal (whose code writes the second toggle into the wrong slot), the degree-mixing proposal, and the stratified proposal of the `strat` hint. The bounded-degree proposal is realized as the tie/no-tie proposal with a rejection of moves that violate the bounds, which has the same target distribution.

- **Unbounded loops.** The degree-preserving proposals of R can loop forever on a state without a valid tetrad (for example on 4-node directed networks); `nwergm` bounds the search at 10^6 tries and treats the step as a null move.

- **Terms.** The arguments listed in the remarks of Chapter 5 that are not available, in particular `levels=` for the terms of `structural()`; the form of `S()` with two sets of nodes; and the `Curve()` operator beyond linear maps.

- **Missing dyads and constraints together.** Missing dyads cannot be combined with `freedyads()`, `blockdiag()` or `fixdensity`; an observation constraint in general is available through `obsconstraints()`.

9.4 How to reproduce

The files named here are in the GitHub repository of `nwergm` (https://github.com/thomas-u-grund/nwergm). The comparison of Section 9.2 is rerun by `make compare` in its folder `manual` (Stata, and R with `ergm` 4.12.0; about ten minutes): Stata writes the networks and its

estimates to `manual/ex/r`, and the R scripts there read them and write their own. The tests of the package, with the reference values of `ergm` built in, are run by `cscripts/run_tests.sh`; the R scripts that generated the reference values are in `dev`. The benchmark and the checks of Appendix C are in the folder `replication`, the replication files of the article on `nwergm`.

Chapter 10

Limits, known issues and messages

This chapter says what `nwergm` cannot do and which problems are known, and explains the messages that a user is likely to meet. The terms that are not implemented are listed in Chapter 5, and the places where `nwergm` differs from `ergm` in Section 9.3. The size limits and the run times are in Chapter 11.

10.1 Dialog boxes

The three dialogs are generated from one specification and checked automatically: a script builds the command that the dialog would build for every term and option and runs it in Stata. They were not tested by hand on Windows or Linux, and the screen layout was not tested on other platforms than macOS. A dialog builds a single command line; model specifications that are easier to write by hand (long operator formulas, `For()` loops) are better typed in a do-file.

10.2 Known issues

- **Valued networks are fitted as binary graphs.** A network flagged valued is fitted to the graph in which a tie is present when its value is not zero, with one note; the values are not an outcome of the model and there is no valued ERGM (Section 3.1, Section 8.9). Temporal networks are refused.
- **Reading results of other versions.** Results of fits made with other versions of `nwergm` may differ; report the version printed by `nwergm version`.
- **Degenerate and poorly mixing models.** A model that the data do not support makes the Markov chain degenerate. The fit then stops with the error of `ergm` (Section 8.4) or, worse, converges although the chain mixes poorly (Section 8.3). Once the chain does not mix, the adaptive sample is extended and thinned in every iteration (at most 16 runs), a failed convergence test raises the target and the base size (up to 32 times their starting values), and every iteration costs a multiple of the first; the default 60 iterations then take many minutes. Lower `mcmleiterations()` for a model that you suspect to be degenerate, and always look at `estat mcmcdiag`.
- **The Mata sampler can be extremely slow.** A fit, `estat gof` or `nwergm simulate` that runs on the Mata sampler instead of the plugin prints one warning line with the reason (Section 11.2). For the Lazega model with the default settings one fit took 1,158 seconds against a median of 1.9 seconds with the plugin (Table 11.2). `nonative` is for testing and comparison.
- **Dyad-dependent constraints are slow.** A fit of `edges mutual nodematch(prac)` to the 71-node Lazega network with `constraints(bd(maxout=30))` took 3.7 seconds with the

default settings, against 0.3 seconds without the constraint (the bound does not bind: the largest out-degree is 22; both fits needed three iterations; `manual/ex/ex9d_bdtime.do`). Dyad-dependent constraints change the proposal and the starting values, and the time of such a fit is not predictable from that of the unconstrained model.

- **Standard errors of a pseudolikelihood fit of a dyad-dependent model** are not valid (Section 2.2.1); `mplecovariance(godambe)` or `mplecovariance(bootstrap)` give covariances for the pseudolikelihood estimate.
- **No likelihood for a curved Monte Carlo fit.** `e(ll)`, AIC and BIC are missing after a fit with a free decay by Monte Carlo maximum likelihood (the pseudolikelihood fit has them), and `estat ic` refuses.
- **Missing dyads and constraints.** Missing dyads cannot be combined with `freedyads()`, `blockdiag()` or `fixdensity`.
- **Collinear terms.** A set of terms that is exactly collinear (for example all levels of `nodefactor()` with `base(0)` together with `edges`) is reported with a coefficient of 0 and no standard error; a statistic that never varies in the sample produces the warning that the model may be nonidentifiable.
- **Operators.** Operators cannot be combined with constraints, and `S()` and `Project()` do not accept dyadic covariates (Section 6.2).

10.3 Messages and what to do

The messages that the examples of this manual produced, and the usual remedy.

Message (return code)	Meaning and remedy
`option edges is required` (198)	Every model has the term `edges`; `structural(density)` or `structural(meandeg)` can stand in for it.
`option base() not allowed` (198)	A suboption was typed outside the parentheses of its term. Write `nodefactor(x, base(0))`, not `nodefactor(x) base(0)`.
`In term nodecov: attribute wealth has missing data, which is not currently supported.` (416)	A node attribute that a term uses has a missing value, as in `ergm`. Drop or impute it (Section 2.4).
`Unconstrained MCMC sampling did not mix at all. Optimization cannot continue.` (498)	The statistics of the sample do not vary: the model is degenerate or the starting values are far from any fit. Simplify the model, replace a count term by its geometrically weighted version, or give starting values with `init()` (Section 8.4).
`Warning: MCMLE iter 1 - linear dependence detected among the model statistics`	Some statistics are collinear in the sample, for example a term that is constant under a constraint or is a function of other terms. The model may be nonidentifiable; remove the term.
`Note: the constraints hold edges constant; they are dropped from the model.`	As in `ergm`: a statistic that the constraint fixes is shown as omitted (Section 2.5).
`Note: method(mple) requested for a dyad-dependent model`	The pseudolikelihood is not the likelihood and its standard errors are not valid (Section 2.2.1).
`The MPLE does not exist!` (warning)	The data are separated for this model. Use `mpletype(penalized)` or drop the term (Section 2.2.1).
`Note: network ... has 3 missing dyad(s)`	Missing dyads are excluded from the likelihood and handled as in `ergm` (Section 2.4).
`estat mcmcdiag is only available after method(mcmle)` (498)	There is no Markov chain after a pseudolikelihood fit; also not available after `nomcmcsample`.
`... need the native plugin` (198)	The option exists only in the plugin, and no plugin is available for the model or the platform, or `nonative` was given (Section 11.2).

Message (return code)	Meaning and remedy
`The installed core nwcommands and nwergm were built for different versions` (3021 or 3260)	The Mata libraries of the core and of `nwergm` do not match. Reinstall the models with `nwinstall, models` or all packages with `nwinstall, everything` (Section 11.2).

Chapter 11

Technical notes

This chapter is for users who fit large models, who run `nwergm` on several platforms, or who want to read or extend the source. It describes the three layers of the program, the native plugin and how to tell which engine ran, reproducibility, the size limits of Stata and of `nwergm`, the run times to expect, and the layout of the source files.

11.1 The three layers

`nwergm` consists of three layers.

1. The **ado layer** (`nwergm.ado`, `nwergm_estat.ado`, `nwergm_p.ado` and the helper files `_nwergm_*.ado`) parses the terms and options, checks the network and the combinations of options, builds the model, calls the estimator and posts the results in `e()`.
2. The **Mata core** (`unw_ergm.do`, compiled into the library `lnwergm.mlib`) holds the model and the network, the statistics and change statistics of every term, the pseudolikelihood fit, the Monte Carlo estimator with its adaptive sample, convergence test and step length, the covariance, the bridge sampling of the log-likelihood, the diagnostics, and a reference sampler written in Mata.
3. The **native plugin** (`native/ergm_mcmc.c`, compiled for each platform into `lib/plugins/<platform>/ergm_mcmc_<platform>.plugin`) runs the Metropolis–Hastings sampler with the change statistics, the observed statistics of large networks, the design matrix of the pseudolikelihood, the constraints and the operators. Almost all of the computing time of a Monte Carlo fit is spent in it.

The Mata layer calls the plugin once per sample and not once per proposal: a call passes the model, the network and the coefficients, runs the burn-in and all draws of the sample, and returns the statistics of the draws. The estimating step, the covariance, the convergence test and the output stay in Mata and ado.

11.2 The native plugin

11.2.1 Which engine ran

After a fit, `e(native)` is 1 if the plugin was used (the sampler of a Monte Carlo fit, the design matrix of a pseudolikelihood fit) and 0 if Mata was. `nwergm` prints one warning line before a Monte Carlo fit, `estat gof` or `nwergm simulate` that runs on the Mata sampler instead of the plugin, with the reason: the plugin file is missing for the platform, `nonative` was given, or the model is outside the capacity of the plugin. No warning is printed when the plugin is used. The Mata sampler runs the same algorithm and gives statistically equivalent results (the random streams differ, so that the digits differ), but it is slower by orders of magnitude (Section 11.5). `nonative` is for testing and comparison.

11.2.2 Platforms

The native plugin is built for macOS (universal binary, compiled on the development machine), Linux and Windows (compiled by continuous integration). All the output of this manual comes from the macOS build with Stata/BE 18. The full test suite also passes on Windows with Stata/MP 19.5 and the Windows plugin; the Linux plugin has not been tested on a Linux machine, and Stata/SE was not tested. The plugin reports its version, and a plugin that is older than the `.ado` files is detected. A platform without a plugin runs the Mata sampler, which is much slower for the models that have one (Table 11.2) and does not exist for `structural()`, `operators()`, `constraints()`, `obsconstraints()` and `mcmcmaxedges()`.

Two practical traps. A plugin in Stata's PLUS directory from an older installation shadows the plugin of a repository on the ado-path, so that a freshly built plugin can silently not be used; test it through a scratch ado-path. On macOS, overwriting a plugin that has already been loaded kills Stata without a message (the code signature is cached by the system); remove the old file first, then copy.

11.2.3 Source code of the plugin

The C source of the plugin is part of the package. `net install` installs the compiled plugins only; the command

```
. net get nwergm, from(https://raw.githubusercontent.com/thomas-u-grund/nwergm/main)
```

copies the source files, the build rules and a short build note into the current directory. The same files are in the folder `native/` of the repository (https://github.com/thomas-u-grund/nwergm). `nwergm` is free software under the GNU General Public License, version 3; the plugin interface files in `native/spi/` are StataCorp's Stata Plugin Interface, redistributed unchanged.

11.2.4 Building the plugin on your computer

If the plugin of your platform is missing or does not work on your computer, compile it there with

```
. nwinstall, buildplugin(nwergm)
```

The command copies the source from the repository, compiles it with the compiler of your computer (a C compiler and `make` are needed: on macOS the command line tools of Xcode, on Linux `gcc` and `make`), and installs the plugin next to the one that `net install` put into the PLUS directory; an existing file is kept with the extension `.prev`. Restart Stata afterwards. The plugin depends on the operating system and the processor, not on the version or the flavour of Stata. The routine was tested on macOS; on Windows it stops with a message and the build note that comes with the source gives the command for the Visual Studio command line.

11.2.5 Fixed limits of the plugin

The plugin has fixed capacities: 8192 coefficients (columns of the model; a model with more than 256 receives its term table and returns its sample in frame variables), 64 distinct node attributes and 8 distinct dyadic covariate matrices in one model. The dyadic covariates are passed as their nonzero entries, so that their size is not limited by the number of variables of Stata/BE. `nwergm` checks that the plugin found matches the version of the package. A model that exceeds them is not refused; it runs in the Mata sampler as a whole, never partly in C and partly in Mata, and is much slower (and `structural()`, `operators()` and the constraints, which exist only in the plugin, are refused).

11.3 Reproducibility

The plugin runs on one thread. A Monte Carlo fit, `estat gof` and `nwergm simulate` draw their random numbers from a stream set by `seed()`, so that the same `seed()`, model, network and version give identical results; the test suite checks this for fits and for simulation. The plugin and the Mata sampler use the random numbers differently, so that their draws differ. Results change if the model, the seed, the options or the version of the program change; report the version printed by `nwergm version`.

11.4 Size limits

11.4.1 What costs what

The pseudolikelihood is built over *all* dyads, $n(n-1)/2$ in an undirected network and $n(n-1)$ in a directed one, so that time and memory of a pseudolikelihood fit grow with the square of the number of nodes, whatever the number of ties. The Monte Carlo sampler works on the ties and their neighborhoods and its cost per proposal depends on the degrees, not on n^2, but every Monte Carlo fit starts from a pseudolikelihood fit (unless all coefficients are given in `init()`) and so pays the cost of the dyad-level fit once, and the log-likelihood evaluation by bridge sampling needs further chains. A network is held in a sparse structure; dense $n \times n$ objects exist only for dyadic covariates (`edgecov()`, `hamming()`, `dyadcov()`) and for `predict, matrix()`.

11.4.2 What Stata limits

Stata/BE has a maximum matrix dimension of 800 (`c(max_matdim)`) and at most 2048 variables. A network with more than 800 nodes therefore cannot be declared from a Stata matrix with `nwset, mat()`; build it from an edge list (`nwfromedge`), as the networks of Table 11.1 were. The same holds for `predict, matrix()`; `predict, frame()` has one row per dyad and is limited by memory. Coefficient vectors wider than 800 (a model with one coefficient per node on more than about 400 nodes) are posted and read through Mata, so that `e(b)` and `e(V)` and the postestimation commands work. The matrix `e(mcmcsample)` has one row per draw; filling it from Mata is the slowest step for a very large `mcmcsamplesize()` (more than 30 seconds at 100,000 rows in a test of the development of the package), and `nomcmcsample` skips it. Other editions of Stata have larger limits; they were not tested.

11.4.3 Models with one coefficient per node

`sender` and `receiver`, and factor terms on an attribute with many levels, give a model with hundreds or thousands of coefficients. A dyad-independent model of this kind is fitted by the pseudolikelihood, which is then the likelihood, with a sparse design: `edges sender receiver` on a random directed network of 400 nodes (799 coefficients) took 1.0 second, against 573 seconds for `ergm`, with estimates that agree to 1.4×10^{-6}, and about 8 seconds on 1000 nodes (1999 coefficients, Stata/BE). A dyad-dependent model needs Monte Carlo maximum likelihood. As in `ergm`, every iteration draws at least 32 networks per coefficient, so that the work of an iteration grows with the square of the number of coefficients, and such fits take much longer than fits of the same network without the per-node terms. For larger networks, replace the per-node terms by node covariates or degree terms, or use the pseudolikelihood, which for a dyad-dependent model is only an approximation with too small standard errors.

11.4.4 Not measured

Not measured: networks above 10,000 nodes, networks with mean degree above 6, models with several dense dyadic covariates, memory use as a function of n, and fits of a directed network of

the sizes in the table.

11.5 Performance

The following run times were measured on one machine with the native plugin and are indications, not benchmarks; Appendix C compares the run times with `ergm`'s and explains where the difference comes from.

Table 11.1 gives the run time of `nwergm` on random undirected networks with mean degree 6 and three random groups, with the default settings and the native plugin, one fit per row (so the figures are indications, not benchmarks). The networks were generated by `manual/ex/ex98_scaling.do` and `ex99_scaling2.do`; all rows were measured with `nwergm` 1.5.2 on 9 November 2026 on the machine of Appendix C (one-minute load average 2.6 to 2.9 at the start of the runs and 7.5 at the end). The dyad-independent model has the terms `edges`, `nodematch(g)` and `nodecov(x)`; the dyad-dependent model adds `gwesp(0.5)` (Monte Carlo maximum likelihood with the default settings, with bridge sampling for the log-likelihood). The times of the pseudolikelihood column do not depend on the Markov chain; they vary between runs by tens of percent on a machine that is also used for other jobs, and a single fit is a single measurement.

Table 11.1: Run time in seconds of `nwergm` on random networks of n nodes (mean degree 6).

n	dyads	pseudolikelihood (3 terms)	Monte Carlo (4 terms with `gwesp`)
200	19,900	0.1	0.3 (2 iterations)
500	124,750	0.2	0.5 (3 iterations)
1000	499,500	0.6	1.9 (4 iterations)
2000	1,999,000	3.0	6.3 (4 iterations)
4000	7,998,000	13.3	not run
5000	12,497,500	not run	38.5 (3 iterations)
8000	31,996,000	60.2	not run
10000	49,995,000	not run	150.8 (2 iterations)

A random network has no clustering, so that the shared-partner statistics are small and the sampler is fast; a network with high degrees and heavy clustering is costlier per proposal and mixes more slowly, and the time of a Monte Carlo fit is dominated by the number of iterations and by the adaptive growth of the interval. These figures are *not* a statement about such networks. For the Lazega network of Chapter 8 (71 nodes, 575 ties, five terms) Table 11.2 gives the run times of five seeds each: the default, the fixed-size algorithm (`mcmleeffectivesize(0)`, 3000 draws per iteration), the fixed-size algorithm with a short chain (burn-in 3000, interval 50), the Mata sampler (one fit) and `ergm`.

Table 11.2: Run time in seconds of the Lazega model (five seeds). nwergm with default settings, with fixed-size samples and with fixed-size samples and a short chain (native plugin), the Mata sampler with default settings (one run), and R's `ergm()` with its defaults, on the machine of Appendix C (9 November 2026, `nwergm` 1.5.2).

setting	runs	median	range	iterations
nwergm, default settings	5	1.9	1.5–2.6	5–7
nwergm, fixed 3000 draws (`mcmleeffectivesize(0)`)	5	1.8	1.8–1.8	1–1
nwergm, fixed 3000 draws, burn-in 3000, interval 50	5	0.3	0.3–0.3	1–1
nwergm, Mata sampler (`nonative`, default settings)	1	1158.4	1158.4–1158.4	12–12
R `ergm()`, default settings	5	9.3	4.8–14.2	3–8

On the Lazega model of this table `nwergm` took a median of 1.9 seconds (5 to 7 iterations) and `ergm` 9.3 seconds (3 to 8 iterations), measured one after the other; the same model fitted with fixed 3000 draws took 1.8 seconds in one iteration for every seed. The Mata sampler (`nonative`) is the slow fallback: 1,158 seconds (12 iterations) for one fit. A single model is not a comparison of the two programs; Appendix C compares them on eight models.

Postestimation and the other algorithms. Single runs with 1.5.2 on the same machine (9 November 2026, `manual/ex/ex9f_posttime.do`; one run each, so the figures are indications): `estat gof, gof(model distance odegree idegree espartners)` after the Lazega model of Table 11.2 took 1.4 seconds and `nwergm simulate, nsim(100) output(stats)` 0.1 seconds; `estat gof` with its default statistics after `edges nodematch(grade racen) gwesp(.25)` on Faux Mesa High took 1.1 seconds. On a random undirected network of 1000 nodes (mean degree 6) with `edges gwesp(.5) nodefactor(g) nodecov(x)`, the fit by Monte Carlo maximum likelihood took 2.4 seconds, `estat gof` 26 seconds, `nwergm simulate, nsim(100) output(stats)` 0.15 seconds and the fit by stochastic approximation (`mainmethod(sa)`) 16 seconds. These steps use the triad census by the algorithm of Batagelj and Mrvar, geodesic distances by breadth-first search over neighbour lists, observed statistics of large networks computed in the plugin, and chains that continue from the statistics of the previous draw instead of recomputing them.

Some hints. Shorten exploratory runs with `mcmleeffectivesize(0)` and a short chain, and use the defaults for the final fit, because the convergence test and the standard errors come from the adaptive sample. Look at `estat mcmcdiag` after every Monte Carlo fit. Models that the data do not support take more iterations, not fewer (Section 10.2).

11.6 File layout

File	Content
`nwergm.ado`	The command: parsing, checks, model, estimation dispatch, results.
`_nwergm_*.ado`	Helpers: the two-mode, structural, operator, constraint and covariate terms, missing dyads, the estimation methods, the goodness of fit and its graphs, the warnings.
`nwergm_estat.ado`, `nwergm_p.ado`	`estat` (`gof`, `mcmcdiag`, `ic`) and `predict`.
`nwergm_version.ado`	`nwergm version`: the versions of the package, the core and the plugin.
`unw_ergm.do`	The Mata core: model, terms, estimators, diagnostics, reference sampler, interface to the plugin.
`lib/lnwergm.mlib`	The compiled Mata library, built by `lib/build.do`.
`native/ergm_mcmc.c`	The C plugin source; `native/Makefile` builds it.
`lib/plugins/<platform>/`	The compiled plugins for `macos`, `unix` and `windows`.
`nwergm.sthlp`, `nwergm_*.sthlp`	The help file and its parts (estimation, terms, constraints, operators, postestimation).
`cscripts/test_nwergm_*.do`	The regression tests, one file per feature, and `cscripts/run_tests.sh`, which runs them.
`dev/`	The R scripts that generated the reference values of the tests.
`manual/`	The source of this manual; `manual/ex` holds the do-files of its examples.

Testing. The tests run against the Mata sources loaded directly (`do unw_core.do`, `do unw_ergm.do`) and the plugin of the checkout, with an empty PLUS directory, so that no installed copy shadows them. They compare the statistics, change statistics and pseudolikelihood fits with reference values of `ergm`, the sampler with exact enumeration, and the Monte Carlo fits with runs of `ergm` (Chapter 9).

Appendix A

From R's ergm to nwergm

The table translates the functions and arguments of R's ergm (and statnet) to nwergm. The right column is the Stata command or option; net stands for a network that was declared with nwset and is named in the command.

R (ergm, statnet)	nwergm
net <- network(...)	The network is declared with nwset (or loaded with nwuse, nwwebuse, or built with nwfromedge) and named in the command: nwergm net, ... Node attributes are numeric variables of the current dataset, one row per node.
ergm(net ~ edges + nodematch("sex"))	nwergm net, edges nodematch(sex)
nodematch("sex", diff=TRUE)	nodematchdiff(sex) or nodematch(sex, diff)
nodefactor("x", levels=-1)	nodefactor(x, levels(-1))
absdiff("age"), absdiff("age", pow=2)	absdist(age), absdist(age, pow(2))
b1degree(0:2), gwb2degree(0.5, fixed=TRUE)	bdegree1(0/2), bgwdegree2(0.5)
gwesp(0.5, fixed=TRUE), gwesp(0.5, fixed=FALSE)	gwesp(0.5), gwespfree(0.5)
gwesp(0.5, fixed=TRUE, type="ITP")	gwesp(0.5) type(ITP) (applies to every shared-partner term of the model)
twopath, cycle(3:4), altkstar(0.8, fixed=TRUE)	structural(twopath cycle(k=3 4) altkstar(lambda=0.8 fixed))
Symmetrize(~triangle, rule="strong")	operators(Symmetrize(triangle, rule(strong)))
Sum(~edges | ~triangle, label="a")	operators(Sum(edges | triangle, label(a)))
offset(edges) with offset.coef	offset(edges #)
Label(~nodematch("sex"), "same")	coeflabel(nodematch_sex same) or operators(Label(...))
ergm(..., estimate="MPLE")	estimate(mple)
ergm(..., estimate="CD")	estimate(cd)
ergm(..., main.method= "Stochastic-Approximation")	mainmethod(sa)
control.ergm(MCMLE.burnin=, MCMLE.interval=)	mcmcburnin(), mcmcinterval()
control.ergm(MCMLE.samplesize.min=, MCMLE.samplesize.per_theta=) (base size of the adaptive sample, MCMLE.samplesize=NULL)	mcmcsamplesize() (default the larger of 256 and 32 times the number of coefficients)
control.ergm(MCMLE.maxit=, MCMLE.termination=, MCMLE.confidence=)	mcmleiterations(), mcmletermination(), mcmleconfidence()
control.ergm(MCMLE.steplength=, MCMLE.steplength.margin=)	mcmlesteplength(), mcmlestepmargin()
control.ergm(MCMLE.effectiveSize=64), MCMLE.samplesize=3000 with MCMLE.effectiveSize=NULL (fixed-size samples)	mcmleeffectivesize(64), mcmleeffectivesize(0) mcmcsamplesize(3000)
control.ergm(MPLE.type=, MPLE.check=, MPLE.covariance.method=)	mpletype(), mplecheck(), mplecovariance()
control.ergm(init=)	init(numlist)

R (ergm, statnet)	nwergm
control.ergm(CD.nsteps=), SA.burnin ...	cdnsteps(), saburnin(), sainterval(), saphase3(), sagain(), sansub()
control.ergm(MCMC.prop.weights=, MCMC.prop=, MCMC.maxedges=)	proposal(), mcmcprop(), mcmcmaxedges()
control.ergm(seed=) or set.seed()	seed()
control.ergm(bridge.nsteps=), eval.loglik=FALSE	bridgesteps(), nologlik
constraints = ~edges, ~bd(maxout=3)	constraints(edges), constraints(bd(maxout=3))
constraints = ~fixallbut(free), ~blockdiag("g")	constraints(fixallbut(free)) or freedyads(free); constraints(blockdiag(g)) or blockdiag(g)
obs.constraints = ~...	obsconstraints(...)
coef(fit), vcov(fit)	e(b), e(V); matrix list e(b)
summary(fit)	The table printed by nwergm; ereturn list
summary(net ~ terms) (observed statistics)	The column *obs* of estat gof, gof(model); e(obs_stats) after a pseudolikelihood fit
simulate(fit, nsim=100)	nwergm simulate, nsim(100)
simulate(net ~ terms, coef=, nsim=)	nwergm simulate *nodes*, edges *terms* theta(...) nsim()
simulate(..., output="stats")	nwergm simulate, output(stats); the result is r(stats)
simulate(..., basis=), sequential=FALSE	basis(), nosequential
gof(fit), gof(fit, GOF=~degree + espartners)	estat gof, estat gof, gof(degree espartners)
plot(gof(fit))	estat gof, plot
mcmc.diagnostics(fit)	estat mcmcdiag, report (with plot for trace and density plots)
logLik(fit), AIC(fit), BIC(fit)	estat ic; e(ll), e(aic), e(bic); estimates stats
anova(fit1, fit2)	lrtest
predict(fit)	predict, frame(name) pr or predict, matrix(name) pr
packageVersion("ergm")	nwergm version

Where the two differ in what they do

- The default of gwesp() (and of the other geometrically weighted terms) in ergm is the curved form (fixed=FALSE); in nwergm the option with a number has a fixed decay and the curved form is the option ending in free.
- The sampler defaults are ergm's (Section 2.2.2); both programs choose the sample sizes and intervals adaptively (Section 2.2.2); the differences that remain are listed in Section 4.3.3.
- A term with a digit followed by a letter in its name has the digit moved to the end (b1degree → bdegree1, gwb1dsp → bgwdsp1).
- nwergm writes the period of R's coefficient names as an underscore (nodecov.wealth → nodecov_wealth).
- The order of the coefficients follows nwergm's processing of the options, not the order of the formula.
- ergm accepts character attributes and network objects with vertex and edge attributes; nwergm reads numeric variables and networks in memory.

Appendix B

`ergm` definitions reproduced on purpose

`nwergm` is built to give the same numbers as `ergm`. Where a definition or a computation of `ergm` 4.12.0 has a feature that a reader might not expect, `nwergm` reproduces it on purpose, because a different definition would give statistics and estimates that no longer match `ergm`'s. Where `ergm` is plainly wrong, `nwergm` computes the correct value and says so. This appendix lists the cases that matter to a user who compares the output of the two programs, so that a difference from the theory in the literature is not mistaken for a mistake in `nwergm`. Each was found by comparing statistics or estimates with those of `ergm` (Chapter 9).

- **`triangle` on a directed network.** In `ergm`'s change statistic a node that is both an in- and an out-neighbor of the head is visited twice and the change adds the ties in both directions; `nwergm` copies this (statistic and pseudolikelihood agree with R).
- **Single precision.** `tripercent` computes its ratios in single precision in `ergm`, and its observed statistic is the sum of the single-step changes along a build-up of the network; `nwergm` copies both, and agrees with R's statistic to 2×10^{-7}.
- **Scaling by the observed network.** `degcrossprod` and `degcor` are scaled by quantities of the observed network at the moment of model creation (twice the number of ties, the variance of the observed degrees), and for `degcor` the summary statistic is the true degree correlation whereas the change statistic is an approximation. `nwergm` copies this; the terms are therefore not available in `nwergm simulate`, where there is no observed network.
- **`Prod()`.** `ergm` computes the product as `Exp(Sum(Log()))` with $\log 0 = -1/\sqrt{\epsilon}$. Built up tie by tie from the empty network, the statistic passes through values of about 7×10^7 and loses eight digits: for Mesa High School R gives 522.48718785 for a product whose exact value is $(3491 \cdot 659 \cdot 62)^{1/3} = 522.48719458$, the value that `nwergm` computes. The tests of `Prod()` therefore use the tolerance 10^{-6} for the statistics; the pseudolikelihood coefficients agree to 10^{-5}.
- **`Symmetrize(rule="weak")`.** `summary()` of a formula with this operator is right, because `ergm` adds the arcs one by one. But `ergm()` and `ergmMPLE()` start from the observed network and fill the auxiliary network with an initialization rule that, for `weak`, toggles an arc unless its reverse exists, so that every reciprocated dyad is missing from the symmetrized network on which the pseudolikelihood change statistics are computed (108 of 182 change statistics differed from brute-force differences of `summary()` on a 14-node directed network; the rules `strong`, `upper` and `lower` agree). `nwergm` computes the exact change statistics, so that its pseudolikelihood equals R's only on a network without reciprocated ties. The test therefore compares `rule(weak)` on networks with reciprocated ties with a pseudolikelihood

fitted to R's own `summary()` differences, and on a network without reciprocated ties with `ergm(estimate="MPLE")` itself. This is not to be "fixed" to reproduce R's initial network.

- **`gwesp` and `esp` with `type="RTP"`.** `ergm` 4.12.0 returns wrong values for the reciprocated two-path definition by default (a defect that is corrected in the development version); `nwergm` agrees with R when R is run with `term.options=list(cache.sp=FALSE)`, to 10^{-7}.
- **The penalized pseudolikelihood.** In `ergm` 4.12.0 `MPLE.type="penalized"` stops with an error under R 4.3 or later (a logical of length 3 in a condition); the development version fixes it. `nwergm` reproduces the fixed behavior, which was compared with the development version to 10^{-6} on five networks, one of them separated.
- **`mcmc.diagnostics()`.** The multivariate spectral variance used for the Hotelling omnibus test and the joint Geweke test is that of `ergm` 4.12.0 (a Yule–Walker autoregression); the development version uses an ordinary-least-squares vector autoregression, which changes the second or third digit of these p-values.
- **Constraints.** R refuses `blockdiag` on non-contiguous blocks under the default `noncontig="merge"`, and `nwergm` too. The constraint `hamming` is broken in R and refused; `odegreedist` on a directed network and the degree-distribution constraint on two-mode networks fail in R's released code and are refused. R's `degrees` and `degreedist` proposals for directed networks are not exactly symmetric; `nwergm` reproduces the same chain, which agrees with R's `simulate()` within Monte Carlo error but not exactly with an exact enumeration (by at most 1% on a 4-node directed network).
- **Contrastive divergence.** `ergm` reports the covariance of a contrastive-divergence estimate as missing; `nwergm` stores the covariance of the sample (Section 2.2.4).
- **Separation.** Where a level has no node or is perfectly predicted, R reports `-Inf` or `NA` and `nwergm` reports 0 with a standard error of 0 (for example for `idegree` and `sender`).

Appendix C

Comparison with ergm: the same estimator, and where the speed comes from

`nwergm` fitted eight benchmark models 2.1 to 4.0 times faster than `ergm` 4.12.0, each program with its default settings. A faster program may be suspected of doing less: drawing fewer networks, stopping the estimation earlier, or approximating. This appendix sets out what was measured to rule that out. It answers four questions: whether both programs run the same algorithm, whether they simulate the same model, whether they give the same answers, and where the difference in time comes from.

The scripts, data and results of the comparisons are the replication files of the article on `nwergm`, in the folder `replication` of the repository (https://github.com/thomas-u-grund/nwergm/tree/main/replication); its `README.md` says which script produced which number, and the script names below are relative to that folder. The exactness tests of the sampler are in `cscripts/test_nwergm_spdyad.do`. The timings were made with `nwergm` 1.5.2, which gives the same results as the version described here, on 9 November 2026 on an Apple-silicon Mac with 18 cores, with R 4.6.0 and Stata/BE 18, with no other Stata or R job running. Timings depend on the machine; the ratios between the two programs are more stable than the seconds.

The benchmark models. B1 to B6 are models of simulated networks, stored in `replication/data`: B1, 30 nodes, directed, `edges mutual`; B2, 100 nodes, directed, with `nodematch()` added; B3, 100 nodes, undirected, `edges gwesp()`; B4, 500 nodes, sparse, undirected, with `nodematch()` and `gwesp()`; B5, 1000 nodes, directed, `edges mutual nodematch()`; B6, 1000 nodes, sparse, undirected, `edges gwesp() gwnsp()`. Lazega is the seven-term model of the Lazega advice network (71 nodes) and Mesa the model with `gwesp(.25)` of Faux Mesa High (205 nodes).

C.1 The same algorithm

Both programs fit by Monte Carlo maximum likelihood with `ergm`'s defaults; `nwergm` takes them from `control.ergm()` of `ergm` 4.12.0 (Chapter 2). Here p is the number of coefficients.

setting	`nwergm`	`ergm`
proposal	SPDyad on one-mode networks: a tie/no-tie step with probability 0.75, otherwise a dyad drawn among those with a shared partner; tie/no-tie on two-mode networks	the same (the default `MCMC.prop`)

setting	`nwergm`	`ergm`
burn-in, interval	16384 and 1024	the same (`MCMC.burnin`, `MCMC.interval`)
sample size	a base sample of $\max(256, 32p)$ draws, extended until the effective sample size of the estimating equations reaches 64; each iteration starts from half the interval of the previous one	the same (`MCMLE.samplesize`, `MCMLE.effectiveSize`)
effective size, burn-in test	from the multivariate autoregressive spectral density at frequency zero of the estimating equations; burn-in by `geweke.diag.mv()`	the same (`spectrum0.mvar()`)
convergence	the estimating equations inside a tolerance region (0.1 of their covariance) at confidence 0.99; the sample is enlarged while the test fails	the same (`MCMLE.termination`)
step	Hummel step length	the same
iterations	at most 60	the same (`MCMLE.maxit`)
log-likelihood	bridge sampling with 16 bridges	the same (`bridge.nsteps`)

What is computed differently. One safeguard differs: `nwergm` caps the target and the base size at 32 times their starting values, which `ergm` does not do and which matters only in a fit that never converges. The random numbers differ, so that the estimates of the two programs are not equal digit for digit. On the final sample of a Mesa fit, `ergm`'s `spectrum0.mvar()` and `nwergm`'s port give the same inflation factor (2.329), and so the same effective sample size.

Comparing the traces. With the option `verbose`, `nwergm` prints for every iteration the step length and the coefficients; `ergm` prints the same with `verbose=TRUE`. The numbers of iterations of the two programs vary with the seed in the same range (Table C.3).

C.2 The same simulation

SPDyad is exact. The test enumerates every network of a small model, a 5-node undirected network with `edges` and `triangle` (1024 networks) and a 4-node directed network with `edges` and `transitiveties` (4096 networks), and computes the exact means of the statistics. Chains drawn with SPDyad, by the Mata sampler and by the native sampler, with a share of triadic steps of 0.25 and of 0.75, reproduce these means within 4.5 batch-means standard errors. A wrong Hastings ratio of the triadic steps would show up here, because the triadic steps propose some dyads much more often than others. The same test checks that the cache of shared partners equals a recount after thousands of random toggles, in the Mata sampler and (with a debug build) in the native sampler.

SPDyad mixes as in `ergm`. Table C.2 gives the effective sample size per 1000 draws, the mean over the statistics of each model, of five chains of 1000 draws per program and proposal, at the same coefficients, with interval 1024 and burn-in 16384. The two programs agree with each other for both proposals. SPDyad helps where triadic dependence is strong, most of all on Mesa (74 against 17 with tie/no-tie steps in `nwergm`, 76 against 24 in `ergm`), and costs a little on models with weak triadic terms. The means of the statistics of the two programs agree within Monte Carlo error ($|t|$ mostly below 2), and so do the means and standard deviations of the chains of Table 9.4.

Table C.2: Effective sample size per 1000 draws (mean over the statistics; five chains of 1000 draws, interval 1024).

	SPDyad (default)		tie/no-tie (`mcmcprop(sparse)`)	
Model	`ergm`	`nwergm`	`ergm`	`nwergm`
B1	1008	975	931	978
B2	722	711	810	802
B3	837	879	916	924
B4	197	201	230	224
B5	40	41	53	51
B6	115	112	137	145
Lazega	173	168	168	166
Mesa	76	74	24	17

C.3 The same answers

The eight models were fitted five times each by both programs (seeds 1 to 5, default settings). Over the 27 coefficients the mean estimates differ by at most 0.09 of `ergm`'s standard error, and the standard errors of `nwergm` are 0.96 to 1.04 times those of `ergm`. Table 9.3 shows the same for five fits of the Lazega model, coefficient by coefficient. The speed is not bought with agreement.

C.4 Where the speed comes from

Table C.3 and Figure C.1 give the run times of whole fits: the median of seven fits (seeds 1 to 7) on identical networks, the programs run in turn, one fit at a time, each fit in its own process after one untimed warm-up fit, the time that of the estimation call alone. The one-minute load average before the fits was 1.8 to 3.4 (median 2.6, about one of it the fit itself). The three undirected models were loaded as undirected networks for both programs. `nwergm` was faster on all eight models, by factors of 2.1 to 4.0, and in every pair of fits with the same seed. The times of `ergm` vary more between seeds, because its number of iterations does, so the ratios are those of the medians.

Table C.3: Median run time in seconds of seven fits with default settings. Ratio: `ergm` divided by `nwergm`. Iterations: the range of Monte Carlo iterations over the seven fits, for `ergm` and for `nwergm`.

Model	`ergm`	`nwergm`	ratio `ergm`/`nwergm`	iterations (R; `nwergm`)
B1 30 nodes, directed: edges, mutual	0.19	0.07	2.9	1; 1
B2 100 nodes, directed: + nodematch	0.30	0.14	2.1	1–3; 1–4
B3 100 nodes, undirected: edges, gwesp	0.30	0.09	3.5	1; 1
B4 500 nodes, sparse, undirected: + nodematch, gwesp	2.8	0.68	4.0	3–5; 2–6
B5 1000 nodes, directed: edges, mutual, nodematch	7.8	2.3	3.4	2–3; 1–3
B6 1000 nodes, sparse, undirected: edges, gwesp, gwnsp	6.6	1.7	3.8	2–6; 2–4
Lazega advice, 71 nodes: seven terms	9.7	2.5	3.9	6–8; 6–9
Faux Mesa High, 205 nodes: gwesp(.25)	3.8	1.0	3.7	5–11; 6–10

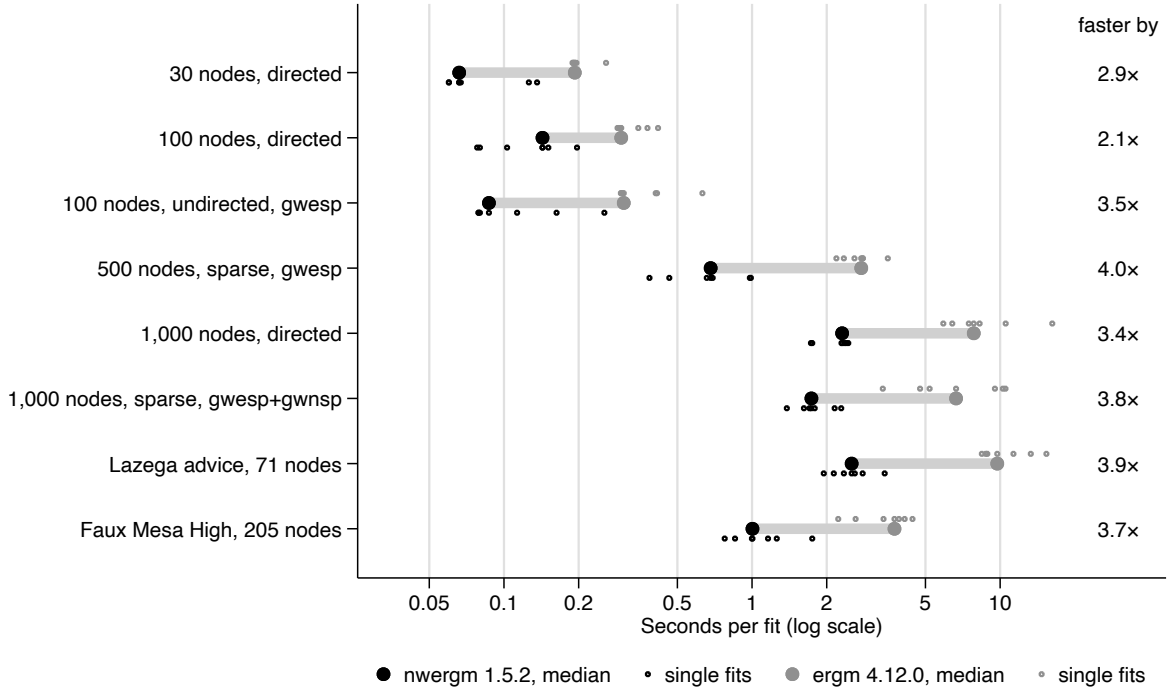


Figure C.1: Time of seven fits of each benchmark model (seeds 1 to 7) with `nwergm` (black) and `ergm` 4.12.0 (grey), default settings, log scale. Small circles are single fits, large circles the medians; the number at the right is the ratio of the medians.

C.4.1 The amount of work

Table C.4 counts the Metropolis–Hastings proposals of complete fits with default settings, including the burn-in and the bridge sampling of the log-likelihood (mean of seeds 1 to 3): in `ergm` by a trace on its sampler, in `nwergm` by a counter before each call of the sampler, in a copy of the package with only the counter added, which gives the same fits. `nwergm` makes 0.7 to 1.5 times as many proposals as `ergm` and keeps final samples of about `ergm`'s size, and the differences follow the numbers of iterations, which vary from seed to seed in both programs. On B5 `nwergm` makes fewer proposals than `ergm`, because its fits ended after fewer iterations with these seeds; on B1, Lazega and Mesa it makes more and is still faster. The number of proposals therefore accounts for little of the speed-up. Dividing the time of these fits by their proposals gives 2.2 to 4.6 times less time per proposal for `nwergm`.

Table C.4: Proposals per fit (millions), Monte Carlo iterations and draws in the final sample; mean of seeds 1 to 3.

	`ergm`			`nwergm`		
Model	proposals	iter.	draws	proposals	iter.	draws
B1	0.53	1.0	243	0.80	1.0	284
B2	0.86	1.3	460	0.58	1.3	327
B3	0.62	1.0	324	0.53	1.0	243
B4	4.61	3.3	601	3.80	3.0	518
B5	18.01	2.7	467	12.50	2.0	430
B6	8.57	4.3	598	6.98	3.0	510
Lazega	9.10	7.0	872	9.88	7.3	750
Mesa	7.77	6.3	1505	8.13	9.3	1460

C.4.2 Time for a fixed amount of work

`R/07_fixed_workload.R` and `stata/07_fixed_workload.do` time the same work in both programs, without the adaptive sample size: one Monte Carlo iteration from the pseudolikelihood estimate with 4000 draws at interval 1024 after a burn-in of 16384, without the log-likelihood. `nwergm` then calls its sampler twice, for the sample of the iteration and for a fresh sample at the final coefficients from which the covariance is computed, and doubles the interval of the second sample while it is strongly autocorrelated; its proposals are counted accordingly. Table C.5 gives the time per proposal.

Table C.5: Microseconds per Metropolis–Hastings proposal with a fixed workload, and the ratio of whole fits from Table C.3 for comparison.

Model	`ergm`	`nwergm`	ratio	ratio of whole fits
B1	0.197	0.080	2.5	2.9
B2	0.199	0.080	2.5	2.1
B3	0.367	0.098	3.7	3.5
B4	0.496	0.132	3.8	4.0
B5	0.392	0.183	2.1	3.4
B6	0.864	0.250	3.5	3.8
Lazega	1.043	0.236	4.4	3.9
Mesa	0.290	0.095	3.1	3.7

With the work held fixed, a proposal of `nwergm` costs 2.1 to 4.4 times less than one of `ergm`. These ratios are of the same size as those of whole fits, so that the speed-up of a fit is the speed of the sampler, not a shorter estimation. The largest gap between the two ratios is on B5, whose fits needed fewer proposals in `nwergm` with these seeds (Table C.4).

C.4.3 The sampler itself

The rest of the speed-up comes from the time a proposal takes in compiled code. Both programs evaluate, for every proposed toggle, the change statistics of every term and accept or reject the toggle. The plugin keeps the adjacency as a bit matrix, updates the change statistics of a proposed toggle from the neighborhoods of its two nodes, tabulates the geometrically weighted kernel once per decay value, draws a random dyad in constant time, and reads the shared-partner counts of `gwesp`, `gwdsp`, `esp` and `dsp` from the cache that SPDyad keeps, as `ergm` does. These are choices about how to compute the same change statistics, as the comparison of the chains in Section C.2 shows. `ergm`'s C code was not instrumented, so how much each of these choices contributes was not measured.

Two parts of a fit outside the sampler are also compiled. The design matrix of the pseudolikelihood, from which every Monte Carlo fit starts, is built in the plugin, and for large networks the pseudolikelihood is computed on the distinct rows of the design matrix with weights: a directed network of 1000 nodes has 999,000 dyads but few distinct rows. The observed statistics of networks of 500 nodes or more are computed in the plugin as well. The estimating step, the covariance, the convergence test, the diagnostics and the output stay in Mata.

The implementation matters more than the algorithm. `nwergm` contains a second implementation of the sampler in Mata, which runs the same algorithm (`nonative`). On the Lazega model it took 1,158 seconds for one fit (12 iterations), against a median of 1.9 seconds for the plugin (Table 11.2): a factor of about 600 from the implementation alone.

C.4.4 Small models

Every call of `nwergm` has a fixed cost of about 0.05 to 0.25 seconds for parsing the command, setting up the model in Mata and returning the results, and the first fit of a session also loads the Mata library. On a model that `ergm` fits in 0.2 seconds most of the difference is this cost; the lowest ratio of a pair of fits, 1.4, was on B1.

C.4.5 What was not measured

`ergm`'s C code was not profiled internally, so the split of the speed-up among the design choices above is an inference from the source code. The timings come from one machine, macOS and Stata/BE 18; Linux and Windows, Stata/SE and Stata/MP, and models outside these eight were not timed.

Bibliography

[1] Hunter, D.R., Handcock, M.S., Butts, C.T., Goodreau, S.M., Morris, M. (2008). ergm: A package to fit, simulate and diagnose exponential-family models for networks. *Journal of Statistical Software* 24(3). https://doi.org/10.18637/jss.v024.i03

[2] Handcock, M.S., Hunter, D.R., Butts, C.T., Goodreau, S.M., Morris, M. (2008). statnet: Software tools for the representation, visualization, analysis and simulation of network data. *Journal of Statistical Software* 24(1). https://doi.org/10.18637/jss.v024.i01

[3] Hunter, D.R., Handcock, M.S. (2006). Inference in curved exponential family models for networks. *Journal of Computational and Graphical Statistics* 15(3), 565–583. https://doi.org/10.1198/106186006X133069

[4] Hunter, D.R., Goodreau, S.M., Handcock, M.S. (2008). Goodness of fit of social network models. *Journal of the American Statistical Association* 103, 248–258. https://doi.org/10.1198/016214507000000446

[5] Handcock, M.S., Gile, K.J. (2010). Modeling social networks from sampled data. *Annals of Applied Statistics* 4(1), 5–25. https://doi.org/10.1214/08-AOAS221

[6] Krivitsky, P.N., Handcock, M.S. (2014). A separable model for dynamic networks. *Journal of the Royal Statistical Society, Series B* 76, 29–46. https://doi.org/10.1111/rssb.12014

[7] Snijders, T.A.B. (2002). Markov chain Monte Carlo estimation of exponential random graph models. *Journal of Social Structure* 3(2), 1–40.

[8] Robins, G., Pattison, P., Kalish, Y., Lusher, D. (2007). An introduction to exponential random graph (p^*) models for social networks. *Social Networks* 29, 173–191. https://doi.org/10.1016/j.socnet.2006.08.002

[9] Wasserman, S., Pattison, P. (1996). Logit models and logistic regressions for social networks: I. An introduction to Markov graphs and p^*. *Psychometrika* 61, 401–425. https://doi.org/10.1007/BF02294547

[10] Frank, O., Strauss, D. (1986). Markov graphs. *Journal of the American Statistical Association* 81, 832–842. https://doi.org/10.1080/01621459.1986.10478342

[11] Hummel, R.M., Hunter, D.R., Handcock, M.S. (2012). Improving simulation-based algorithms for fitting ERGMs. *Journal of Computational and Graphical Statistics* 21(4), 920–939. https://doi.org/10.1080/10618600.2012.679224

[12] Geyer, C.J., Thompson, E.A. (1992). Constrained Monte Carlo maximum likelihood for dependent data. *Journal of the Royal Statistical Society, Series B* 54(3), 657–683. https://doi.org/10.1111/j.2517-6161.1992.tb01443.x

[13] Hunter, D.R. (2007). Curved exponential family models for social networks. *Social Networks* 29(2), 216–230. https://doi.org/10.1016/j.socnet.2006.08.005

[14] Morris, M., Handcock, M.S., Hunter, D.R. (2008). Specification of exponential-family random graph models: terms and computational aspects. *Journal of Statistical Software* 24(4). https://doi.org/10.18637/jss.v024.i04

[15] Krivitsky, P.N. (2012). Exponential-family random graph models for valued networks. *Electronic Journal of Statistics* 6, 1100–1128. https://doi.org/10.1214/12-EJS696

[16] Lazega, E. (2001). *The Collegial Phenomenon: The Social Mechanisms of Cooperation Among Peers in a Corporate Law Partnership*. Oxford: Oxford University Press.

[17] Padgett, J.F., Ansell, C.K. (1993). Robust action and the rise of the Medici, 1400–1434. *American Journal of Sociology* 98, 1259–1319. https://doi.org/10.1086/230190

[18] Kapferer, B. (1972). *Strategy and Transaction in an African Factory: African Workers and Indian Management in a Zambian Town*. Manchester: Manchester University Press.